Содержание

Введение……..……..…...4
1 Структура и общая технологическая схема производства, характеристики основных видов продукции ……...……...…...6
2 Должностные обязанности оператора ЭВМ, техника-программиста, инженера-программиста…….…..8
3Анализ задачи……...……....…….9
3.1 Анализ предметной области……..…….9
3.2 Инструменты разработки ……..……....9
3.3 Диаграмма вариантов использования……..10
3.4 Модель данных……..10
3.5 Постановка задачи……..……...10
4 Проектирование задачи……..……12
5 Реализация ……..……..16
6 Тестирование…….……17
7 Применение……...19
7.1 Назначение и условия применения программы……..19
7.2 Порядок использования программы……20
7.3 Завершение работы с программой……...28
Заключение ……29
Список использованных источников……....30
Приложение А Листинг программы …….
Приложение Б Диаграммы UML……
Приложение В Тест-кейсы……..

Введение

В рамках технологической практики разрабатывается игровое приложение «Шпион», предназначенное для проведения игры со скрытыми ролями в Telegram. Проект объединяет игроков, позволяет создавать онлайн-комнаты, приглашать участников и проводить партии, а также играть на одном телефоне. Цель - обеспечить удобное взаимодействие игроков и доступ к игровым возможностям: комнатам, наборам слов, истории, профилю и рейтингу. Пользователи могут создавать собственные наборы, добавлять слова, использовать избранное и отправлять жалобы; предусмотрены возможности игрока, ведущего комнаты и администратора.

Предметная область - организация коллективной игры со скрытыми ролями. При проведении игры необходимо распределять роли, выбирать слово, соблюдать очерёдность ходов и определять результат. Приложение «Шпион» объединяет эти действия: предлагает онлайн-режим и режим на одном телефоне, хранит наборы слов, историю и настройки. Аудитория - пользователи Telegram, заинтересованные в совместных играх. Программа реализована на Python с использованием aiogram; данные хранятся в MySQL, доступ осуществляется через SQLAlchemy. Учитываются удобная навигация, разграничение прав и единое оформление. Для защиты используются проверка Telegram ID, параметризованные запросы, ограничение частоты обращений и экранирование выводимого текста.

Первый раздел документации включает описание ООО «РБИ-софт» в Гродно, основного направления деятельности компании, характеристик специализированного программного обеспечения для энергосистем и промышленных предприятий, общей технологической схемы разработки и роли компьютерной техники в выполнении этих задач.

Второй раздел посвящён должностным обязанностям оператора ЭВМ, техника-программиста и инженера-программиста: обработке информации, подготовке документов, установке и настройке программ, диагностике техники, технической помощи сотрудникам и сопровождению программного обеспечения.

Третий раздел содержит анализ задачи для приложения «Шпион» - назначение проекта, целевую аудиторию, проблемы организации игры, функциональные возможности, инструменты разработки, диаграмму вариантов использования, модель данных и постановку задачи.

Четвёртый раздел описывает проектирование задачи: структуру приложения, взаимосвязи компонентов, взаимодействие обработчиков, сервисов и моделей, основные переходы, сценарии работы игроков, организацию взаимодействия с MySQL и разграничение прав доступа.

Пятый раздел посвящён реализации: особенностям игровых экранов, интерфейсу средствами Telegram, обработке событий на Python, подключению MySQL через SQLAlchemy, комнатам, локальной игре, профилю, наборам слов, истории, рейтингу и инструментам администрирования.

Шестой раздел описывает тестирование: переходы между экранами, вход в комнату, проведение раунда, сохранение истории, работу с наборами слов, разграничение прав, обработку ввода, работу с базой данных и корректность интерфейса с использованием тестовых сценариев.

Седьмой раздел посвящён применению: назначению приложения, требованиям к программным средствам, условиям запуска, подготовке Python и MySQL, открытию бота в Telegram, работе с разделами, созданию комнат, входу по коду и администрированию.

В заключении подводятся основные результаты практики, оценивается достижение цели и формулируются выводы о выполненной работе, включая анализ предметной области, проектирование структуры, реализацию функций и проверку работоспособности компонентов. Также указываются направления дальнейшего развития: расширение наборов слов, совершенствование интерфейса и резервного копирования.

В приложении А - представлены листинги программного кода.

В приложении Б представлены диаграмма вариантов использования и диаграмма модели данных, показывающая взаимосвязи основных таблиц базы данных.

1 Структура и общая технологическая схема производства, характеристики основных видов продукции

ООО «РБИ-софт» осуществляет деятельность в области компьютерного программирования в Гродно. Основные направления - разработка и продажа специализированного программного обеспечения для энергосистем и промышленных предприятий. К программным решениям компании относится ПО «Оперативные заявки».

Работа в области компьютерного программирования объединяет задачи анализа требований, проектирования, разработки, тестирования и сопровождения программ. Для их выполнения используются компьютерная техника и средства разработки. Согласованность этих работ необходима для создания программного продукта, соответствующего требованиям пользователей.

Основная продукция - специализированное программное обеспечение для энергосистем и промышленных предприятий. Такие программы предназначены для решения задач определённой предметной области. Их разработка требует учёта назначения продукта, условий применения и потребностей пользователей.

К характеристикам программной продукции относятся состав функций, удобство интерфейса, корректность обработки данных и устойчивость работы. Программа должна выполнять предусмотренные действия и сохранять необходимую информацию. Для её использования важны исправность компьютерной техники и корректная настройка программного обеспечения.

Процесс разработки в общем виде начинается с постановки задачи и уточнения требований. Определяются назначение программы, необходимые функции и исходные данные. Полнота требований влияет на последующее проектирование и реализацию.

После уточнения требований определяется структура приложения и разрабатывается программный код. Создаются пользовательский интерфейс и средства работы с данными. Соблюдение последовательности этих этапов позволяет связать реализованные функции с поставленной задачей.

После реализации проверяется работоспособность программы. Контролируются обработка входных данных, выполнение основных функций и сохранение результатов. Выявленные ошибки исправляются, затем проводится повторная проверка. Такой контроль необходим для подготовки программы к использованию.

Подготовленный программный продукт сопровождается документацией. В ней указываются назначение программы, условия запуска и порядок работы пользователя. Эти материалы необходимы для применения и дальнейшего сопровождения программного обеспечения.

Дополнительное направление - информационная поддержка программного продукта. Она связана с описанием возможностей программы, подготовкой инструкций и разъяснением порядка работы. При сопровождении проверяются работоспособность функций, понятность сообщений и актуальность документации.

Таким образом, деятельность ООО «РБИ-софт» связана с разработкой и продажей специализированного программного обеспечения. Создание программного продукта включает организационные и технические задачи. Для качественного выполнения работ необходимы исправная компьютерная техника, корректная работа программ и контроль полученных результатов.

2 Должностные обязанности оператора ЭВМ, техника-программиста, инженера-программиста

В деятельности ООО «РБИ-софт» информационные технологии связаны с разработкой программ, обработкой информации и подготовкой технической документации. Выполнение этих задач требует работы специалистов, отвечающих за обработку данных, обслуживание компьютерной техники и создание программного обеспечения.

Оператор ЭВМ выполняет ввод, обработку и систематизацию информации, работает с электронными документами и таблицами, подготавливает материалы к печати, сканирует и копирует документы. Важными обязанностями являются проверка правильности введённых данных, соблюдение порядка оформления документов и обеспечение сохранности служебной информации.

Техник-программист занимается установкой и настройкой программного обеспечения, диагностикой неисправностей компьютерной техники, подключением периферийных устройств и оказанием технической помощи сотрудникам. Он проверяет работоспособность программ и оборудования, устраняет несложные неисправности в пределах своих полномочий и ведёт необходимую техническую документацию.

Инженер-программист отвечает за проектирование, разработку, тестирование и сопровождение программного обеспечения. Его обязанности включают анализ требований пользователей, разработку алгоритмов, автоматизацию обработки информации, исправление ошибок и подготовку технической документации. При выполнении работ специалист должен обеспечивать корректность функционирования программ и соблюдать требования информационной безопасности.

Технологическая практика проходила в ООО «РБИ-софт» по профилю техника-программиста. Во время практики выполнялись настройка программного обеспечения, работа с базой данных, тестирование отдельных функций приложения и подготовка технической документации. В проектной части разрабатывалось игровое приложение «Шпион»: рассматривались его структура, основные функции, работа с данными и проверка программного кода.

3 Анализ задачи

3.1 Анализ предметной области

В игре «Шпион» участники получают скрытые роли. Один из игроков становится шпионом и не знает выбранного слова, остальные получают общее слово. Задача обычных игроков — определить шпиона, задача шпиона — остаться нераскрытым или угадать слово. Для проведения игры необходимо выбрать тему, распределить роли, организовать обсуждение и голосование. При ручной организации эти действия выполняет ведущий. Приложение «Шпион» объединяет их в интерфейсе Telegram: пользователь может создать онлайн-комнату, пригласить участников и начать партию. Для компании, использующей одно устройство, предусмотрен режим игры на одном телефоне.

Наборы слов содержат темы и слова для игровых раундов. Пользователь может просматривать доступные наборы, создавать личные, добавлять слова, копировать наборы и сохранять их в избранное. Публичные наборы доступны другим игрокам; для публикации требуется не менее десяти слов. История содержит сведения о проведённых раундах, участниках, результатах и использованных наборах. В профиле и рейтинге отображаются игровые показатели. В настройках выбираются часовой пояс, эмодзи и цветное оформление кнопок.

Поскольку пользовательские наборы могут содержать неуместные сведения, предусмотрены жалобы и административное управление. Владелец редактирует собственный набор и его доступность, администратор управляет наборами и блокировками. Ведущий комнаты выбирает параметры партии и запускает раунды, остальные участники подтверждают готовность и выполняют игровые действия. Роли и слово передаются участникам отдельно от общих игровых экранов. Таким образом, предметная область проекта включает организацию игры, хранение её результатов и управление пользовательскими наборами слов.

3.2 Инструменты разработки

Для разработки приложения «Шпион» используются язык Python и библиотека aiogram 3. Python применяется для реализации игровых правил, распределения ролей, обработки действий участников и подсчёта результатов. Библиотека aiogram обеспечивает взаимодействие с Telegram Bot API: получение сообщений, обработку нажатий кнопок и отправку ответов. Интерфейс формируется из сообщений, нижней клавиатуры и кнопок текущего экрана.

Обработчики принимают действия пользователей, сервисы выполняют операции над комнатами, наборами и историей, а модели описывают сохраняемые данные.

Для хранения информации используется MySQL 8. Взаимодействие с базой данных выполняется через SQLAlchemy и драйвер PyMySQL. В базе хранятся комнаты, участники, наборы слов, результаты игр, настройки и блокировки. При передаче пользовательских значений применяются параметризованные запросы. Для запуска на Windows необходимы Python, установленные зависимости из requirements.txt и работающий сервер MySQL. Токен бота и параметры подключения указываются в файле .env. Проверка интерфейса и работа пользователя выполняются в Telegram.

3.3 Диаграмма вариантов использования

Диаграмма вариантов использования игрового приложения «Шпион» отражает действия, доступные игроку, ведущему комнаты и администратору. Изображение диаграммы представлено на рисунке П.Б.1 в приложении Б.

3.4 Модель данных

Модель данных реализована в реляционной базе MySQL и включает таблицы для комнат, участников, наборов слов, истории игр, настроек и блокировок. Упрощённая диаграмма основных таблиц и связей, заданных внешними ключами, приведена на рисунке П.Б.2 в приложении Б.

3.5 Постановка задачи

В рамках проектной части поставлена задача разработать игровое приложение «Шпион», предназначенное для проведения коллективной игры со скрытыми ролями в Telegram. Программа должна обеспечивать игру в онлайн-комнате с отдельных устройств и локальную игру на одном телефоне. Основными функциональными требованиями являются: открытие главного меню и правил; создание комнаты и присоединение по коду или ссылке-приглашению; выбор наборов слов, числа раундов и времени хода; подтверждение готовности и запуск партии ведущим; распределение ролей и показ секретной информации участникам; проведение обсуждения, голосования и попытки угадать слово; определение результата и однократное начисление очков за раунд; сохранение истории онлайн-игр с участниками, результатами и использованными наборами; просмотр профиля и рейтинга; создание, редактирование, копирование, поиск, импорт и экспорт

наборов слов, добавление в избранное и изменение доступности собственных наборов; отправка жалоб и административное управление наборами и блокировками; настройка часового пояса, эмодзи и цветных кнопок. Для публикации пользовательского набора необходимо не менее десяти слов. К нефункциональным требованиям относятся единое оформление экранов, понятная навигация, возможность отмены ввода и возврата в меню, обновление текущих сообщений без лишнего засорения чата, защита активных игровых панелей при переходах между разделами, проверка входных данных и разграничение прав доступа. Приложение должно работать на Python с использованием aiogram, а данные хранить в MySQL через SQLAlchemy.

Ожидаемым результатом является игровое приложение с единым интерфейсом Telegram для подготовки и проведения партий, работы с наборами слов и просмотра результатов. Программа должна сохранять состояние комнат и незавершённых локальных игр, обеспечивать последовательную обработку действий участников и исключать повторное применение результата одного раунда. Роли и слово не должны раскрываться на общих игровых экранах до объявления результатов раунда. Доступ к изменению наборов, параметрам комнаты и административным операциям должен проверяться в соответствии с правами пользователя. Пользователь должен иметь возможность продолжить сохранённую локальную игру или вернуться к своей онлайн-комнате через меню.

4 Проектирование задачи

При проектировании игрового приложения «Шпион» была определена общая структура программы, назначение её основных частей и порядок взаимодействия пользователя с ботом. Программа построена таким образом, чтобы обработка действий пользователей, работа с данными и формирование экранов выполнялись отдельными частями приложения. Это позволяет распределить программный код по назначению и упростить его дальнейшее изменение.

Основным файлом проекта является app/main.py. Он подготавливает настройки, подключение к базе данных и обработчики Telegram. В зависимости от сообщения или нажатой кнопки управление передаётся соответствующему обработчику. Обработчик проверяет контекст действия и обращается к сервису. Сервис выполняет операции с игровым состоянием и базой данных: получает, добавляет или изменяет информацию. После этого результат используется для отображения экрана в Telegram.

Например, при открытии истории программа получает записи из базы данных и формирует список игр. При создании набора введённые пользователем данные проверяются, после чего сохраняются в базе данных. Таким образом, каждая часть приложения выполняет свою задачу, что позволяет поддерживать порядок в программном коде.

Файлы проекта распределены по отдельным папкам в зависимости от их назначения. Основные каталоги и файлы представлены в таблице 1.

Таблица 1 – Основные каталоги и файлы проекта

Путь	Назначение
1	2
app/main.py	Основной файл запуска и подключения обработчиков.
app/core/config.py	Параметры бота и подключения к MySQL.
app/database/session.py	Создание соединения с базой данных.
app/core/security.py	Проверка административных прав по Telegram ID.
app/bot/handlers/	Обработчики разделов и действий игроков.
app/bot/keyboards/	Клавиатуры и кнопки интерфейса.
app/game/	Игровые правила и отображение состояния.
app/database/models.py	Модели таблиц базы данных.
app/services/	Операции комнат, наборов и истории.
app/bot/ui.py	Отслеживание и обновление экранов.

Переход между разделами выполняется с помощью нижней клавиатуры и кнопок под сообщениями. По тексту сообщения или callback-данным программа определяет, какой экран необходимо открыть. Например, кнопка «Играть онлайн» открывает выбор создания комнаты и входа по коду, а «История игр» - сохранённые результаты. Устаревшие игровые действия проверяются по текущему состоянию. При ошибке ввода пользователю выводится пояснение и предоставляется возможность вернуться в меню.

Основные разделы приложения и их условия доступа представлены в таблице 2.

Таблица 2 – Основные разделы приложения

Раздел приложения	Назначение	Доступ
1	2	3
Главное меню	Выбор режима и раздела.	Все пользователи.
Играть онлайн	Создание комнаты и вход по коду.	Пользователи без блокировки.
Один телефон	Игра на одном устройстве.	Все пользователи.
Наборы слов	Просмотр библиотеки и своих наборов.	С учётом видимости набора.
История игр	Просмотр лобби и раундов.	Участники и администратор.
Профиль	Просмотр личной статистики.	Владелец профиля.
Рейтинг	Просмотр результатов игроков.	Все пользователи.
Мои наборы	Создание и изменение слов.	Владелец набора.
Избранное	Сохранение доступных наборов.	Текущий пользователь.
Настройки	Часовой пояс и оформление.	Текущий пользователь.
Моя комната	Возврат в активную комнату.	Участник комнаты.
Администрирование	Наборы, жалобы и блокировки.	Администратор.

При проектировании приложения также был предусмотрен порядок создания и публикации пользовательских наборов. Для того чтобы в общей библиотеке отображались доступные другим игрокам наборы, владелец отдельно изменяет видимость. После создания набор сохраняется как личный. Для публикации необходимо добавить не менее десяти слов. Администратор может скрывать наборы, а пользователи - отправлять жалобы.

Основной порядок создания набора включает следующие этапы:

– пользователь открывает главное меню бота;

– переходит в раздел «Наборы слов» и выбирает создание набора;

– указывает название и добавляет слова, после чего данные сохраняются в базе;

– проверяет содержимое и при необходимости включает публичный доступ;

– после публикации набор становится доступен другим игрокам с учётом правил доступа.

Для собственных наборов предусмотрены редактирование, импорт, экспорт и удаление. Публичный набор можно добавить в избранное или скопировать. При обнаружении нежелательного содержимого отправляется жалоба. Администратор рассматривает доступные ему сведения и изменяет состояние набора. Выбранные темы используются при подготовке раунда; сведения о наборе сохраняются в истории вместе с результатом игры.

Отдельное внимание при проектировании было уделено защите информации и ограничению доступа к функциям приложения. Для этого в программе применяются следующие средства:

– проверка Telegram ID пользователя при выполнении действий;

– параметризованные запросы для передачи пользовательских данных в MySQL;

– проверка состояния комнаты и актуальности игровой панели;

– экранирование текста перед включением в сообщения с HTML-разметкой;

– проверка владельца набора и административных прав;

– ограничение частоты событий и попыток входа в комнаты;

– проверка длины вводимого текста и формата кода комнаты.

В системе предусмотрены возможности игрока, ведущего и администратора. Игроки могут пользоваться основными разделами, создавать наборы и участвовать в партиях. Ведущий настраивает и запускает комнату. Администратор дополнительно управляет наборами, жалобами и блокировками. Возможности ведущего относятся к конкретной комнате; административные права определяются настроенным числовым Telegram ID.

Перечисленные средства позволяют снизить отдельные риски при обработке пользовательских данных, однако не гарантируют полной безопасности приложения. Перед эксплуатацией необходимо проверить настройки подключения к MySQL, сохранность токена бота и доступ к файлу .env, организовать резервное копирование и проверить актуальность зависимостей.

Таким образом, при проектировании игрового приложения «Шпион» была определена его общая структура, назначение основных файлов, порядок перехода между разделами и правила взаимодействия пользователей с системой. Были

предусмотрены разграничение прав доступа, проверка ввода и основные средства защиты информации. Это позволило подготовить основу для реализации игровых функций.

5 Реализация

Игровое приложение «Шпион» реализовано на Python с базой данных MySQL. Обработчики aiogram принимают события Telegram, сервисы работают с игровым состоянием, модели SQLAlchemy описывают данные. Общие функции отвечают за подключение к БД, проверку прав и служебные сообщения. Данные обрабатываются через сервисы комнат, наборов и истории. Подготовленные запросы и параметры SQLAlchemy снижают риск SQL-инъекций; база включает таблицы комнат, участников, наборов, слов, истории, настроек и блокировок. Связи дочерних записей заданы внешними ключами.

Интерфейс построен на сообщениях и клавиатурах Telegram. Нижняя клавиатура обеспечивает доступ к основным разделам, кнопки под сообщениями - действия текущего экрана. Обработчики в app/bot/handlers охватывают главное меню, онлайн-игру, локальную игру, наборы, историю, профиль, рейтинг и администрирование. Формирование игровых сообщений находится в app/game, оформление кнопок - в общих средствах интерфейса. Текущий экран по возможности редактируется, а активные игровые сообщения защищаются от случайного удаления при навигации.

Доступ разграничен по Telegram ID, принадлежности к комнате и владению набором. Для изменения собственных наборов проверяется владелец, административные операции доступны настроенному администратору. Онлайн-комната хранит состав участников, параметры партии и текущее состояние. Локальная игра сохраняется отдельно по владельцу. В истории записываются результаты раундов и использованные наборы. При ошибке Telegram предусмотрено повторное обновление экранов; синхронные операции базы данных выполняются через отдельный рабочий поток.

6 Тестирование

Для проверки игрового приложения «Шпион» отобрано 50 основных тест-кейсов. Они охватывают навигацию, вход в комнату, проведение раунда, работу с наборами слов, историю и права доступа. Календарный план предусматривает работы с 29 октября по 9 ноября 2026 года. Длительность в таблицах 3 и 4 указана как плановая; результаты в таблицах 5–7 относятся к ранее проверенной контрольной версии приложения.

Команда тестировщиков описана в таблице 3

Таблица 3 – Команда тестировщиков

ФИО	Дата	Описание	Длительность, ч (план)
Тупик П. Н.	29.10.2026	Разработка тест-кейсов	2

Проверяемая среда включает Windows, Telegram и базу данных MySQL. Основное внимание уделяется правильности игровых действий, переходам между разделами, обработке некорректного ввода и сохранению результатов.

Таблица 4 – Расписание проведения тестирования

ФИО	Дата	Описание	Длительность, ч (план)
Тупик П. Н.	29.10.2026	Разработка тест-кейсов	3
Тупик П. Н.	02.11.2026	Проверка игровых сценариев	2
Тупик П. Н.	03.11.2026	Анализ выполнения тест-кейсов	2
Тупик П. Н.	05.11.2026	Повторная проверка исправлений	1
Тупик П. Н.	09.11.2026	Оформление результатов	2

Для проверки подготовлен набор из 50 тест-кейсов, охватывающих основные функциональные требования. Шаги выполнения, ожидаемые и фактические результаты представлены в таблице В.1 приложения В.

Сводная статистика по результатам проверки 50 тест-кейсов приведена в таблице 5.

Таблица 5 – Статистика по разработанным тест-кейсам

Статус	Количество тест-кейсов	Приоритет				
		Крайне низкий	Низкий	Средний	Высокий	Крайне высокий
Пройдено	50	0	0	20	22	8
Не пройдено	0	0	0	0	0	0
Заблокировано	0	0	0	0	0	0
Не протестировано	0	0	0	0	0	0

В таблице 6 представлена статистика по всем дефектам

Таблица 6 – Статистика по выявленным ошибкам

Статус	Количество ошибок	Важность			
		Критическая	Высокая	Средняя	Низкая
Найдено	4	0	2	2	0
Исправлено	4	0	2	2	0
Проверено	4	0	2	2	0
Открыто заново	0	0	0	0	0
Отклонено	0	0	0	0	0

Список найденных дефектов представлен в таблице 7

Таблица 7 – Список найденных дефектов

Идентификатор	Важность	Описание	Статус
B-01	Высокая	Задержка онлайн-хода из-за неточного времени. Проверка T_22.	Исправлено
B-02	Высокая	Ошибка лобби после удаления темы. Проверка T_34.	Исправлено
B-03	Средняя	Таймер мешал очистке заброшенной комнаты. Проверка T_36.	Исправлено
B-04	Средняя	Публичный набор содержал менее 10 слов. Проверка T_33.	Исправлено

Вывод: для выбранных 50 тест-кейсов подтверждён ожидаемый результат. Четыре ранее выявленных дефекта исправлены и проверены повторно. Результаты относятся к перечисленным сценариям контрольной версии приложения.

7 Применение

7.1 Назначение и условия применения программы

Программный продукт имеет название игровое приложение «Шпион». Его основное назначение заключается в проведении совместной игры со скрытыми ролями в Telegram, организации онлайн-комнат и локальных партий, сохранении результатов и использовании тематических наборов слов.

Приложение позволяет создавать комнаты, приглашать участников, распределять роли и проводить раунды. Пользователи могут создавать собственные наборы, добавлять слова, просматривать историю, профиль и рейтинг, изменять настройки интерфейса.

Для организации работы приложения предусмотрены следующие возможности пользователей:

– игрок - использование основных разделов и участие в партии;

– владелец набора - создание и изменение собственных наборов слов;

– ведущий комнаты - выбор параметров и запуск раундов;

– администратор - управление наборами, жалобами и блокировками.

Область применения системы - совместный досуг пользователей Telegram, играющих с отдельных устройств или использующих одно устройство для всей компании.

Периодичность использования приложения не ограничена определённым временем. Пользователи могут создавать партии, просматривать историю и работать с наборами по мере необходимости.

Для функционирования программы используются:

– язык программирования Python;

– библиотека aiogram для Telegram Bot API;

– SQLAlchemy для работы с данными;

– драйвер PyMySQL;

– система управления базами данных MySQL;

Для работы с приложением со стороны пользователя необходим Telegram и подключение к Интернету. Бот может использоваться на персональных компьютерах, ноутбуках и мобильных устройствах, поддерживающих Telegram.

Серверная часть программы использует Python и MySQL. Необходимо установить зависимости из requirements.txt, указать токен бота и параметры подключения к базе данных в .env. Для запуска предусмотрена команда python -m app.main.

7.2 Порядок использования программы

Для начала работы необходимо открыть чат бота в Telegram и вызвать команду /start. Отдельная регистрация с паролем не требуется: приложение использует Telegram ID.

Просмотр главного меню и основных разделов

Главное меню показано на рисунке 1. Нижняя клавиатура содержит кнопки игровых режимов, наборов слов, истории, профиля, рейтинга и настроек.

– Откройте чат бота в Telegram.

– Вызовите команду /start.

– Выберите нужный раздел на нижней клавиатуре.

– Для возврата используйте «Назад», «В меню» или «Закрыть».

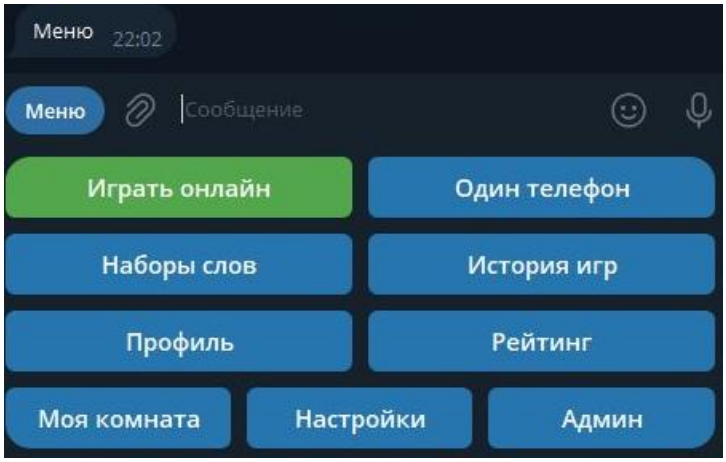


Рисунок 1 – Главное меню приложения

Открытие онлайн-игры

Для совместной игры с отдельных устройств откройте раздел «Играть онлайн», представленный на рисунке 2.

– Нажмите «Создать комнату» для новой партии.

– Для подключения к чужой комнате выберите «Войти по коду».

– При необходимости вернитесь в главное меню кнопкой «Закрыть».

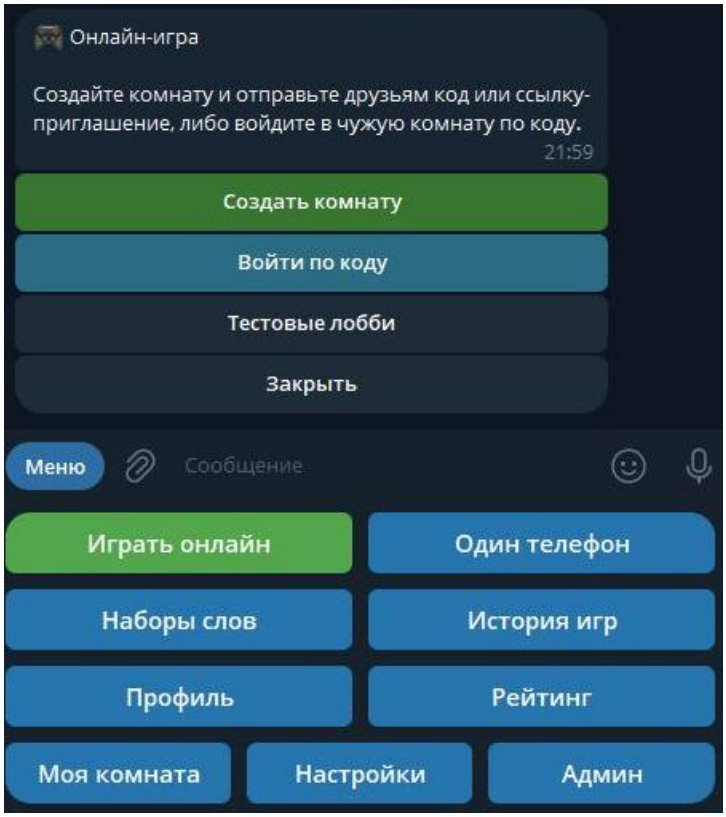


Рисунок 2 – Раздел онлайн-игры

Создание комнаты и приглашение участников

После создания комнаты открывается лобби (рисунок 3). В нём отображаются код, ссылка, список игроков, готовность участников и параметры партии.

– Передайте участникам код комнаты или ссылку.

– Для отправки приглашения нажмите «Пригласить».

– Для копирования ссылки используйте «Скопировать ссылку».

– Перед запуском выберите темы и дождитесь готовности игроков.

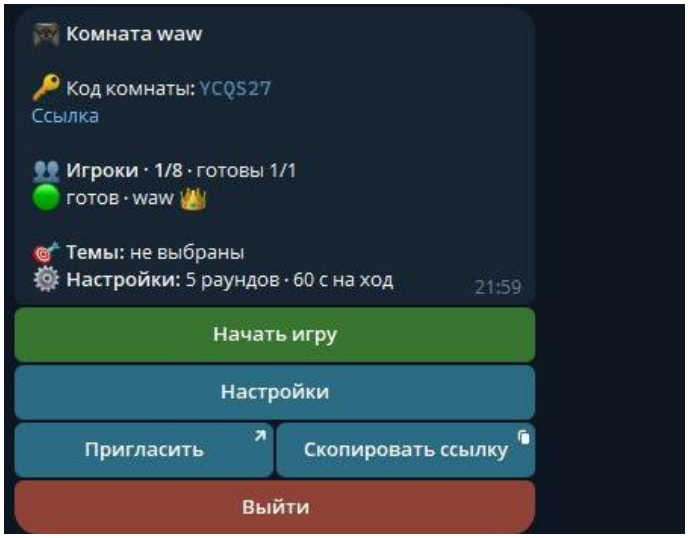


Рисунок 3 – Лобби онлайн-комнаты

Настройка онлайн-партии

Ведущий открывает «Настройки» в лобби. На рисунке 4 показан экран изменения параметров комнаты.

– Укажите допустимое число игроков и количество раундов.

– Выберите длительность хода.

– Откройте «Темы» и отметьте нужные наборы.

– Нажмите «Готово, назад в лобби».

Значения сохраняются при нажатии. Для обычной онлайн-партии необходимы не менее трёх участников, выбранная тема и готовность игроков.

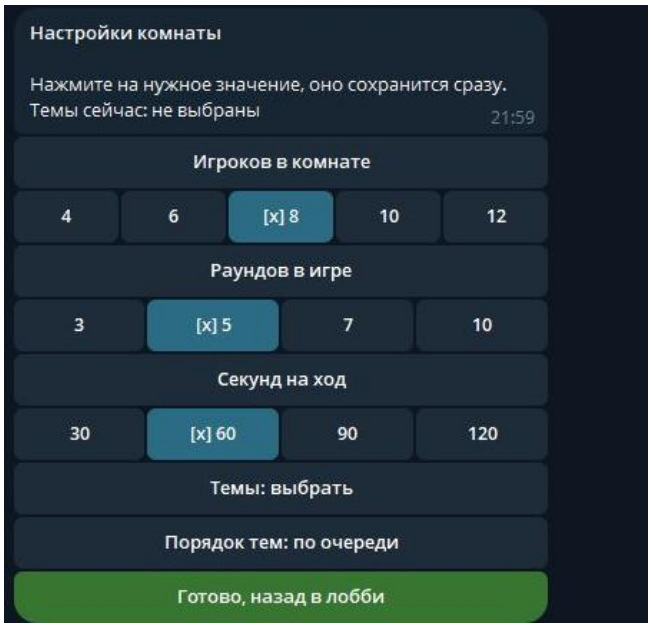


Рисунок 4 – Настройки онлайн-комнаты

Вход в комнату по коду

Экран ввода кода приведён на рисунке 5. Код состоит из шести символов и передаётся участнику ведущим.

– Откройте «Играть онлайн» и нажмите «Войти по коду».

– Введите полученный код сообщением в чат.

– При ошибке проверьте код и повторите ввод.

– Для отмены нажмите «Отмена» или «В меню».

После успешного ввода пользователь присоединяется к комнате. Если комната закрыта или недоступна, бот выводит пояснение.

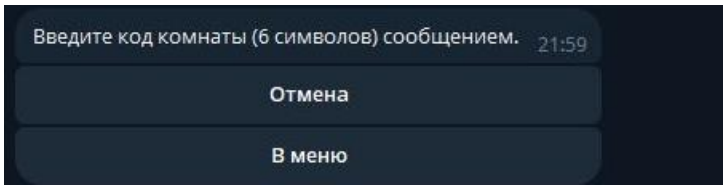


Рисунок 5 – Ввод кода комнаты

Подготовка игры на одном устройстве

Для компании с одним телефоном предусмотрен локальный режим. Выбор количества участников показан на рисунке 6.

– Откройте раздел «Один телефон».

– Выберите количество игроков от трёх до двенадцати.

– Последовательно введите имена участников сообщениями.

– Для исправления состава вернитесь к выбору количества игроков.

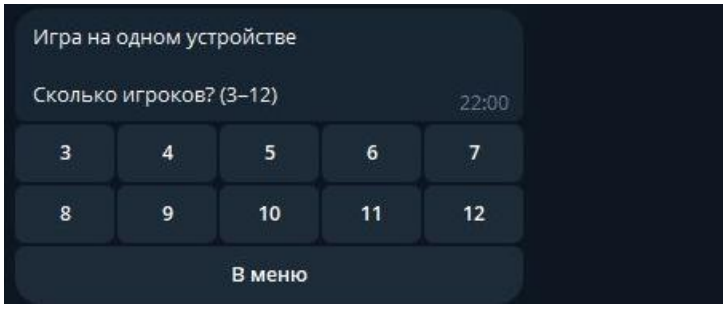


Рисунок 6 – Выбор количества игроков

Выбор тем и запуск локальной игры

На экране «Темы» отметьте один или несколько наборов и нажмите «Готово» (рисунок 7). Проверьте имена и выбранные темы, затем нажмите «Начать игру» (рисунок 8).



Рисунок 7 – Выбор тем локальной игры

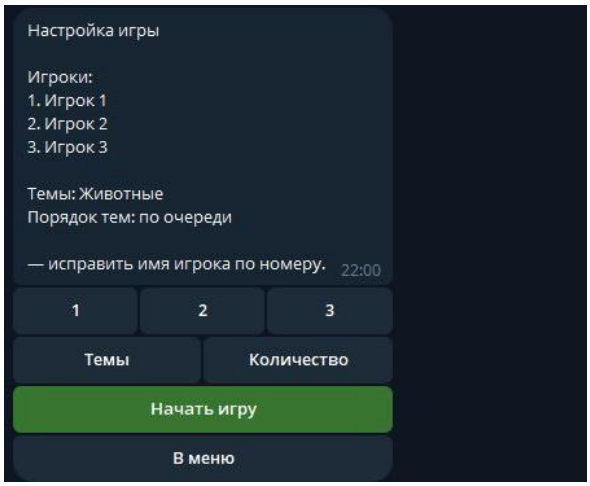


Рисунок 8 – Подготовленная локальная игра

Получение роли участником

После запуска бот предлагает передать устройство очередному игроку (рисунок 9). Остальные участники не должны смотреть на экран.

– Передайте телефон указанному игроку.

– Нажмите «Показать роль», затем «Показать».

– Прочитайте роль во всплывающем окне (рисунок 10).

– Закройте окно, нажмите «Дальше» и передайте устройство.

Роль и слово не остаются отдельным сообщением в чате. После выдачи ролей всем участникам начинается обсуждение.

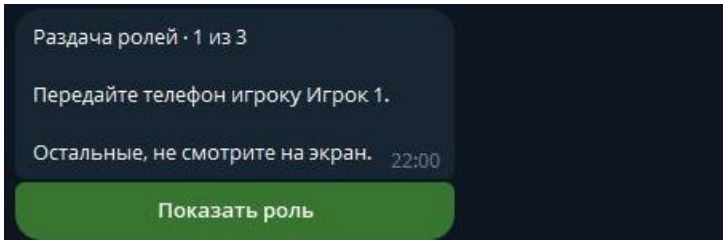


Рисунок 9 – Передача устройства для получения роли

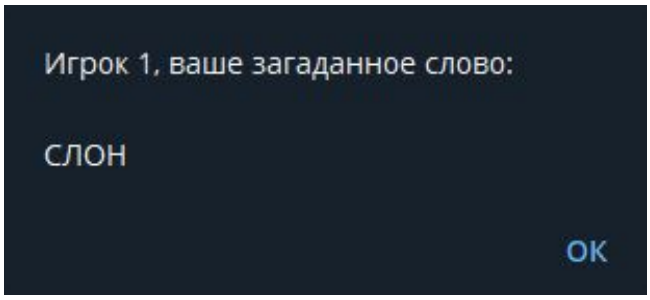


Рисунок 10 – Всплывающее окно с ролью игрока

Просмотр профиля и истории игр

В разделе «Профиль» отображаются результаты пользователя. Если онлайн-раундов ещё нет, выводится соответствующее сообщение (рисунок 11).

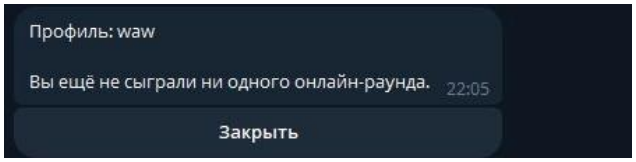


Рисунок 11 – Профиль пользователя

Для просмотра проведённых партий откройте «История игр», выберите «Мои игры» либо «История лобби» и откройте запись (рисунок 12).

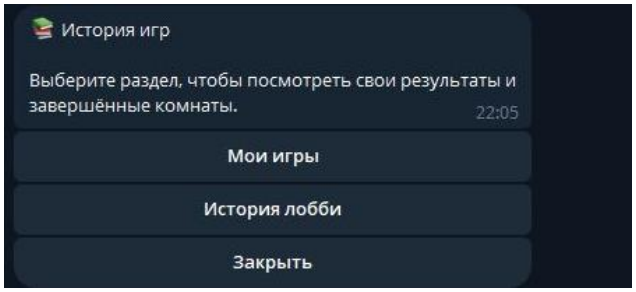


Рисунок 12 – Раздел истории игр

Работа с наборами слов

Для выбора тем и подготовки собственных слов откройте раздел «Наборы слов» (рисунок 13).

– Выберите собственные, готовые либо публичные наборы.

– Для нового набора нажмите «Создать набор» и укажите название.

– Добавьте слова сообщением или используйте импорт.

– Для публикации подготовьте не менее десяти слов.

Раздел «Избранное» обеспечивает быстрый доступ к сохранённым наборам.

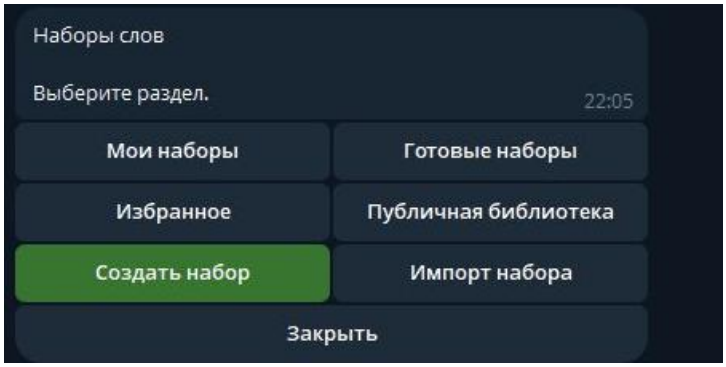


Рисунок 13 – Раздел наборов слов

Изменение собственного набора

Откройте «Мои наборы» и выберите нужную карточку. Доступные действия показаны на рисунке 14.

– Для изменения содержания нажмите «Слова» или «Добавить слова».

– Для изменения названия используйте «Переименовать».

– Выберите личный или публичный доступ соответствующей кнопкой.

– При необходимости используйте копирование, экспорт или избранное.

– Для возврата к списку нажмите «Назад».

Удаление набора требует отдельного действия. Изменять собственный набор может его владелец; административные операции доступны при наличии прав.

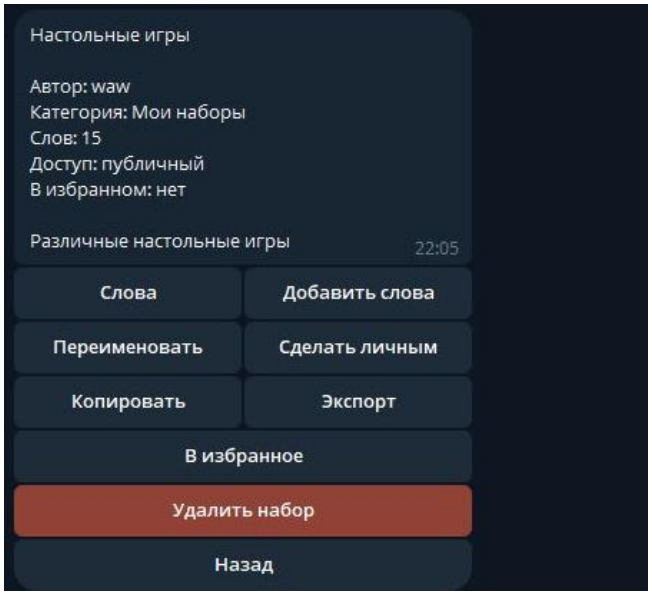


Рисунок 14 – Карточка собственного набора

Изменение настроек

В разделе «Настройки» выбираются оформление кнопок и часовой пояс. Экран настроек представлен на рисунке 15.

– Измените состояние эмодзи или цветных кнопок.

– При необходимости откройте «Часовой пояс».

– Выберите подходящее значение из списка.

– Вернитесь кнопкой «Назад к настройкам» или закройте экран.

Выбранные параметры сохраняются для пользователя. Нижняя клавиатура остаётся доступной при переходах между разделами.

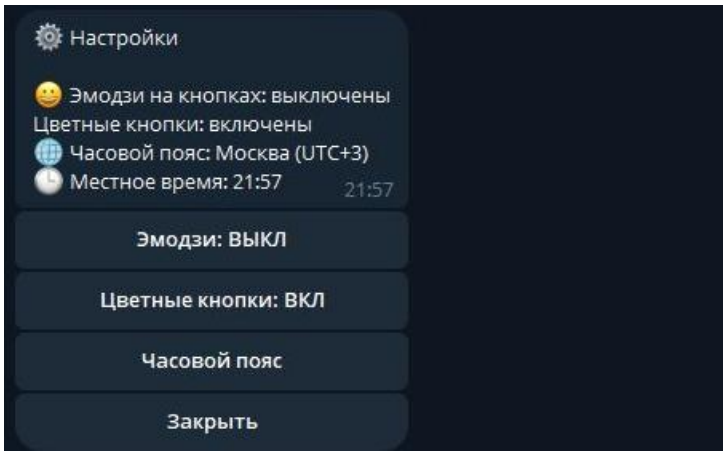


Рисунок 15 – Настройки приложения

Модерация наборов и управление приложением

Панель администратора показана на рисунке 16. Она доступна только настроенному администратору по его Telegram ID.

Откройте «Админ» и выберите нужное действие. Для рассмотрения обращений нажмите «Жалобы на наборы». При отсутствии обращений выводится сообщение, показанное на рисунке 17.

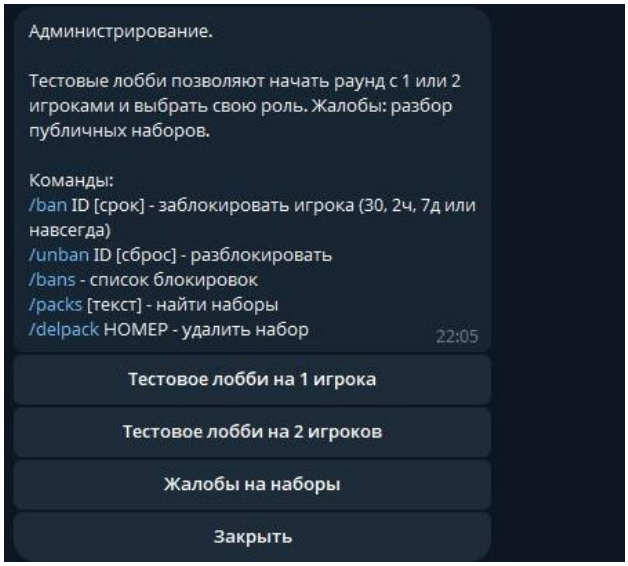


Рисунок 16 – Панель администратора



Рисунок 17 – Раздел жалоб на наборы

7.3 Завершение работы с программой

Для завершения работы с разделом необходимо воспользоваться кнопкой «Закрыть», «Назад» или перейти в главное меню. Для выхода из комнаты применяется отдельная кнопка выхода; во время активной игры может потребоваться подтверждение.

После завершения работы можно закрыть окно Telegram. Сохранённые данные остаются в базе приложения.

Заключение

В ходе технологической практики рассмотрена деятельность ООО «РБИ-софт» в городе Гродно, связанная с компьютерным программированием, разработкой и продажей специализированного программного обеспечения для энергосистем и промышленных предприятий. Описаны основные сведения об организации, характеристики её продукции и общая технологическая схема разработки программ. Рассмотрены должностные обязанности оператора ЭВМ, техника-программиста и инженера-программиста.

В проектной части рассмотрено игровое приложение «Шпион», предназначенное для проведения совместных игр в Telegram. Проанализированы предметная область, назначение приложения и основные категории пользователей. Описаны онлайн-комнаты, локальная игра, наборы слов, история, профиль, рейтинг, настройки и средства администрирования.

При проверке приложения рассмотрены основные игровые сценарии, навигация и работа с данными. Для 50 выбранных тест-кейсов подтверждены ожидаемые результаты. Выявленные ошибки таймера, выбора удалённого набора, очистки заброшенной комнаты и ограничения публичного набора исправлены и проверены повторно.

Таким образом, проектная часть практики охватывает анализ предметной области, описание структуры и функций приложения и проверку его компонентов. Для дальнейшего использования необходимо заполнить фактические сведения о практике, согласовать документацию с актуальными исходниками, выполнить ручную приёмку в Telegram и зафиксировать её результаты. Возможными направлениями развития являются совершенствование резервного копирования, проверки производительности и интерфейса.

Дальнейшее развитие возможно за счёт расширения наборов слов и совершенствования сопровождения программы после проверки актуальной сборки.

Список использованных источников

1 Документация Telegram Bot API [Электронный ресурс] – URL: https://core.telegram.org/bots/api – дата доступа: 06.11.2026.

2 Документация aiogram [Электронный ресурс] – URL: https://docs.aiogram.dev/en/latest/ – дата доступа: 06.11.2026.

3 Документация Python: asyncio [Электронный ресурс] – URL: https://docs.python.org/3/library/asyncio.html – дата доступа: 06.11.2026.

4 Документация SQLAlchemy [Электронный ресурс] – URL: https://docs.sqlalchemy.org/en/20/ – дата доступа: 06.11.2026.

5 Документация MySQL [Электронный ресурс] – URL: https://dev.mysql.com/doc/refman/8.4/en/ – дата доступа: 06.11.2026.

6 Документация SQLAlchemy: работа с параметрами запросов [Электронный ресурс]
– URL: https://docs.sqlalchemy.org/en/20/core/sqlelement.html – дата доступа: 06.11.2026.

Приложение А
Листинг программы

main.py

```
import asyncio
import logging
from pathlib import Path
from aiogram import Bot, Dispatcher
from aiogram.fsm.storage.memory import SimpleEventIsolation
from aiogram.client.session.aiohttp import AiohttpSession
from aiogram.types import BotCommand, MenuButtonCommands
from sqlalchemy.orm import sessionmaker
from app.health import PollingHealthMiddleware, health_path, monitor_health
from app.database.worker import run_db
from app.bot.handlers import admin as admin_handlers
from app.bot.handlers import history, local, menu, online
from app.bot.handlers import packs as packs_handlers
from app.bot.handlers import settings as settings_handlers
from app.bot.handlers import stats as stats_handlers
from app.bot.errors import register_error_handlers
from app.bot.emoji import ButtonStyleMiddleware
from app.bot.middlewares.normalize import NormalizeButtonsMiddleware
from app.bot.middlewares.navigation import KeepMenuMiddleware
from app.bot.middlewares.throttle import ThrottleMiddleware
from app.bot.middlewares.private import PrivateCallbacksMiddleware
from app.bot.middlewares.track import TrackScreensMiddleware
from app.bot.runtime import OnlineRuntime
from app.bot.recovery import EditRecoveryMiddleware
from app.bot.tracker import ScreenTracker
from app.core.config import Settings, load_settings
from app.core.console import start_console_control
from app.database.info import check_connection, ensure_mysql, mask_url
from app.database.migrate import run_migrations
from app.database.pack_repository import PackRepository
from app.database.repository import LocalGameRepository
from app.database.session import make_session_factory
from app.services.admin_service import AdminService
from app.services.history_service import HistoryService
from app.services.pack_service import PackService
from app.services.room_service import RoomService
from app.services.settings_service import SettingsService
from app.services.stats_service import StatsService
def build_dispatcher(
    bot: Bot, bot_username: str, factory: sessionmaker, settings: Settings,
    anchor_file: Path | None = None,
) -> tuple[Dispatcher, OnlineRuntime]:
    pack_repo = PackRepository(factory)
    library = pack_repo.load_library()
    pack_service = PackService(factory, library)
    local_repo = LocalGameRepository(factory)
    user_settings = SettingsService(factory)
    runtime = OnlineRuntime(
        bot,
        RoomService(factory, library=library, pack_service=pack_service),
        bot_username,
        maintenance=[lambda: local_repo.purge_stale(30)],
        prefs_for=user_settings.prefs,
    )
    dp = Dispatcher(events_isolation=SimpleEventIsolation())
    register_error_handlers(dp)
    throttle = ThrottleMiddleware(0.25)
    dp.message.outer_middleware(throttle)

dp.callback_query.outer_middleware(throttle)

dp.callback_query.outer_middleware(PrivateCallbacksMiddleware())
    tracker = ScreenTracker(anchor_file)

dp.callback_query.outer_middleware(TrackScreensMiddleware(tracker))

dp.message.outer_middleware(KeepMenuMiddleware())

dp.callback_query.outer_middleware(KeepMenuMiddleware())
    dp["tracker"] = tracker
    dp["repo"] = local_repo
    dp["runtime"] = runtime
    dp["library"] = library
    dp["settings"] = settings
    dp["factory"] = factory
    dp["admin"] = AdminService(factory)
    dp["history"] = HistoryService(factory)
    dp["packs"] = pack_service
    dp["stats"] = StatsService(factory)
    dp["user_settings"] = user_settings
    runtime.edit_recovery = EditRecoveryMiddleware()

bot.session.middleware(runtime.edit_recovery)

bot.session.middleware(ButtonStyleMiddleware(user_settings.prefs))

dp.message.outer_middleware(NormalizeButtonsMiddleware())

dp.include_router(admin_handlers.router)
    dp.include_router(packs_handlers.router)
    dp.include_router(history.router)
    dp.include_router(online.router)
    dp.include_router(stats_handlers.router)

dp.include_router(settings_handlers.router)
    dp.include_router(menu.router)
    dp.include_router(local.router)
    return dp, runtime
async def main() -> None:

logging.basicConfig(level=logging.INFO)
    health_path().unlink(missing_ok=True)
    settings = load_settings()
    if settings.admin_id is None or settings.admin_id <= 0:
        logging.warning("ADMIN_ID is not configured: administrator actions are disabled")
    ensure_mysql(settings.database_url)
    logging.info("База данных: %s", mask_url(settings.database_url))
    try:
        await run_db(check_connection, settings.database_url)
    except Exception as err:
        raise SystemExit(
            f"Не удалось подключиться к MySQL ({mask_url(settings.database_url)}): {err}\n"
            "Проверьте, что служба MySQL запущена, а логин и пароль в DATABASE_URL верны."
        ) from err
    await run_db(run_migrations, settings.database_url)
    factory = await run_db(make_session_factory, settings.database_url)
    session = AiohttpSession(proxy=settings.proxy_url) if settings.proxy_url else None
    bot = Bot(token=settings.bot_token, session=session)
    me = await bot.get_me()
    await bot.set_my_commands(
        [
            BotCommand(command="start", description="Меню"),
            BotCommand(command="rules", description="Правила"),

BotCommand(command="settings", description="Настройки"),
            BotCommand(command="menu", description="Главное меню"),
            BotCommand(command="cancel", description="Отменить ввод и открыть меню"),
        ]
    )
    await bot.set_chat_menu_button(menu_button=MenuButtonCommands())
    logging.info(
        "Администратор: логин=%s, ID=%s", settings.admin_username, settings.admin_id or "не задан"
    )
    dp, runtime = await run_db(
        build_dispatcher,
        bot, me.username, factory, settings,

anchor_file=Path(__file__).resolve().parents[1] / ".ui_anchors.json",
    )
    start_console_control()
    polling_health = PollingHealthMiddleware()
    bot.session.middleware(polling_health)
    ticker = asyncio.create_task(runtime.run_ticker(), name="room-timer")
    health = asyncio.create_task(
        monitor_health(factory, runtime, ticker, polling_health), name="health-monitor",
    )
    try:
        await dp.start_polling(bot)
    finally:
        ticker.cancel()
        health.cancel()
```

```
    await asyncio.gather(ticker, health, return_exceptions=True)

health_path().unlink(missing_ok=True)
    await bot.session.close()
if __name__ == "__main__":
  asyncio.run(main())
__init__.py
__init__.py
confirmations.py
import secrets
import time
from collections import OrderedDict
from dataclasses import dataclass
@dataclass(frozen=True)
class LeaveConfirmation:
  token: str
  room_id: int
  panel_id: int
  message_id: int
  text: str
  expires_at: float
class LeaveConfirmations:
  """Single-use prompts bound to their sender, room, panel and message.
  Restarting intentionally invalidates all outstanding confirmations.
  """
  def __init__(self, ttl=120.0, max_entries=4096, clock=time.monotonic):
    self.ttl = ttl
    self.max_entries = max_entries
    self.clock = clock
    self._items = OrderedDict()
  def get(self, user_id):
    item = self._items.get(user_id)
    return item if item and item.expires_at > self.clock() else None
  def pop(self, user_id):
    return self._items.pop(user_id, None)
  def new_token(self):
    return secrets.token_urlsafe(9)
  def register(self, user_id, room_id, panel_id, message_id, text, token):
    self.pop(user_id)
    while len(self._items) >= self.max_entries:
      self._items.popitem(last=False)
    self._items[user_id] = LeaveConfirmation(token, room_id, panel_id, message_id, text, self.clock() + self.ttl)
  def consume(self, user_id, message_id, token):
    if not isinstance(token, str) or not token.isascii() or len(token) > 64:
      return None
    item = self.get(user_id)
    if item is None or item.message_id != message_id or not secrets.compare_digest(item.token, token):
      return None
    return self.pop(user_id)
emoji.py
from app.database.worker import run_db
import re
from collections.abc import Callable
from aiogram.client.session.middlewares.base import BaseRequestMiddleware
from aiogram.types import (
  InlineKeyboardButton,
  InlineKeyboardMarkup,
  KeyboardButton,
  ReplyKeyboardMarkup,
)
SELECTED = "[x] "
UNSELECTED = "[ ] "
LABEL_EMOJI = {
  "Играть онлайн": "",
  "Один телефон": "",
  "Наборы слов": "",
  "История игр": "",
  "Профиль": "",
  "Рейтинг": "",
  "Моя комната": "",
  "Настройки": "",
  "Часовой пояс": "",
  "Админ": "",
  "Закрыть": "",
  "Назад": "",
  "Вперёд": "",
  "Отмена": "",
  "Создать комнату": "",
  "Войти по коду": "",
  "Вернуться в комнату": "",
  "Тестовые лобби": "",
  "Тестовое лобби на 1 игрока": "",
  "Тестовое лобби на 2 игроков": "",
  "Жалобы на наборы": "",
  "Продолжить игру": "",
  "Начать новую": "",
  "Начать игру": "",
  "Начать раунд": "",
  "Начать обсуждение": "",
  "Следующий раунд": "",
  "Готов": "",
  "Не готов": "",
  "Пригласить": "",
  "Скопировать ссылку": "",
  "Выйти": "",
  "Показать роль": "",
  "Показать": "",
  "Моя роль": "",
  "Закончить ход": "",
  "Закончить ход и предложить голосование": "",
  "Передать ход": "",
  "Передать ход и предложить голосование": "",
  "Пропустить ход игрока": "",
  "Дальше": "",
  "Голосовать": "",
  "Угадать слово": "",
  "Не угадывать": "",
  "Да": "",
  "Да, выйти": "",
  "Другое слово": "",
  "Изменить настройки": "",
  "Завершить игру": "",
  "Темы": "",
  "Темы: выбрать": "",
  "Количество": "",
  "Выбрать все": "",
  "Очистить": "",
  "Готово": "",
  "Готово, назад в лобби": "",
  "Правила игры": "",
  "Мои наборы": "",
  "Готовые наборы": "",
  "Избранное": "",
  "Публичная библиотека": "",
  "Создать набор": "",
  "Импорт набора": "",
  "К разделам": "",
  "Поиск": "",
  "Слова": "",
  "Добавить слова": "",
  "Переименовать": "",
  "Сделать публичным": "",
  "Сделать личным": "",
  "Копировать": "",
  "Экспорт": "",
  "В избранное": "",
  "Убрать из избранного": "☆",
  "Удалить набор": "",
  "Да, удалить": "",
  "Удалить": "",
  "Изменить": "",
  "Пожаловаться": "",
  "К набору": "",
  "Категория": "",
  "Все категории": "",
  "Сбросить фильтры": "",
  "Оставить и снять жалобы": "",
  "Скрыть": "",
  "Мои игры": "",
  "История лобби": "",
  "Игры этого лобби": "",
  "К списку лобби": "",
  "К лобби": "",
  "По очкам": "",
  "По победам": "",
  "Победы за шпиона": "",
  "Процент побед": "",
  "В меню": "",
}
PREFIX_EMOJI = {
  "Сортировка:": "",
  "Порядок тем:": "",
  "Моя роль:": "",
  "Эмодзи на кнопках:": "",
  "Эмодзи:": "",
  "Цветные кнопки:": "",
  "Раундов в игре": "",
  "Секунд на ход": "",
  "Игроков в комнате": "",
}
THEME_EMOJI = {
  "Животные": "",
  "Еда и напитки": "",
  "Города и страны": "",
  "Фильмы и сериалы": "",
  "Видеоигры": "",
  "Технологии": "",
  "Профессии": "",
  "Спорт": "",
}
SUCCESS = {
  "Начать игру", "Начать раунд", "Следующий раунд", "Да", "Готов", "Создать комнату",
  "Создать набор", "Показать роль", "Показать", "Голосовать", "Закончить ход",
  "Закончить ход и предложить голосование", "Готово", "Готово, назад в лобби",
  "Продолжить игру", "Играть онлайн", "Начать обсуждение",
}
DANGER = {
  "Выйти", "Удалить набор", "Да, удалить", "Пожаловаться", "Скрыть", "Удалить",
  "Завершить игру", "Не готов", "Не угадывать",
```

```
}
PRIMARY = {
    "Один телефон", "Наборы слов", "Моя комната", "Угадать слово", "Моя роль", "Настройки",
    "Пригласить", "Скопировать ссылку", "Войти по коду", "Поиск", "Рейтинг", "Профиль", "История игр", "Админ",
    "Начать новую", "Передать ход и предложить голосование",
}
_LEADING = re.compile(r"^[^\w\[(]+")
_COUNT = re.compile(r"\s*\(\d+\)\s*$")
def split_marker(label: str) -> tuple[str | None, str]:
    if label.startswith(SELECTED):
        return "on", label[len(SELECTED) :]
    if label.startswith(UNSELECTED):
        return "off", label[len(UNSELECTED) :]
    return None, label
def strip_decor(text: str) -> str:
    return _LEADING.sub("", text).strip()
def emoji_for(label: str) -> str | None:
    if label in LABEL_EMOJI:
        return LABEL_EMOJI[label]
    for prefix, emoji in PREFIX_EMOJI.items():
        if label.startswith(prefix):
            return emoji
    title = _COUNT.sub("", label)
    for name, emoji in THEME_EMOJI.items():
        if title == name:
            return emoji
    return None
def decorate_text(label: str) -> str:
    marker, rest = split_marker(label)
    if marker == "on":
        return f"🟢 {rest}"
    if marker == "off":
        return f"◯ {rest}"
    emoji = emoji_for(rest)
    return f"{emoji} {rest}" if emoji else label
def style_for(label: str) -> str | None:
    marker, rest = split_marker(label)
    if marker == "on":
        return "primary"
    key = rest
    if key in DANGER:
        return "danger"
    if key in SUCCESS:
        return "success"
    if key in PRIMARY:
        return "primary"
    return None
def decorate_markup(markup, emoji: bool, colors: bool):
    def apply(button, cls):
        data = button.model_dump(exclude_none=True)
        label = data["text"]
        if colors:
            style = style_for(label)
            if style:
                data["style"] = style
        if emoji:
            data["text"] = decorate_text(label)
        return cls(**data)
    if isinstance(markup, InlineKeyboardMarkup):
        return InlineKeyboardMarkup(
            inline_keyboard=[[apply(b, InlineKeyboardButton) for b in row] for row in markup.inline_keyboard]
        )
    if isinstance(markup, ReplyKeyboardMarkup):
        data = markup.model_dump(exclude_none=True)
        data["keyboard"] = [[apply(b, KeyboardButton) for b in row] for row in markup.keyboard]
        return ReplyKeyboardMarkup(**data)
    return markup
class ButtonStyleMiddleware(BaseRequestMiddleware):
    def __init__(self, prefs_for: Callable[[int], object]):
        self._prefs_for = prefs_for
    async def __call__(self, make_request, bot, method):
        markup = getattr(method, "reply_markup", None)
        chat_id = getattr(method, "chat_id", None)
        if isinstance(chat_id, int) and isinstance(markup, (InlineKeyboardMarkup, ReplyKeyboardMarkup)):
            prefs = (await run_db(self._prefs_for, chat_id))
            if prefs.emoji or prefs.colors:
                method = method.model_copy(
                    update={"reply_markup": decorate_markup(markup, prefs.emoji, prefs.colors)}
                )
        return await make_request(bot, method)
errors.py
import logging
from aiogram import Dispatcher
from aiogram.exceptions import (
    TelegramBadRequest,
    TelegramNetworkError,
    TelegramRetryAfter,
    TelegramServerError,
)
from aiogram.filters import ExceptionTypeFilter, Filter
from aiogram.types import ErrorEvent
log = logging.getLogger(__name__)
HARMLESS_BAD_REQUESTS = (
    "query is too old",
    "message is not modified",
    "message to edit not found",
    "message can't be edited",
    "message to delete not found",
)
class HarmlessBadRequest(Filter):
    async def __call__(self, event: ErrorEvent) -> bool:
        text = str(event.exception).lower()
        return isinstance(event.exception, TelegramBadRequest) and any(
            marker in text for marker in HARMLESS_BAD_REQUESTS
        )
async def on_network_error(event: ErrorEvent) -> bool:
    log.warning("Проблема связи с Telegram, событие пропущено: %s", event.exception)
    return True
async def on_harmless_bad_request(event: ErrorEvent) -> bool:
    log.warning("Устаревшее действие, пропущено: %s", event.exception)
    return True
def register_error_handlers(dp: Dispatcher) -> None:
    dp.errors.register(
        on_network_error,
        ExceptionTypeFilter(TelegramNetworkError, TelegramRetryAfter, TelegramServerError),
    )

dp.errors.register(on_harmless_bad_request, HarmlessBadRequest())
__init__.py
admin.py
from app.database.worker import run_db
from aiogram import F, Router
from aiogram.exceptions import TelegramBadRequest
from aiogram.filters import Command, CommandObject
from aiogram.types import CallbackQuery, InlineKeyboardButton, InlineKeyboardMarkup, Message
from app.bot.keyboards.menu import back_menu
from app.bot.tracker import ScreenTracker
from app.bot.ui import present, safe_answer
from app.core.config import Settings
from app.core.security import is_admin
from app.services.admin_service import AdminError, AdminService, parse_duration
from app.services.penalty import describe_ban
router = Router()
BAN_USAGE = "Использование: /ban ID [срок]. Срок: 30 (минут), 2ч, 7д или навсегда. По умолчанию 1д."
UNBAN_USAGE = "Использование: /unban ID [сброс]. Слово «сброс» обнуляет счётчик нарушений."
DELPACK_USAGE = "Использование: /delpack НОМЕР. Номера наборов показывает команда /packs."
def _admin(settings: Settings, user) -> bool:
    return is_admin(settings, user.id, user.username)
async def _cleanup(message: Message) -> None:
    try:
        await message.delete()
    except TelegramBadRequest:
        pass
async def _reply(message: Message, tracker: ScreenTracker, text: str, markup=None) -> None:
    await _cleanup(message)
    await present(message, tracker, text, markup or back_menu())
def _args(command: CommandObject) -> list[str]:
    return (command.args or "").split()
@router.message(Command("ban"), F.chat.type == "private")
async def on_ban(
    message: Message,
    command: CommandObject,
    settings: Settings,
    admin: AdminService,
    tracker: ScreenTracker,
) -> None:
    if not _admin(settings, message.from_user):
        return
    args = _args(command)
    if not args or not args[0].isdigit():
        await _reply(message, tracker, BAN_USAGE)
        return
    target = int(args[0])
    if target == message.from_user.id or target == settings.admin_id:
```

```
        await _reply(message, tracker, "Нельзя заблокировать администратора.")
        return
    try:
        minutes = parse_duration(args[1]) if len(args) > 1 else 1440
        until = (await run_db(admin.ban, target, minutes))
    except (ValueError, AdminError) as err:
        await _reply(message, tracker, f"{err}\n\n{BAN_USAGE}")
        return
    label = "навсегда" if minutes is None else f"на {args[1] if len(args) > 1 else '1д'}"
    await _reply(message, tracker, f"Игрок {target} заблокирован {label}.")
@router.message(Command("unban"), F.chat.type == "private")
async def on_unban(
    message: Message,
    command: CommandObject,
    settings: Settings,
    admin: AdminService,
    tracker: ScreenTracker,
) -> None:
    if not _admin(settings, message.from_user):
        return
    args = _args(command)
    if not args or not args[0].isdigit():
        await _reply(message, tracker, UNBAN_USAGE)
        return
    reset = len(args) > 1 and args[1].lower() in ("сброс", "reset")
    changed = (await run_db(admin.unban, int(args[0]), reset_strikes=reset))
    if changed:
        text = f"Игрок {args[0]} разблокирован" + (", счётчик нарушений обнулён." if reset else ".")
    else:
        text = "У этого игрока нет блокировки."
    await _reply(message, tracker, text)
@router.message(Command("bans"), F.chat.type == "private")
async def on_bans(
    message: Message, settings: Settings, admin: AdminService, tracker: ScreenTracker
) -> None:
    if not _admin(settings, message.from_user):
        return
    rows = (await run_db(admin.list_bans))
    if not rows:
        await _reply(message, tracker, "Активных блокировок нет.")
        return
    lines = [
        f"{r.user_id} ({r.name}): {r.left_text}, причина: {r.reason_text}, нарушений: {r.strikes}"
        for r in rows[:30]
    ]
    await _reply(message, tracker, "Блокировки:\n" + "\n".join(lines))
@router.message(Command("packs"), F.chat.type == "private")
async def on_packs(
    message: Message,
    command: CommandObject,
    settings: Settings,
    admin: AdminService,
    tracker: ScreenTracker,
) -> None:
    if not _admin(settings, message.from_user):
        return
    packs, total = (await run_db(admin.list_packs, command.args, limit=15))
    if not packs:
        await _reply(message, tracker, "Наборы не найдены.")
        return
    text = "Наборы (номер: название):\n" + "\n".join(p.line() for p in packs)
    if total > len(packs):
        text += f"\n\nПоказано {len(packs)} из {total}. Уточните поиск: /packs текст"
    await _reply(message, tracker, text + "\n\nУдалить: /delpack НОМЕР")
@router.message(Command("delpack"), F.chat.type == "private")
async def on_delpack(
    message: Message,
    command: CommandObject,
    settings: Settings,
    admin: AdminService,
    tracker: ScreenTracker,
) -> None:
    if not _admin(settings, message.from_user):
        return
    args = _args(command)
    if not args or not args[0].isdigit():
        await _reply(message, tracker, DELPACK_USAGE)
        return
    try:
        pack = (await run_db(admin.get_pack, int(args[0])))
    except AdminError as err:
        await _reply(message, tracker, str(err))
        return
    markup = InlineKeyboardMarkup(
        inline_keyboard=[
            [InlineKeyboardButton(text="Да, удалить", callback_data=f"adm:del:{pack.id}")],
            [InlineKeyboardButton(text="Отмена", callback_data="menu:home")],
        ]
    )
    await _reply(
        message,
        tracker,
        f"Удалить набор?\n\n{pack.line()}\n\nЭто действие нельзя отменить.",
        markup,
    )
@router.callback_query(F.data.startswith("adm:del:"))
async def on_delpack_confirm(call: CallbackQuery, settings: Settings, admin: AdminService) -> None:
    if not _admin(settings, call.from_user):
        await safe_answer(call, "Недоступно", show_alert=True)
        return
    raw = call.data.rsplit(":", 1)[1]
    if not raw.isdigit():
        await safe_answer(call)
        return
    try:
        pack = (await run_db(admin.delete_pack, int(raw)))
    except AdminError as err:
        await safe_answer(call, str(err), show_alert=True)
        return
    try:
        await call.message.edit_text(f"Набор «{pack.title}» удалён.", reply_markup=back_menu())
    except TelegramBadRequest:
        pass
    await safe_answer(call, "Удалено")
history.py
from app.database.worker import run_db
from aiogram import F, Router
from aiogram.fsm.context import FSMContext
from aiogram.exceptions import TelegramBadRequest
from aiogram.types import CallbackQuery, InlineKeyboardButton, InlineKeyboardMarkup
from app.bot import history_view as view
from app.bot.ui import safe_answer
from app.game.session import Screen
from app.services.history_service import HistoryService
from app.services.settings_service import SettingsService
router = Router()
def _markup(screen: Screen) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [InlineKeyboardButton(text=label, callback_data=action) for label, action in row]
            for row in screen.rows
        ]
    )
async def _show(call: CallbackQuery, screen: Screen) -> None:
    try:
        await call.message.edit_text(screen.text, reply_markup=_markup(screen))
    except TelegramBadRequest as err:
        if "not modified" not in str(err):
            raise
    await safe_answer(call)
def _int(value: str, default: int = 0) -> int:
    return int(value) if value.isdigit() else default
@router.callback_query(F.data == "hist:menu")
async def on_menu(call: CallbackQuery, state: FSMContext) -> None:
    await state.clear()
    await _show(call, view.menu_screen())
@router.callback_query(F.data.startswith("hist:games:"))
async def on_games(
    call: CallbackQuery, history: HistoryService, user_settings: SettingsService, state: FSMContext
) -> None:
    await state.clear()
    page = _int(call.data.split(":")[2])
    rows, total = (await run_db(history.user_rounds, call.from_user.id, page, view.PAGE_SIZE))
    tz = (await run_db(user_settings.tz_minutes, call.from_user.id))
    await _show(call, view.games_screen(rows, total, page, tz))
@router.callback_query(F.data.startswith("hist:lobbies:"))
async def on_lobbies(
    call: CallbackQuery, history: HistoryService, user_settings: SettingsService, state: FSMContext
```

```
) -> None:
    await state.clear()
    page = _int(call.data.split(":")[2])
    rows, total = (await run_db(history.user_lobbies, call.from_user.id, page, view.PAGE_SIZE))
    tz = (await run_db(user_settings.tz_minutes, call.from_user.id))
    await _show(call, view.lobbies_screen(rows, total, page, tz))
@router.callback_query(F.data.startswith("hist:lobby:"))
async def on_lobby(
    call: CallbackQuery, history: HistoryService, user_settings: SettingsService, state: FSMContext
) -> None:
    await state.clear()
    session_id = _int(call.data.split(":")[2], -1)
    row = (await run_db(history.lobby, call.from_user.id, session_id))
    if row is None:
        await safe_answer(call, "История этого лобби недоступна", show_alert=True)
        return
    await _show(call, view.lobby_screen(row, (await run_db(user_settings.tz_minutes, call.from_user.id))))
@router.callback_query(F.data.startswith("hist:lr:"))
async def on_lobby_rounds(
    call: CallbackQuery, history: HistoryService, user_settings: SettingsService, state: FSMContext
) -> None:
    await state.clear()
    parts = call.data.split(":")
    session_id = _int(parts[2], -1)
    page = _int(parts[3]) if len(parts) > 3 else 0
    result = (await run_db(history.lobby_rounds, call.from_user.id, session_id, page, view.PAGE_SIZE))
    if result is None:
        await safe_answer(call, "История этого лобби недоступна", show_alert=True)
        return
    rows, total = result
    tz = (await run_db(user_settings.tz_minutes, call.from_user.id))
    await _show(call, view.lobby_rounds_screen(session_id, rows, total, page, tz))
from app.bot.handlers.history_management import router as management_router
router.include_router(management_router)
history_management.py
from app.database.worker import run_db
from aiogram import F, Router
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
from aiogram.types import CallbackQuery, Message
from app.bot.keyboards.reply import ALL_BUTTONS
from app.bot.keyboards.menu import back_menu
from app.bot.tracker import ScreenTracker
from app.bot.ui import present, safe_answer
from app.core.config import Settings
from app.core.security import is_admin
from app.game.session import Screen
from app.services.admin_service import AdminError, AdminService
from app.services.history_service import HistoryService
from app.services.history_admin_service import HistoryAdminService
from app.services.pack_service import PackError, PackService
from app.services.penalty import describe_ban
router = Router()
class HistoryEdit(StatesGroup):
    value = State()
def _admin(settings, user):
    return is_admin(settings, user.id, user.username)
def _back(rid):
    return [("К раунду", f"hist:rd:{rid}")]
def _round_screen(detail):
    rid = detail["id"]
    players = "\n".join(f"{name}: {score:+d}" for _, name, score, _ in detail["players"])
    pack = detail["pack_title"] or detail["theme"]
    rows = [[(name[:50], f"hist:player:{rid}:{uid}")] for uid, name, _, _ in detail["players"]]
    rows += [[(f"Набор: {pack}"[:60], f"hist:pack:{rid}")], [("К играм лобби", f"hist:lr:{detail['session_id']}:0")], [("Закрыть", "menu:home")]]
    return Screen(f"Раунд {detail['round_no']}\n\nНабор: {pack}\nСлово: {detail['word']}\n\nИгроки и очки:\n{players}\n\nНажмите имя игрока или набор, чтобы открыть карточку.", rows)
def _player_screen(detail, target, admin, privileged, settings, actor):
    player = next((p for p in detail["players"] if p[0] == target), None)
    if player is None:
        raise AdminError("Игрок не участвовал в этом раунде")
    rid = detail["id"]
    text = f"Игрок: {player[1]}\nTelegram ID: {target}"
    rows = []
    if privileged:
        info = admin.user_info(target)
        text += f"\n\nБлокировка: {describe_ban(info['ban_seconds'])}\nРаундов: {info['rounds']}\nСоздано наборов: {info['packs']}"
        if target not in (actor, settings.admin_id):
            for label, duration in [("Бан на 1 час", "60"), ("Бан на 1 день", "1440"), ("Бан на 7 дней", "10080"), ("Бан навсегда", "perm")]:
                rows.append([(label, f"hist:admin:ban:{rid}:{target}:{duration}")])
            rows.append([("Снять блокировку", f"hist:admin:unban:{rid}:{target}")])
    rows += [_back(rid), [("Закрыть", "menu:home")]]
    return Screen(text, rows)
def _pack_screen(detail, factory, privileged, packs, actor):
    rid = detail["id"]
    historical = detail["pack_title"] or detail["theme"]
    pid = detail["pack_id"]
    if pid is None:
        return Screen(f"Набор в истории: {historical}\n\nВ старом раунде точный ID набора не сохранялся. Управление недоступно, чтобы не изменить чужой набор с похожим названием.", [_back(rid)])
    if not privileged:
        try:
            info = packs.info(actor, pid)
            text = f"Набор в раунде: {historical}\nСейчас: {info.title}\nСлов: {info.words_count}"
        except PackError:
            text = f"Набор в раунде: {historical}\nСейчас набор удалён или недоступен."
        return Screen(text, [_back(rid)])
    try:
        p = HistoryAdminService(factory).pack(pid)
    except AdminError as err:
        return Screen(f"Набор в истории: {historical}\nID: {pid}\n\n{err}", [_back(rid)])
    status = "заблокирован" if p["hidden"] else ("публичный" if p["public"] else "приватный")
    words = ", ".join(p["words"][:15])
    text = f"Набор: {p['title']}\nНазвание в истории: {historical}\nID: {pid}\nАвтор ID: {p['owner'] if p['owner'] is not None else 'встроенный набор'}\nСтатус: {status}\nСлов: {len(p['words'])}\n\nПримеры: {words}"
    rows = [[("Изменить название", f"hist:admin:rename:{rid}")], [("Заменить слова", f"hist:admin:words:{rid}")], [("Сделать приватным" if p["public"] else "Сделать публичным", f"hist:admin:{'private' if p['public'] else 'public'}:{rid}")], [("Снять блокировку набора" if p["hidden"] else "Заблокировать набор", f"hist:admin:{'restore' if p['hidden'] else 'hide'}:{rid}")], [("Удалить набор", f"hist:admin:delete:{rid}")], _back(rid), [("Закрыть", "menu:home")]]
    return Screen(text[:3800], rows)
async def _show(call, screen):
    from app.bot.handlers.history import _show as show
    await show(call, screen)
@router.callback_query(F.data.startswith("hist:rd:"))
async def on_round(call: CallbackQuery, history: HistoryService, state: FSMContext):
    await state.clear()
    try:
        detail = (await run_db(history.round_detail, call.from_user.id, int(call.data.split(":")[2])))
    except (ValueError, IndexError):
        detail = None
    if detail is None:
        return await safe_answer(call, "Раунд недоступен", show_alert=True)
    await _show(call, _round_screen(detail))
@router.callback_query(F.data.startswith("hist:sp:"))
@router.callback_query(F.data.startswith("hist:player:"))
```

```
async def on_player(call: CallbackQuery, history: HistoryService, factory, settings: Settings, state: FSMContext):
    await state.clear()
    try:
        _, kind, context, target = call.data.split(":")
        rid = (await run_db(history.player_round, call.from_user.id, int(context), int(target))) if kind == "sp" else int(context)
        detail = (await run_db(history.round_detail, call.from_user.id, rid)) if rid else None
        if detail is None:
            raise AdminError("Игрок или история недоступны")
        screen = (await run_db(_player_screen, detail, int(target), AdminService(factory), _admin(settings, call.from_user), settings, call.from_user.id))
    except (ValueError, AdminError) as err:
        return await safe_answer(call, str(err), show_alert=True)
    await _show(call, screen)
@router.callback_query(F.data.startswith("hist:pack:"))
async def on_pack(call: CallbackQuery, history: HistoryService, factory, settings: Settings, packs: PackService, state: FSMContext):
    await state.clear()
    try:
        detail = (await run_db(history.round_detail, call.from_user.id, int(call.data.split(":")[2])))
    except (ValueError, IndexError):
        detail = None
    if detail is None:
        return await safe_answer(call, "Раунд недоступен", show_alert=True)
    await _show(call, (await run_db(_pack_screen, detail, factory, _admin(settings, call.from_user), packs, call.from_user.id)))
@router.callback_query(F.data.startswith("hist:admin:"))
async def on_admin_action(call: CallbackQuery, history: HistoryService, factory, settings: Settings, packs: PackService, state: FSMContext):
    if not _admin(settings, call.from_user):
        return await safe_answer(call, "Только для администратора", show_alert=True)
    await state.clear()
    try:
        parts = call.data.split(":")
        action, rid = parts[2], int(parts[3])
        detail = (await run_db(history.round_detail, call.from_user.id, rid))
        if detail is None:
            raise AdminError("Раунд недоступен")
        if action in ("ban", "banok", "unban"):
            target = int(parts[4])
            if not any(p[0] == target for p in detail["players"]):
                raise AdminError("Игрок не участвовал в раунде")
            if action != "unban" and target in (call.from_user.id, settings.admin_id):
                raise AdminError("Нельзя заблокировать администратора")
            admin = AdminService(factory)
            if action == "ban":
                duration = parts[5]
                if duration not in ("60", "1440", "10080", "perm"):
                    raise AdminError("Недопустимый срок")
                label = {"60": "на 1 час", "1440": "на 1 день", "10080": "на 7 дней", "perm": "навсегда"}[duration]
                return await _show(call, Screen(f"Заблокировать игрока {target} {label}?\nБлокировка запретит вход в новые онлайн-комнаты.", [[("Подтвердить бан", f"hist:admin:banok:{rid}:{target}:{duration}")], [("Отмена", f"hist:player:{rid}:{target}")]]))
            if action == "banok":
                if parts[5] not in ("60", "1440", "10080", "perm"):
                    raise AdminError("Недопустимый срок")
                (await run_db(admin.ban, target, None if parts[5] == "perm" else int(parts[5])))
            else:
                (await run_db(admin.unban, target))
            return await _show(call, (await run_db(_player_screen, detail, target, admin, True, settings, call.from_user.id)))
        pid = detail["pack_id"]
        if pid is None:
            raise AdminError("В этом старом раунде ID набора неизвестен")
        manager = HistoryAdminService(factory)
        if action in ("rename", "words"):
            (await run_db(manager.pack, pid))
            await state.set_state(HistoryEdit.value)
            await state.set_data({"round_id": rid, "action": action})
            prompt = "Введите новое название набора." if action == "rename" else "Отправьте полный новый список слов через запятую или с новой строки. Он ЗАМЕНИТ все текущие слова."
            return await _show(call, Screen(prompt, [[("Отмена", f"hist:pack:{rid}")], [("Закрыть", "menu:home")]]))
        if action == "delete":
            p = (await run_db(manager.pack, pid))
            return await _show(call, Screen(f"Удалить набор «{p['title']}» (ID {pid})?\nСлова будут удалены без возможности восстановления. История игр сохранится.", [[("Да, удалить", f"hist:admin:deleteok:{rid}")], [("Отмена", f"hist:pack:{rid}")]]))
        if action not in ("deleteok", "private", "public", "hide", "restore"):
            raise AdminError("Неизвестное действие")
        (await run_db(manager.update, pid, "delete" if action == "deleteok" else action))
        await _show(call, (await run_db(_pack_screen, detail, factory, True, packs, call.from_user.id)))
    except (ValueError, IndexError, AdminError, PackError) as err:
        await safe_answer(call, str(err), show_alert=True)
@router.message(HistoryEdit.value, F.text, ~F.text.startswith("/"), ~F.text.in_(ALL_BUTTONS), F.chat.type == "private")
async def on_edit_value(message: Message, state: FSMContext, history: HistoryService, factory, settings: Settings, packs: PackService, tracker: ScreenTracker):
    if not _admin(settings, message.from_user):
        await state.clear()
        return
    data = await state.get_data()
    rid = data.get("round_id")
    detail = (await run_db(history.round_detail, message.from_user.id, rid)) if rid else None
    if detail is None or detail["pack_id"] is None:
        await state.clear()
        return await present(message, tracker, "Запись или набор недоступны.", back_menu())
    from app.bot.handlers.history import _markup
    from app.bot.ui import delete_quietly
    await delete_quietly(message.bot, message.chat.id, message.message_id)
    try:
        (await run_db(HistoryAdminService(factory).update, detail["pack_id"], data["action"], message.text))
    except (AdminError, PackError) as err:
        screen = Screen(f"{err}\nПовторите ввод или отмените действие.", [[("Отмена", f"hist:pack:{rid}")], [("Закрыть", "menu:home")]])
        return await present(message, tracker, screen.text, _markup(screen))
    await state.clear()
    screen = (await run_db(_pack_screen, detail, factory, True, packs, message.from_user.id))
    await present(message, tracker, screen.text, _markup(screen))
```

local.py

```
from app.database.worker import run_db
import asyncio
from weakref import WeakValueDictionary
from aiogram import Bot, F, Router
from aiogram.exceptions import TelegramBadRequest
from aiogram.fsm.context import FSMContext
from aiogram.types import CallbackQuery, Message
from app.bot.keyboards.menu import RULES_TEXT, back_menu
from app.bot.tracker import ScreenTracker
from app.bot.ui import delete_quietly, safe_answer
from app.bot.keyboards.render import to_markup
from app.bot.keyboards.reply import ALL_BUTTONS
from app.database.repository import LocalGameRepository
from app.game.packs import PackLibrary
from app.services.pack_service import PackService
from app.game.session import LocalSession
router = Router()
_locks = WeakValueDictionary()
def _lock(user_id: int) -> asyncio.Lock:
    return _locks.setdefault(user_id, asyncio.Lock())
def _load(
    repo: LocalGameRepository, library: PackLibrary, user_id: int
) -> LocalSession | None:
    data = repo.load(user_id)
```

```
    return LocalSession.from_dict(data, library=library) if data else None
async def _safe_edit(call: CallbackQuery, text: str, markup) -> None:
    try:
        await call.message.edit_text(text, reply_markup=markup)
    except TelegramBadRequest as err:
        if "not modified" not in str(err):
            raise
@router.callback_query(F.data == "menu:home")
async def on_home(call: CallbackQuery, state: FSMContext, tracker: ScreenTracker) -> None:
    uid = call.from_user.id
    current = tracker.get(uid)
    if current is not None and current != call.message.message_id:
        await delete_quietly(call.bot, call.message.chat.id, call.message.message_id)
        await safe_answer(call)
        return
    await state.clear()
    tracker.discard(uid, call.message.message_id)
    await delete_quietly(call.bot, call.message.chat.id, call.message.message_id)
    await safe_answer(call)
@router.callback_query(F.data == "menu:rules")
async def on_rules(call: CallbackQuery) -> None:
    await _safe_edit(call, RULES_TEXT, back_menu())
    await safe_answer(call)
@router.callback_query(F.data == "menu:local")
async def on_new_local(
    call: CallbackQuery, repo: LocalGameRepository, packs: PackService, state: FSMContext
) -> None:
    await state.clear()
    async with _lock(call.from_user.id):
        previous = (await run_db(repo.load, call.from_user.id))
        if previous and previous.get("message_id") != call.message.message_id and previous.get("message_id"):
            await delete_quietly(call.bot, call.message.chat.id, previous["message_id"])
        session = LocalSession(library=(await run_db(packs.library_for, call.from_user.id)))
        session.message_id = call.message.message_id
        (await run_db(repo.save, call.from_user.id, session.to_dict()))
    await _safe_edit(call, session.screen().text, to_markup(session.screen(), session.seq))
    await safe_answer(call)
@router.callback_query(F.data == "menu:resume")
async def on_resume(
    call: CallbackQuery,
    repo: LocalGameRepository,
    packs: PackService,
    state: FSMContext,
) -> None:
    await state.clear()
    async with _lock(call.from_user.id):
        session = (await run_db(_load, repo, (await run_db(packs.library_for, call.from_user.id)), call.from_user.id))
        if session is None:
            await safe_answer(call, "Сохранённой игры нет", show_alert=True)
            await delete_quietly(call.bot, call.message.chat.id, call.message.message_id)
            return
        if session.message_id and session.message_id != call.message.message_id:
            await delete_quietly(call.bot, call.message.chat.id, session.message_id)
        session.neutralize()
        session.message_id = call.message.message_id
        (await run_db(repo.save, call.from_user.id, session.to_dict()))
    screen = session.screen()
    await _safe_edit(call, screen.text, to_markup(screen, session.seq))
    await safe_answer(call)
@router.callback_query(F.data.startswith("lg:"))
async def on_action(
    call: CallbackQuery,
    repo: LocalGameRepository,
    packs: PackService,
) -> None:
    try:
        _, seq, action = call.data.split(":", 2)
        seq_number = int(seq)
    except ValueError:
        await safe_answer(call)
        return
    async with _lock(call.from_user.id):
        session = (await run_db(_load, repo, (await run_db(packs.library_for, call.from_user.id)), call.from_user.id))
        if session is None:
            await safe_answer(call, "Игра не найдена", show_alert=True)
            await delete_quietly(call.bot, call.message.chat.id, call.message.message_id)
            return
        if session.message_id and session.message_id != call.message.message_id:
            await safe_answer(call, "Это старый экран. Откройте текущую игру.", show_alert=True)
            return
        response = session.act(seq_number, action)
        if response.exit:
            (await run_db(repo.delete, call.from_user.id))
        else:
            session.message_id = call.message.message_id
            (await run_db(repo.save, call.from_user.id, session.to_dict()))
        seq_now = session.seq
    if response.exit:
        await delete_quietly(call.bot, call.message.chat.id, call.message.message_id)
        await safe_answer(call)
        return
    if response.screen is not None:
        await _safe_edit(call, response.screen.text, to_markup(response.screen, seq_now))
    await safe_answer(call, response.alert, show_alert=bool(response.alert))
@router.message(
    F.text, ~F.text.startswith("/"), ~F.text.in_(ALL_BUTTONS), F.chat.type == "private"
)
async def on_text(
    message: Message, bot: Bot, repo: LocalGameRepository, packs: PackService, tracker: ScreenTracker
) -> None:
    if tracker.get(message.from_user.id) is not None:
        return
    async with _lock(message.from_user.id):
        session = (await run_db(_load, repo, (await run_db(packs.library_for, message.from_user.id)), message.from_user.id))
        if session is None:
            return
        response = session.handle_text(message.text)
        if response is None or response.screen is None:
            return
        (await run_db(repo.save, message.from_user.id, session.to_dict()))
        message_id = session.message_id
        seq_now = session.seq
    try:
        await message.delete()
    except TelegramBadRequest:
        pass
    if message_id is None:
        return
    try:
        await bot.edit_message_text(
            response.screen.text,
            chat_id=message.chat.id,
            message_id=message_id,

reply_markup=to_markup(response.screen, seq_now),
        )
    except TelegramBadRequest as err:
        if "not modified" not in str(err):
            raise
```

menu.py

```
from app.database.worker import run_db
import logging
from aiogram import Bot, F, Router
from aiogram.exceptions import TelegramAPIError, TelegramBadRequest
from aiogram.filters import Command, CommandStart
from aiogram.fsm.context import FSMContext
from sqlalchemy.orm import sessionmaker
from aiogram.types import Message
from app.bot import history_view, pack_view, settings_view, stats_view
from app.bot.handlers.history import _markup as history_markup
from app.bot.keyboards.menu import (
    MENU_TEXT,
    online_text,
    RULES_TEXT,
    admin_lobby_menu,
    back_menu,
    local_choice_menu,
    online_menu,
)
from app.bot.keyboards.render import to_markup, to_plain_markup
from app.bot.keyboards.reply import (
    BTN_ADMIN,
```

```
    BTN_HISTORY,
    BTN_LOCAL,
    BTN_ONLINE,
    BTN_PACKS,
    BTN_PROFILE,
    BTN_RATING,
    BTN_ROOM,
    BTN_SETTINGS,
    main_reply_keyboard,
)
from app.bot.runtime import OnlineRuntime
from app.bot.tracker import ScreenTracker
from app.bot.ui import clear_tracked, delete_quietly, present
from app.core.config import Settings
from app.core.security import is_admin
from app.database.info import describe, format_info
from app.database.repository import LocalGameRepository
from app.game.session import LocalSession
from app.services.pack_service import PackService
from app.services.settings_service import SettingsService
from app.services.stats_service import StatsService
log = logging.getLogger(__name__)
router = Router()
def _admin(settings: Settings, message: Message) -> bool:
    return is_admin(settings, message.from_user.id, message.from_user.username)
async def _cleanup(message: Message) -> None:
    try:
        await message.delete()
    except TelegramBadRequest:
        pass
@router.message(Command("menu", "cancel"), F.chat.type == "private")
@router.message(CommandStart(), F.chat.type == "private")
async def on_start(
    message: Message,
    bot: Bot,
    settings: Settings,
    state: FSMContext,
    tracker: ScreenTracker,
    repo: LocalGameRepository,
    runtime: OnlineRuntime,
) -> None:
    await state.clear()
    await clear_tracked(bot, message.chat.id, message.from_user.id, tracker)
    await _cleanup(message)
    previous = tracker.pop_anchor(message.from_user.id)
    if previous:
        await delete_quietly(bot, message.chat.id, previous)
    sent = await message.answer(MENU_TEXT, reply_markup=main_reply_keyboard(_admin(settings, message)))

    tracker.set_anchor(message.from_user.id, sent.message_id)
@router.message(Command("rules"), F.chat.type == "private")
async def on_rules_command(message: Message, state: FSMContext, tracker: ScreenTracker) -> None:
    await state.clear()
    await _cleanup(message)
    await present(message, tracker, RULES_TEXT, back_menu())
@router.message(Command("whoami"), F.chat.type == "private")
async def on_whoami(message: Message, settings: Settings) -> None:
    user = message.from_user
    admin = "да" if _admin(settings, message) else "нет"
    username = f"@{user.username}" if user.username else "не задан"
    await message.answer(f"Ваш ID: {user.id}\nЛогин: {username}\nАдминистратор: {admin}")
@router.message(F.text == BTN_ONLINE, F.chat.type == "private")
async def on_online(
    message: Message,
    runtime: OnlineRuntime,
    settings: Settings,
    state: FSMContext,
    tracker: ScreenTracker,
) -> None:
    await state.clear()
    await _cleanup(message)
    await present(
        message,
        tracker,
        online_text(*(await run_db(runtime.service.ban_status, message.from_user.id))),
        online_menu((await run_db(runtime.in_room, message.from_user.id)), _admin(settings, message)),
    )
@router.message(F.text == BTN_LOCAL, F.chat.type == "private")
async def on_local(
    message: Message,
    repo: LocalGameRepository,
    packs: PackService,
    state: FSMContext,
    tracker: ScreenTracker,
) -> None:
    await state.clear()
    await _cleanup(message)
    uid = message.from_user.id
    if (await run_db(repo.load, uid)) is not None:
        await present(
            message,
            tracker,
            "У вас есть незавершённая игра на одном устройстве.",
            local_choice_menu(),
        )
        return
    await clear_tracked(message.bot, message.chat.id, uid, tracker)
    session = LocalSession(library=(await run_db(packs.library_for, uid)))
    screen = session.screen()
    sent = await message.answer(screen.text, reply_markup=to_markup(screen, session.seq))
    session.message_id = sent.message_id
    (await run_db(repo.save, uid, session.to_dict()))
@router.message(F.text == BTN_HISTORY, F.chat.type == "private")
async def on_history(message: Message, state: FSMContext, tracker: ScreenTracker) -> None:
    await state.clear()
    await _cleanup(message)
    screen = history_view.menu_screen()
    await present(message, tracker, screen.text, history_markup(screen))
@router.message(F.text == BTN_ROOM, F.chat.type == "private")
async def on_room(
    message: Message, runtime: OnlineRuntime, state: FSMContext, tracker: ScreenTracker
) -> None:
    await state.clear()
    await _cleanup(message)
    uid = message.from_user.id
    room_id = (await run_db(runtime.service.room_id_of, uid))
    if room_id is None:
        await present(message, tracker, "Вы сейчас не в комнате. Нажмите «Играть онлайн».", back_menu())
        return
    await clear_tracked(message.bot, message.chat.id, uid, tracker)
    panel = await message.answer("Подключаюсь к комнате...")
    async with runtime.lock(room_id):
        await runtime.adopt_panel(room_id, uid, panel.message_id)
        await runtime.refresh(room_id)
async def _show_admin(
    message: Message, settings: Settings, state: FSMContext, tracker: ScreenTracker
) -> None:
    await state.clear()
    await _cleanup(message)
    if not _admin(settings, message):
        return
    await present(
        message,
        tracker,
        "Администрирование.\n\nТестовые лобби позволяют начать раунд с 1 или 2 игроками "
        "и выбрать свою роль. Жалобы: разбор публичных наборов.\n\n"
        "Команды:\n"
        "/ban ID [срок] - заблокировать игрока (30, 2ч, 7д или навсегда)\n"
        "/unban ID [сброс] - разблокировать\n"
        "/bans - список блокировок\n"
        "/packs [текст] - найти наборы\n"
        "/delpack НОМЕР - удалить набор",
        admin_lobby_menu(),
    )
@router.message(F.text == BTN_ADMIN, F.chat.type == "private")
async def on_admin_button(
    message: Message, settings: Settings, state: FSMContext, tracker: ScreenTracker
) -> None:
    await _show_admin(message, settings, state, tracker)
@router.message(Command("admin"), F.chat.type == "private")
async def on_admin_command(
    message: Message, settings: Settings, state: FSMContext, tracker: ScreenTracker
) -> None:
    await _show_admin(message, settings, state, tracker)
@router.message(F.text == BTN_PACKS, F.chat.type == "private")
async def on_packs(message: Message, state: FSMContext, tracker: ScreenTracker) -> None:
    await state.clear()
    await _cleanup(message)
    screen = pack_view.menu_screen()
```

```
    await present(message, tracker, screen.text, to_plain_markup(screen))
@router.message(F.text == BTN_PROFILE, F.chat.type == "private")
async def on_profile(
    message: Message, stats: StatsService, state: FSMContext, tracker: ScreenTracker
) -> None:
    await state.clear()
    await _cleanup(message)
    name = message.from_user.first_name or message.from_user.username or "Игрок"
    screen = stats_view.profile_screen((await run_db(stats.profile, message.from_user.id)), name)
    await present(message, tracker, screen.text, to_plain_markup(screen))
@router.message(F.text == BTN_RATING, F.chat.type == "private")
async def on_rating(
    message: Message, stats: StatsService, state: FSMContext, tracker: ScreenTracker
) -> None:
    await state.clear()
    await _cleanup(message)
    screen = stats_view.rating_screen((await run_db(stats.leaderboard, "points")), "points", message.from_user.id)
    await present(message, tracker, screen.text, to_plain_markup(screen))
async def _show_settings(
    message: Message, user_settings: SettingsService, state: FSMContext, tracker: ScreenTracker
) -> None:
    await state.clear()
    await _cleanup(message)
    screen = settings_view.settings_screen((await run_db(user_settings.prefs, message.from_user.id)))
    await present(message, tracker, screen.text, to_plain_markup(screen))
@router.message(F.text == BTN_SETTINGS, F.chat.type == "private")
async def on_settings_button(
    message: Message, user_settings: SettingsService, state: FSMContext, tracker: ScreenTracker
) -> None:
    await _show_settings(message, user_settings, state, tracker)
@router.message(Command("settings"), F.chat.type == "private")
async def on_settings_command(
    message: Message, user_settings: SettingsService, state: FSMContext, tracker: ScreenTracker
) -> None:
    await _show_settings(message, user_settings, state, tracker)
@router.message(Command("dbinfo"), F.chat.type == "private")
async def on_dbinfo(message: Message, settings: Settings, factory: sessionmaker) -> None:
    await _cleanup(message)
    if not _admin(settings, message):
        return
    info = await run_db(describe, factory, settings.database_url)
    await message.answer(format_info(info))
```

online.py

```
from app.database.worker import run_db
import asyncio
import time
from aiogram import F, Router
from aiogram.dispatcher.event.bases import SkipHandler
from aiogram.exceptions import TelegramBadRequest
from aiogram.filters import CommandObject, CommandStart, Filter
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
from aiogram.types import CallbackQuery, InlineKeyboardButton, InlineKeyboardMarkup, Message, User
from app.bot.keyboards.menu import admin_lobby_menu, online_menu, online_text
from app.bot.keyboards.reply import ALL_BUTTONS
from app.bot.runtime import OnlineRuntime
from app.bot.tracker import ScreenTracker
from app.bot.ui import clear_tracked, delete_quietly, present, safe_answer
from app.services.penalty import format_duration
from app.core.config import Settings
from app.core.security import is_admin
from app.game.room import RoomError
router = Router()
MESSAGE_COOLDOWN = 1.0
class JoinStates(StatesGroup):
    code = State()
class InRoom(Filter):
    async def __call__(self, message: Message, runtime: OnlineRuntime) -> bool:
        return (await run_db(runtime.in_room, message.from_user.id))
def display_name(user: User) -> str:
    return user.first_name or user.username or "Игрок"
async def _edit(call: CallbackQuery, text: str, markup) -> None:
    try:
        await call.message.edit_text(text, reply_markup=markup)
    except TelegramBadRequest as err:
        if "not modified" not in str(err):
            raise
async def _delete_later(message: Message, delay: float = 4.0) -> None:
    await asyncio.sleep(delay)
    try:
        await message.delete()
    except TelegramBadRequest:
        pass
async def _safe_delete(message: Message) -> None:
    try:
        await message.delete()
    except TelegramBadRequest:
        pass
async def _enter_room(
    message: Message,
    runtime: OnlineRuntime,
    room_id: int,
    user_id: int,
    tracker: ScreenTracker,
    settings: Settings,
) -> None:
    await clear_tracked(message.bot, message.chat.id, user_id, tracker)
    panel = await message.answer("Подключаюсь к комнате...")
    async with runtime.lock(room_id):
        await runtime.adopt_panel(room_id, user_id, panel.message_id)
        await runtime.refresh(room_id)
@router.message(CommandStart(deep_link=True), F.chat.type == "private")
async def on_deeplink(
    message: Message,
    command: CommandObject,
    runtime: OnlineRuntime,
    state: FSMContext,
    tracker: ScreenTracker,
    settings: Settings,
) -> None:
    args = command.args or ""
    if not args.startswith("join_"):
        raise SkipHandler
    await state.clear()
    await _safe_delete(message)
    uid = message.from_user.id
    try:
        update = (await run_db(runtime.service.join_token, uid, display_name(message.from_user), args[5:]))
    except RoomError as err:
        await present(
            message, tracker, str(err),
            online_menu((await run_db(runtime.in_room, uid)), is_admin(settings, uid, message.from_user.username)),
        )
        return
    await _enter_room(message, runtime, update.room_id, uid, tracker, settings)
@router.callback_query(F.data == "menu:online")
async def on_online_menu(
    call: CallbackQuery, runtime: OnlineRuntime, state: FSMContext, settings: Settings
) -> None:
    await state.clear()
    admin = is_admin(settings, call.from_user.id, call.from_user.username)
    await _edit(
        call,
        online_text(*(await run_db(runtime.service.ban_status, call.from_user.id))),
        online_menu((await run_db(runtime.in_room, call.from_user.id)), admin),
    )
    await safe_answer(call)
@router.callback_query(F.data == "online:testmenu")
async def on_test_menu(call: CallbackQuery, settings: Settings) -> None:
    if not is_admin(settings, call.from_user.id, call.from_user.username):
        await safe_answer(call, "Недоступно", show_alert=True)
        return
    await _edit(
        call,
        "Тестовые лобби.\n\nПозволяют начать раунд с 1 или 2 игроками и выбрать свою роль.",
        admin_lobby_menu(),
    )
    await safe_answer(call)
@router.callback_query(F.data.startswith("online:test:"))
async def on_create_test(
    call: CallbackQuery, runtime: OnlineRuntime, settings: Settings
) -> None:
    uid = call.from_user.id
```

```
    if not is_admin(settings, uid, call.from_user.username):
        await safe_answer(call, "Недоступно", show_alert=True)
        return
    size = call.data.rsplit(":", 1)[1]
    if size not in ("1", "2"):
        await safe_answer(call)
        return
    try:
        update = (await run_db(runtime.service.create, uid, display_name(call.from_user), test_size=int(size)))
    except RoomError as err:
        await safe_answer(call, str(err), show_alert=True)
        return
    async with runtime.lock(update.room_id):
        await runtime.adopt_panel(update.room_id, uid, call.message.message_id)
        await runtime.refresh(update.room_id)
    await safe_answer(call)
@router.callback_query(F.data == "online:create")
async def on_create(call: CallbackQuery, runtime: OnlineRuntime) -> None:
    uid = call.from_user.id
    try:
        update = (await run_db(runtime.service.create, uid, display_name(call.from_user)))
    except RoomError as err:
        await safe_answer(call, str(err), show_alert=True)
        return
    async with runtime.lock(update.room_id):
        await runtime.adopt_panel(update.room_id, uid, call.message.message_id)
        await runtime.refresh(update.room_id)
    await safe_answer(call)
@router.callback_query(F.data == "online:join")
async def on_join_prompt(call: CallbackQuery, state: FSMContext) -> None:
    await state.set_state(JoinStates.code)
    await _edit(
        call,
        "Введите код комнаты (6 символов) сообщением.",
        online_menu_back(),
    )
    await safe_answer(call)
def online_menu_back():
    return InlineKeyboardMarkup(
        inline_keyboard=[

[InlineKeyboardButton(text="Отмена", callback_data="menu:online")],
            [InlineKeyboardButton(text="В меню", callback_data="menu:home")],
        ]
    )
@router.message(
    JoinStates.code,
    F.text,
    ~F.text.startswith("/"),
    ~F.text.in_(ALL_BUTTONS),
    F.chat.type == "private",
)
async def on_code(
    message: Message, runtime: OnlineRuntime, state: FSMContext, tracker: ScreenTracker, settings: Settings
) -> None:
    uid = message.from_user.id
    await _safe_delete(message)
    try:
        update = (await run_db(runtime.service.join_code, uid, display_name(message.from_user), message.text))
    except RoomError as err:
        await present(message, tracker, f"Ошибка: {err}\nВведите код ещё раз или отмените ввод.", online_menu_back())
        return
    await state.clear()
    await _enter_room(message, runtime, update.room_id, uid, tracker, settings)
@router.callback_query(F.data == "online:resume")
async def on_resume(call: CallbackQuery, runtime: OnlineRuntime) -> None:
    uid = call.from_user.id
    room_id = (await run_db(runtime.service.room_id_of, uid))
    if room_id is None:
        await safe_answer(call, "Вы не в комнате", show_alert=True)
        await delete_quietly(call.bot, call.message.chat.id, call.message.message_id)
        return
    async with runtime.lock(room_id):
        await runtime.adopt_panel(room_id, uid, call.message.message_id)
        await runtime.refresh(room_id)
    await safe_answer(call)
@router.callback_query(F.data == "rm:leave")
async def on_leave_request(call: CallbackQuery, runtime: OnlineRuntime) -> None:
    uid = call.from_user.id
    room_id = (await run_db(runtime.service.room_id_of, uid))
    if room_id is None:
        await safe_answer(call, "Вы не в комнате", show_alert=True)
        return
    try:
        async with runtime.lock(room_id):
            text = (await run_db(runtime.service.leave_confirmation,
                uid, expected_room_id=room_id, expected_panel_id=call.message.message_id,
            ))
            pending = runtime.leave_confirmations.get(uid)
            if pending and (pending.room_id, pending.panel_id, pending.text) == (room_id, call.message.message_id, text):
                await safe_answer(call, "Подтвердите выход или нажмите «Отмена» в открытом сообщении.")
                return
            previous = runtime.leave_confirmations.pop(uid)
            if previous:
                await delete_quietly(call.bot, call.message.chat.id, previous.message_id)
            token = runtime.leave_confirmations.new_token()
            markup = InlineKeyboardMarkup(inline_keyboard=[[
                InlineKeyboardButton(text="Да, выйти", callback_data=f"rm_leave:confirm:{token}"),

InlineKeyboardButton(text="Отмена", callback_data=f"rm_leave:cancel:{token}"),
            ]])
            sent = await call.bot.send_message(call.message.chat.id, text, reply_markup=markup)

runtime.leave_confirmations.register(uid, room_id, call.message.message_id, sent.message_id, text, token)
    except RoomError as err:
        await safe_answer(call, str(err), show_alert=True)
        return
    await safe_answer(call)
@router.callback_query(F.data.startswith("rm_leave:"))
async def on_leave_confirmation(
    call: CallbackQuery, runtime: OnlineRuntime, tracker: ScreenTracker
) -> None:
    uid = call.from_user.id
    parts = (call.data or "").split(":")
    if len(parts) != 3 or parts[1] not in ("confirm", "cancel"):
        await safe_answer(call, "Подтверждение устарело. Нажмите «Выйти» заново.", show_alert=True)
        return
    pending = runtime.leave_confirmations.consume(uid, call.message.message_id, parts[2])
    if pending is None:
        await safe_answer(call, "Подтверждение устарело. Нажмите «Выйти» заново.", show_alert=True)
        return
    if parts[1] == "cancel":
        await delete_quietly(call.bot, call.message.chat.id, pending.message_id)
        await safe_answer(call, "Выход отменён")
        return
    try:
        async with runtime.lock(pending.room_id):
            update = (await run_db(runtime.service.act,
                uid, "leave_confirm", expected_room_id=pending.room_id,

expected_panel_id=pending.panel_id, expected_leave_text=pending.text,
            ))
            if update.changed and update.room is not None:
                await runtime.refresh(pending.room_id)
    except RoomError as err:
        await delete_quietly(call.bot, call.message.chat.id, pending.message_id)
        await safe_answer(call, str(err), show_alert=True)
        return
    await delete_quietly(call.bot, call.message.chat.id, pending.message_id)
    await safe_answer(call, "Вы покинули комнату")
    if update.left_user:
        await delete_quietly(call.bot, call.message.chat.id, pending.panel_id)
        if update.penalty:
```

```
        strikes, minutes = update.penalty
        notice = await call.bot.send_message(
            call.message.chat.id,
            "Вы покинули игру во время раунда. Блокировка онлайн-игр: "
            f"{format_duration(minutes * 60)} (нарушение №{strikes}).",
        )
        tracker.set(uid, notice.message_id)
@router.callback_query(F.data.startswith("rm:"))
async def on_room_action(
    call: CallbackQuery, runtime: OnlineRuntime, tracker: ScreenTracker
) -> None:
    uid = call.from_user.id
    action = call.data[3:]
    if action in ("leave", "leave_confirm"):
        await safe_answer(call, "Нажмите «Выйти» и подтвердите действие.", show_alert=True)
        return
    room_id = (await run_db(runtime.service.room_id_of, uid))
    if room_id is None:
        await safe_answer(call, "Вы не в комнате", show_alert=True)
        await delete_quietly(call.bot, call.message.chat.id, call.message.message_id)
        return
    update = None
    error = ""
    async with runtime.lock(room_id):
        try:
            update = (await run_db(runtime.service.act,
                uid, action, expected_room_id=room_id, expected_panel_id=call.message.message_id,
            ))
        except RoomError as err:
            error = str(err)
    if update is None:
        await safe_answer(call, error, show_alert=True)
        return
    await safe_answer(call, update.alert, show_alert=bool(update.alert))
    if update.left_user:
        await delete_quietly(call.bot, call.message.chat.id, call.message.message_id)
        if update.penalty:
            strikes, minutes = update.penalty
            try:
                notice = await call.bot.send_message(
                    call.message.chat.id,
                    "Вы покинули игру во время раунда. Блокировка онлайн-игр: "
                    f"{format_duration(minutes * 60)} (нарушение №{strikes}).",
                )
                tracker.set(call.from_user.id, notice.message_id)
            except Exception:
                pass
    if update.changed and update.room is not None:
        async with runtime.lock(room_id):
            await runtime.refresh(room_id)
@router.message(
    F.text,
    ~F.text.startswith("/"),
    ~F.text.in_(ALL_BUTTONS),
    F.chat.type == "private",
    InRoom(),
)
async def on_room_text(message: Message, runtime: OnlineRuntime) -> None:
    uid = message.from_user.id
    await _safe_delete(message)
    if not runtime.allow_chat_message(uid, MESSAGE_COOLDOWN):
        return
    room_id = (await run_db(runtime.service.room_id_of, uid))
    if room_id is None:
        return
    async with runtime.lock(room_id):
        try:
            (await run_db(runtime.service.say, uid, message.text))
        except RoomError as err:
            warn = await message.answer(f"Ошибка: {err}")

            asyncio.create_task(_delete_later(warn))
            return
        await runtime.refresh(room_id)
```

```
packs.py
from app.database.worker import run_db
import io
from aiogram import Bot, F, Router
from aiogram.exceptions import TelegramBadRequest
from aiogram.filters import StateFilter
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
from aiogram.types import BufferedInputFile, CallbackQuery, Message, User
from app.bot import pack_view as view
from app.bot.keyboards.render import to_plain_markup
from app.bot.keyboards.reply import ALL_BUTTONS
from app.bot.tracker import ScreenTracker
from app.bot.ui import present, safe_answer
from app.core.config import Settings
from app.core.security import is_admin
from app.game.session import Screen
from app.services.pack_service import PackError, PackService, split_words
router = Router()
MAX_IMPORT_BYTES = 200 * 1024
class PackStates(StatesGroup):
    title = State()
    words = State()
    rename = State()
    find = State()
    edit_word = State()
    import_pack = State()
def display_name(user: User) -> str:
    return user.first_name or user.username or "Игрок"
async def _edit(call: CallbackQuery, screen: Screen) -> None:
    try:
        await call.message.edit_text(screen.text, reply_markup=to_plain_markup(screen))
    except TelegramBadRequest as err:
        if "not modified" not in str(err):
            raise
async def _list_screen(packs: PackService, uid: int, state: FSMContext, scope: str, page: int):
    data = await state.get_data()
    active = data.get("scope") == scope
    query = data.get("query") if active else None
    sort = data.get("sort", "new") if active else "new"
    cat = data.get("cat") if active else None
    infos, total = (await run_db(packs.list_packs, uid, scope, page, view.PAGE_SIZE, query, sort, cat))
    pages = max(1, -(-total // view.PAGE_SIZE))
    if page >= pages and page > 0:
        return await _list_screen(packs, uid, state, scope, pages - 1)
    cat_title = next((t for i, t in (await run_db(packs.categories)) if i == cat), None) if cat else None
    return view.list_screen(scope, infos, total, page, query, sort, cat_title)
async def _set_filters(state: FSMContext, scope: str, **changes) -> None:
    data = await state.get_data()
    if data.get("scope") != scope:
        data = {"scope": scope}
    data.update(changes)
    await state.set_data(data)
@router.callback_query(F.data.startswith("pk:"))
async def on_pack_action(
    call: CallbackQuery, state: FSMContext, packs: PackService, settings: Settings
) -> None:
    parts = call.data.split(":")
    action = parts[1]
    uid = call.from_user.id
    try:
        await _dispatch(call, state, packs, settings, action, parts[2:], uid)
    except PackError as err:
        await safe_answer(call, str(err), show_alert=True)
def _int(value: str, default: int = 0) -> int:
    try:
        return int(value)
    except ValueError:
        return default
async def _dispatch(call, state, packs, settings, action, args, uid) -> None:
    if action == "menu":
        await state.clear()
        await _edit(call, view.menu_screen())
        return await safe_answer(call)
    if action == "list":
        await state.set_state(None)
        scope, page = args[0], _int(args[1])
        await _edit(call, await _list_screen(packs, uid, state, scope, page))
        return await safe_answer(call)
    if action == "find":
        await state.set_state(PackStates.find)
        await state.update_data(find_scope=args[0])
        await _edit(call, view.prompt_screen("Введите текст для поиска по названиям и словам.", f"pk:list:{args[0]}:0"))
        return await safe_answer(call)
    if action == "sort":
        data = await state.get_data()
        current = data.get("sort", "new") if data.get("scope") == args[0] else "new"
        nxt = view.SORT_ORDER[(view.SORT_ORDER.index(current) + 1) % len(view.SORT_ORDER)]
        await _set_filters(state, args[0], sort=nxt)
        await _edit(call, await _list_screen(packs, uid, state, args[0], 0))
```

```
        return await safe_answer(call)
    if action == "catlist":
        await _edit(call, view.category_screen(args[0], (await run_db(packs.categories))))
        return await safe_answer(call)
    if action == "cat":
        await _set_filters(state, args[0], cat=_int(args[1]) or None)
        await _edit(call, await _list_screen(packs, uid, state, args[0], 0))
        return await safe_answer(call)
    if action == "clear":
        await state.set_data({})
        await _edit(call, await _list_screen(packs, uid, state, args[0], 0))
        return await safe_answer(call)
    if action == "view":
        await state.set_state(None)
        await _edit(call, view.card_screen((await run_db(packs.info, uid, _int(args[0]))), uid))
        return await safe_answer(call)
    if action == "words":
        pack_id, page = _int(args[0]), _int(args[1])
        info = (await run_db(packs.info, uid, pack_id))
        words, total = (await run_db(packs.list_words, uid, pack_id, page, view.WORDS_PAGE))
        await _edit(call, view.words_screen(info, words, total, page, info.owner_id == uid))
        return await safe_answer(call)
    if action == "w":
        await state.set_state(None)
        pack_id, word = (await run_db(packs.word, uid, _int(args[0])))
        await _edit(call, view.word_screen(_int(args[0]), word, pack_id))
        return await safe_answer(call)
    if action == "wed":
        pack_id, word = (await run_db(packs.word, uid, _int(args[0])))
        await state.set_state(PackStates.edit_word)
        await state.update_data(word_id=_int(args[0]), pack_id=pack_id)
        await _edit(call, view.prompt_screen(f"Введите новое значение для слова «{word}».", f"pk:w:{args[0]}"))
        return await safe_answer(call)
    if action == "wdel":
        pack_id = (await run_db(packs.delete_word, uid, _int(args[0])))
        info = (await run_db(packs.info, uid, pack_id))
        words, total = (await run_db(packs.list_words, uid, pack_id, 0, view.WORDS_PAGE))
        await _edit(call, view.words_screen(info, words, total, 0, True))
        return await safe_answer(call, "Слово удалено")
    if action == "new":
        await state.set_state(PackStates.title)
        await _edit(call, view.prompt_screen("Введите название нового набора."))
        return await safe_answer(call)
    if action == "add":
        pack_id = _int(args[0])
        (await run_db(packs.info, uid, pack_id))
        await state.set_state(PackStates.words)
        await state.update_data(pack_id=pack_id)
        await _edit(
            call,
            view.done_screen(
                "Отправьте слова сообщением: по одному в строке или через запятую. "
                "Можно отправлять несколько сообщений. Когда закончите, нажмите «Готово».",
                pack_id,
            ),
        )
        return await safe_answer(call)
    if action == "ren":
        pack_id = _int(args[0])
        await state.set_state(PackStates.rename)
        await state.update_data(pack_id=pack_id)
        await _edit(call, view.prompt_screen("Введите новое название набора.", f"pk:view:{pack_id}"))
        return await safe_answer(call)
    if action == "vis":
        pack_id = _int(args[0])
        info = (await run_db(packs.info, uid, pack_id))
        (await run_db(packs.set_public, uid, pack_id, not info.is_public))
        await _edit(call, view.card_screen((await run_db(packs.info, uid, pack_id)), uid))
        return await safe_answer(call, "Набор опубликован" if not info.is_public else "Набор стал личным")
    if action == "copy":
        new_id = (await run_db(packs.copy_pack, uid, display_name(call.from_user), _int(args[0])))
        await _edit(call, view.card_screen((await run_db(packs.info, uid, new_id)), uid))
        return await safe_answer(call, "Копия создана")
    if action == "exp":
        slug, text = (await run_db(packs.export_text, uid, _int(args[0])))
        await call.message.answer_document(
            BufferedInputFile(text.encode("utf-8"), filename=f"{slug}.txt"),
            caption="Файл набора. Его можно загрузить обратно через «Импорт набора».",
        )
        return await safe_answer(call)
    if action == "fav":
        pack_id = _int(args[0])
        state_now = (await run_db(packs.toggle_favorite, uid, pack_id))
        await _edit(call, view.card_screen((await run_db(packs.info, uid, pack_id)), uid))
        return await safe_answer(call, "Добавлено в избранное" if state_now else "Убрано из избранного")
    if action == "del":
        await _edit(call, view.delete_screen((await run_db(packs.info, uid, _int(args[0])))))
        return await safe_answer(call)
    if action == "delyes":
        (await run_db(packs.delete_pack, uid, _int(args[0])))
        await _edit(call, await _list_screen(packs, uid, state, "my", 0))
        return await safe_answer(call, "Набор удалён")
    if action == "rep":
        await _edit(call, view.report_screen((await run_db(packs.info, uid, _int(args[0])))))
        return await safe_answer(call)
    if action == "repdo":
        pack_id = _int(args[0])
        (await run_db(packs.report, uid, pack_id, args[1]))
        await _edit(call, view.card_screen((await run_db(packs.info, uid, pack_id)), uid))
        return await safe_answer(call, "Жалоба отправлена")
    if action == "import":
        await state.set_state(PackStates.import_pack)
        await _edit(
            call,
            view.prompt_screen(
                "Отправьте текстовый файл (.txt) или сообщение со словами. "
                "Первая строка вида «# Название» задаёт название набора. "
                "Слова разделяйте переводами строк, запятыми или точками с запятой."
            ),
        )
        return await safe_answer(call)
    if action in ("mod", "modv", "modr", "modh", "modd"):
        if not is_admin(settings, uid, call.from_user.username):
            return await safe_answer(call, "Недоступно", show_alert=True)
        if action == "mod":
            items, total = (await run_db(packs.moderation_list, _int(args[0]) if args else 0, view.PAGE_SIZE))
            await _edit(call, view.moderation_screen(items, total, _int(args[0]) if args else 0))
            return await safe_answer(call)
        pack_id = _int(args[0])
        if action == "modv":
            info, reasons = (await run_db(packs.moderation_info, pack_id))
            await _edit(call, view.moderation_card(info, reasons))
            return await safe_answer(call)
        (await run_db(packs.moderate, pack_id, {"modr": "restore", "modh": "hide", "modd": "delete"}[action]))
        items, total = (await run_db(packs.moderation_list, 0, view.PAGE_SIZE))
        await _edit(call, view.moderation_screen(items, total, 0))
        return await safe_answer(call, "Готово")
    await safe_answer(call)
async def _finish(message: Message) -> None:
    try:
        await message.delete()
    except TelegramBadRequest:
        pass
async def _say(message: Message, tracker: ScreenTracker, screen: Screen) -> None:
    await present(message, tracker, screen.text, to_plain_markup(screen))
```

```
TEXT_ONLY = (F.text, ~F.text.startswith("/"), ~F.text.in_(ALL_BUTTONS))
@router.message(StateFilter(PackStates.title), *TEXT_ONLY)
async def on_title(
    message: Message, state: FSMContext, packs: PackService, tracker: ScreenTracker
) -> None:
    await _finish(message)
    try:
        pack_id = (await run_db(packs.create_pack, message.from_user.id, display_name(message.from_user), message.text))
    except PackError as err:
        await _say(message, tracker, view.prompt_screen(f"Ошибка: {err}. Введите название ещё раз."))
        return
    await state.set_state(PackStates.words)
    await state.update_data(pack_id=pack_id)
    await _say(
        message,
        tracker,
        view.done_screen(
            f"Набор «{message.text.strip()}» создан и пока личный.\n\n"
            "Отправьте слова сообщением: по одному в строке или через запятую. "
            "Можно отправлять несколько сообщений. Когда закончите, нажмите «Готово».",
            pack_id,
        ),
    )
@router.message(StateFilter(PackStates.words), *TEXT_ONLY)
async def on_words(
    message: Message, state: FSMContext, packs: PackService, tracker: ScreenTracker
) -> None:
    data = await state.get_data()
    pack_id = data.get("pack_id")
    await _finish(message)
    try:
        added, skipped = (await run_db(packs.add_words, message.from_user.id, pack_id, split_words(message.text)))
        info = (await run_db(packs.info, message.from_user.id, pack_id))
    except PackError as err:
        await state.clear()
        await _say(message, tracker, Screen(f"Ошибка: {err}", [[("К разделам", "pk:menu")]]))
        return
    await _say(
        message,
        tracker,
        view.done_screen(
            f"{info.title}: добавлено слов {added}, пропущено (повторы или некорректные) {skipped}.\n"
            f"Всего в наборе: {info.words_count}. Можно отправить ещё.",
            pack_id,
        ),
    )
@router.message(StateFilter(PackStates.rename), *TEXT_ONLY)
async def on_rename(
    message: Message, state: FSMContext, packs: PackService, tracker: ScreenTracker
) -> None:
    data = await state.get_data()
    pack_id = data.get("pack_id")
    await _finish(message)
    try:
        (await run_db(packs.rename, message.from_user.id, pack_id, message.text))
    except PackError as err:
        await _say(message, tracker, view.prompt_screen(f"Ошибка: {err}. Введите название ещё раз.", f"pk:view:{pack_id}"))
        return
    await state.set_state(None)
    info = (await run_db(packs.info, message.from_user.id, pack_id))
    await _say(message, tracker, view.card_screen(info, message.from_user.id))
@router.message(StateFilter(PackStates.find), *TEXT_ONLY)
async def on_find(
    message: Message, state: FSMContext, packs: PackService, tracker: ScreenTracker
) -> None:
    data = await state.get_data()
    scope = data.get("find_scope", "public")
    await state.set_state(None)
    await _set_filters(state, scope, query=message.text.strip()[:60])
    await _finish(message)
    await _say(message, tracker, await _list_screen(packs, message.from_user.id, state, scope, 0))
@router.message(StateFilter(PackStates.edit_word), *TEXT_ONLY)
async def on_edit_word(
    message: Message, state: FSMContext, packs: PackService, tracker: ScreenTracker
) -> None:
    data = await state.get_data()
    await _finish(message)
    try:
        pack_id = (await run_db(packs.edit_word, message.from_user.id, data.get("word_id", 0), message.text))
    except PackError as err:
        await _say(message, tracker, view.prompt_screen(f"Ошибка: {err}. Введите слово ещё раз.", f"pk:w:{data.get('word_id', 0)}"))
        return
    await state.set_state(None)
    info = (await run_db(packs.info, message.from_user.id, pack_id))
    words, total = (await run_db(packs.list_words, message.from_user.id, pack_id, 0, view.WORDS_PAGE))
    await _say(message, tracker, view.words_screen(info, words, total, 0, True))
async def _import(
    message: Message, state: FSMContext, packs: PackService, tracker: ScreenTracker, text: str
) -> None:
    try:
        pack_id = (await run_db(packs.import_text, message.from_user.id, display_name(message.from_user), text))
    except PackError as err:
        await _say(message, tracker, view.prompt_screen(f"Ошибка: {err}"))
        return
    await state.set_state(None)
    info = (await run_db(packs.info, message.from_user.id, pack_id))
    card = view.card_screen(info, message.from_user.id)
    await _say(
        message,
        tracker,
        Screen(f"Набор импортирован. Добавлено слов: {info.words_count}.\n\n{card.text}", card.rows),
    )
@router.message(StateFilter(PackStates.import_pack), *TEXT_ONLY)
async def on_import_text(
    message: Message, state: FSMContext, packs: PackService, tracker: ScreenTracker
) -> None:
    await _finish(message)
    await _import(message, state, packs, tracker, message.text)
@router.message(StateFilter(PackStates.import_pack), F.document)
async def on_import_file(
    message: Message, state: FSMContext, packs: PackService, bot: Bot, tracker: ScreenTracker
) -> None:
    document = message.document
    if document.file_size and document.file_size > MAX_IMPORT_BYTES:
        await _say(message, tracker, view.prompt_screen("Файл слишком большой (максимум 200 КБ)."))
        return
    buffer = io.BytesIO()
    await bot.download(document, destination=buffer)
    await _finish(message)
    try:
        text = buffer.getvalue().decode("utf-8-sig")
    except UnicodeDecodeError:
        await _say(message, tracker, view.prompt_screen("Не удалось прочитать файл. Сохраните его в кодировке UTF-8."))
        return
    await _import(message, state, packs, tracker, text)
```

settings.py

```
from app.database.worker import run_db
from aiogram import F, Router
from aiogram.exceptions import TelegramBadRequest
from aiogram.types import CallbackQuery
from app.bot import settings_view
from app.bot.keyboards.render import to_plain_markup
from app.bot.ui import delete_quietly, refresh_main_keyboard, safe_answer
from app.bot.tracker import ScreenTracker
from app.core.config import Settings
from app.game.session import Screen
from app.services.settings_service import SettingsService
router = Router()
async def _edit(call: CallbackQuery, screen: Screen, tracker: ScreenTracker) -> None:
    uid = call.from_user.id
    anchor = tracker.get_anchor(uid)
    if anchor is not None and anchor > call.message.message_id:
        # Send below the keyboard carrier before deleting the old card, keeping
```

```
        # a usable screen if Telegram rejects sending the replacement.
        sent = await call.message.answer(
            screen.text, reply_markup=to_plain_markup(screen), disable_notification=True,
        )
        tracker.set(uid, sent.message_id)
        await delete_quietly(call.bot, call.message.chat.id, call.message.message_id)
        return
    try:
        await call.message.edit_text(screen.text, reply_markup=to_plain_markup(screen))
    except TelegramBadRequest as err:
        if "not modified" not in str(err):
            raise
@router.callback_query(F.data.startswith("set:"))
async def on_settings(
    call: CallbackQuery,
    user_settings: SettingsService,
    tracker: ScreenTracker,
    settings: Settings,
) -> None:
    parts = call.data.split(":")
    uid = call.from_user.id
    action = parts[1]
    if action == "tzmenu":
        await _edit(call, settings_view.timezone_screen((await run_db(user_settings.prefs, uid))), tracker)
        await safe_answer(call)
        return
    if action == "menu":
        await _edit(call, settings_view.settings_screen((await run_db(user_settings.prefs, uid))), tracker)
        await safe_answer(call)
        return
    if action == "noop":
        await safe_answer(call)
        return
    if action == "tzset" and len(parts) > 2:
        try:
            (await run_db(user_settings.set_tz_hours, uid, int(parts[2])))
        except ValueError:
            await safe_answer(call, "Недопустимый часовой пояс", show_alert=True)
            return
        await _edit(call, settings_view.timezone_screen((await run_db(user_settings.prefs, uid))), tracker)
        await safe_answer(call, "Часовой пояс сохранён")
        return
    if action in ("emoji", "colors") and len(parts) > 2:
        if parts[2] not in ("0", "1"):
            await safe_answer(call)
            return
        enabled = parts[2] == "1"
        previous = (await run_db(user_settings.prefs, uid))
        if action == "emoji":
            (await run_db(user_settings.set_emoji, uid, enabled))
        else:
            (await run_db(user_settings.set_colors, uid, enabled))
        changed = getattr(previous, action) != enabled
        if changed or tracker.get_anchor(uid) is None:
            await refresh_main_keyboard(call, tracker, settings)
        await _edit(call, settings_view.settings_screen((await run_db(user_settings.prefs, uid))), tracker)
        await safe_answer(call, "Сохранено")
        return
    await _edit(call, settings_view.settings_screen((await run_db(user_settings.prefs, uid))), tracker)
    await safe_answer(call)
stats.py
from app.database.worker import run_db
from aiogram import F, Router
from aiogram.exceptions import TelegramBadRequest
from aiogram.types import CallbackQuery
from app.bot import stats_view
from app.bot.ui import safe_answer
from app.bot.keyboards.render import to_plain_markup
from app.game.session import Screen
from app.services.stats_service import SORTS, StatsService
router = Router()
async def _edit(call: CallbackQuery, screen: Screen) -> None:
    try:
        await call.message.edit_text(screen.text, reply_markup=to_plain_markup(screen))
    except TelegramBadRequest as err:
        if "not modified" not in str(err):
            raise
    await safe_answer(call)
@router.callback_query(F.data.startswith("stat:rating:"))
async def on_rating(call: CallbackQuery, stats: StatsService) -> None:
    sort = call.data.split(":")[2]
    if sort not in SORTS:
        sort = "points"
    await _edit(call, stats_view.rating_screen((await run_db(stats.leaderboard, sort)), sort, call.from_user.id))
@router.callback_query(F.data == "stat:profile")
async def on_profile(call: CallbackQuery, stats: StatsService) -> None:
    name = call.from_user.first_name or call.from_user.username or "Игрок"
    await _edit(call, stats_view.profile_screen((await run_db(stats.profile, call.from_user.id)), name))
history_view.py
from datetime import timedelta, timezone
from app.game.session import Screen
from app.services.history_service import LobbyRound, LobbyRow, UserRound
PAGE_SIZE = 5
STATUS_TEXT = {
    "active": "идёт игра",
    "finished": "завершена",
    "abandoned": "прервана",
}
RESULT_TEXT = {"win": "победа", "loss": "поражение", "draw": "ничья"}
ROLE_TEXT = {"spy": "шпион", "civilian": "обычный игрок"}
def _shift(value, tz_minutes: int):
    if value.tzinfo is not None:
        value = value.astimezone(timezone.utc).replace(tzinfo=None)
    return value + timedelta(minutes=tz_minutes)
def tz_label(tz_minutes: int) -> str:
    hours = tz_minutes // 60
    if hours == 0:
        return "UTC"
    return f"UTC{'+' if hours > 0 else '-'}{abs(hours)}"
def fmt_dt(value, tz_minutes: int = 0) -> str:
    if not value:
        return "-"
    return _shift(value, tz_minutes).strftime("%d.%m.%Y %H:%M") + f" ({tz_label(tz_minutes)})"
def fmt_short(value, tz_minutes: int = 0) -> str:
    return _shift(value, tz_minutes).strftime("%d.%m %H:%M") if value else "-"
def outcome_text(outcome: str, spy_name: str) -> str:
    if outcome == "civilians_win":
        return "победили обычные игроки"
    if outcome == "spy_win_vote":
        return f"победил шпион {spy_name}: игроки не нашли шпиона"
    if outcome == "spy_win_guess":
        return f"победил шпион {spy_name}: он угадал слово"
    if outcome == "spy_left":
        return f"победили обычные игроки: шпион {spy_name} покинул игру"
    if outcome == "spy_wrong_guess":
        return f"победили обычные игроки: шпион {spy_name} ошибся при угадывании слова"
    return f"ничья: шпион {spy_name} угадал слово во время голосования"
def winners_text(row: LobbyRow) -> str:
    if row.winners:
        label = ", ".join(row.winners)
        return label if row.status != "active" else f"лидирует {label}"
    return "без победителя" if row.status != "active" else "пока нет лидера"
def _pager(prefix: str, page: int, total: int) -> list[tuple[str, str]]:
    pages = max(1, -(-total // PAGE_SIZE))
    nav = []
    if page > 0:
        nav.append(("Назад", f"{prefix}:{page - 1}"))
    if page < pages - 1:
        nav.append(("Вперёд", f"{prefix}:{page + 1}"))
    return nav
def menu_screen() -> Screen:
    return Screen(
        "🕘 История игр\n\nВыберите раздел, чтобы посмотреть свои результаты и завершённые комнаты.",
        [
            [("Мои игры", "hist:games:0")],
            [("История лобби", "hist:lobbies:0")],
            [("Закрыть", "menu:home")],
        ],
    )
def games_screen(rows: list[UserRound], total: int, page: int, tz_minutes: int = 0) -> Screen:
    if not rows:
        return Screen(
            "Мои игры\n\nВы ещё не сыграли ни одного онлайн-раунда.",
```

```
        [[("Назад", "hist:menu")]],
    )
    pages = max(1, -(-total // PAGE_SIZE))
    blocks = []
    for row in rows:
        sign = "+" if row.score_delta >= 0 else ""
        blocks.append(
            f"🕒 {fmt_dt(row.finished_at, tz_minutes)}\n"
            f"🎮 {row.lobby_title}\n"
            f"🎯 {row.word} · тема: {row.theme}\n"
            f"Набор: {row.pack_title or row.theme}\n"
            f"🕵 {row.spy_name} · {outcome_text(row.outcome, row.spy_name)}\n"
            f"Ваша роль: {ROLE_TEXT[row.role]} · {RESULT_TEXT[row.result]}\n"
            f"⭐ Очки: {sign}{row.score_delta}"
        )
    nav = _pager("hist:games", page, total)
    buttons = [[(f"Раунд: {r.pack_title or r.theme}"[:55], f"hist:rd:{r.round_id}")] for r in rows if r.round_id]
    buttons += ([nav] if nav else []) + [[("Назад", "hist:menu")]]
    return Screen(
        f"Мои игры (страница {page + 1} из {pages})\n\n" + "\n\n".join(blocks), buttons
    )
def lobbies_screen(rows: list[LobbyRow], total: int, page: int, tz_minutes: int = 0) -> Screen:
    if not rows:
        return Screen(
            "История лобби\n\nВы ещё не играли в онлайн-комнатах.",
            [[("Назад", "hist:menu")]],
        )
    pages = max(1, -(-total // PAGE_SIZE))
    buttons = []
    for row in rows:
        label = f"{fmt_short(row.started_at, tz_minutes)}, {winners_text(row)}"
        buttons.append([(label[:60], f"hist:lobby:{row.id}")])
    nav = _pager("hist:lobbies", page, total)
    if nav:
        buttons.append(nav)
    buttons.append([("Назад", "hist:menu")])
    return Screen(
        f"История лобби (страница {page + 1} из {pages})\n\nНажмите на лобби, чтобы открыть подробности.",
        buttons,
    )
def lobby_screen(row: LobbyRow, tz_minutes: int = 0) -> Screen:
    table = "\n".join(
        f"{i}. {name}: {score}" for i, (name, score) in enumerate(row.players, 1)
    )
    text = (
        f"{row.title}\n\n"
        f"Начало: {fmt_dt(row.started_at, tz_minutes)}\n"
        f"Статус: {STATUS_TEXT.get(row.status, row.status)}\n"
        f"Сыграно раундов: {row.rounds_count}\n"
        f"Победитель лобби: {winners_text(row)}\n\n"
        f"Игроки и очки:\n{table}"
    )
    return Screen(
        text,
        [
            *[[(name[:50], f"hist:sp:{row.id}:{uid}")] for uid, name in row.player_ids],
            [("Игры этого лобби", f"hist:lr:{row.id}:0")],
            [("К списку лобби", "hist:lobbies:0")],
        ],
    )
def lobby_rounds_screen(
    session_id: int, rows: list[LobbyRound], total: int, page: int, tz_minutes: int = 0
) -> Screen:
    back = [("К лобби", f"hist:lobby:{session_id}")]
    if not rows:
        return Screen("В этом лобби пока нет завершённых раундов.", [back])
    pages = max(1, -(-total // PAGE_SIZE))
    blocks = []
    for row in rows:
        points = ", ".join(
            f"{name} {'+' if delta >= 0 else ''}{delta}" for name, _, delta in row.scores
        )
        blocks.append(
            f"Раунд {row.round_no}, {fmt_dt(row.finished_at, tz_minutes)}\n"
            f"Тема: {row.theme}. Слово: {row.word}\n"
            f"Набор: {row.pack_title or row.theme}\n"
            f"Шпион: {row.spy_name}, {outcome_text(row.outcome, row.spy_name)}\n"
            f"Очки: {points}"
        )
    nav = _pager(f"hist:lr:{session_id}", page, total)
    buttons = [[(f"Раунд {r.round_no}: игроки и набор"[:55], f"hist:rd:{r.id}")] for r in rows if r.id]
    buttons += ([nav] if nav else []) + [back]
    return Screen(
        f"Игры лобби (страница {page + 1} из {pages})\n\n" + "\n\n".join(blocks), buttons
    )
```

__init__.py

menu.py

```
from aiogram.types import InlineKeyboardButton, InlineKeyboardMarkup
MENU_TEXT = "Меню"
ONLINE_TEXT = (
    "🎮 Онлайн-игра\n\n"
    "Создайте комнату и отправьте друзьям код или ссылку-приглашение, "
    "либо войдите в чужую комнату по коду."
)
def online_text(ban_seconds: float = 0, reason: str = "leave") -> str:
    if ban_seconds > 0:
        from app.services.penalty import ban_message
        return ONLINE_TEXT + "\n\n" + ban_message(ban_seconds, reason)
    return ONLINE_TEXT
RULES_TEXT = (
    "📖 Правила игры\n\n"
    "Все игроки, кроме одного, знают секретное слово. Один игрок — шпион, он слова не знает.\n\n"
    "Игра идёт кругами. В свой ход игрок говорит и сам решает, нужно ли голосование: "
    "он завершает ход обычной кнопкой или кнопкой с предложением голосования. "
    "После того как высказались все, подсчитываются предложения. Если за голосование "
    "больше половины игроков, оно начинается, иначе начинается новый круг.\n\n"
    "Голосование тайное. При ничьей проводится дополнительное голосование. "
    "Когда голосование закончилось, игра сразу заканчивается: если шпиона нашли, "
    "побеждают игроки, если не нашли (ошиблись или снова ничья), побеждает шпион.\n\n"
    "Угадывание слова. У шпиона одна попытка.\n"
    "- До голосования: верный ответ приносит победу шпиону, ошибка сразу приводит к победе игроков.\n"
    "- Во время голосования: верный ответ даёт ничью, ошибка просто сжигает попытку.\n"
    "В онлайн-игре результат применяется сразу. В игре на одном телефоне угадывать "
    "может любой игрок, ответ всегда принимается без показа результата, а итог попытки "
    "шпиона раскрывается в конце раунда: до голосования он решается в момент, когда "
    "голосование должно начаться (голосование при этом не проводится), во время "
    "голосования после его завершения.\n\n"
    "Темы. В каждом раунде играет одна тема из отмеченных: по очереди или в случайном порядке "
    "(настраивается).\n\n"
    "Выход из игры. Покинуть комнату можно в любой момент: нажмите «Выйти» и подтвердите "
    "действие в появившемся окне. Выход во время раунда наказывается блокировкой онлайн-игр: "
    "5 минут, затем 30 минут, 2 часа, 12 часов и 24 часа за каждый следующий выход "
    "(счётчик обнуляется через неделю без выходов). Если вышел шпион, побеждают игроки. "
    "Если вышел игрок, раунд идёт дальше, пока остаётся не меньше трёх игроков.\n\n"
    "Очки:\n"
    "- победа игроков: каждый обычный игрок получает 1 очко;\n"
    "- ничья: 0 очков у всех;\n"
    "- победа шпиона: шпион получает 3 очка."
)
def _btn(text: str, data: str) -> InlineKeyboardButton:
    return InlineKeyboardButton(text=text, callback_data=data)
```

```
def admin_lobby_menu() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [_btn("Тестовое лобби на 1 игрока", "online:test:1")],
            [_btn("Тестовое лобби на 2 игроков", "online:test:2")],
            [_btn("Жалобы на наборы", "pk:mod:0")],
            [_btn("Закрыть", "menu:home")],
        ]
    )
def local_choice_menu() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [_btn("Продолжить игру", "menu:resume")],
            [_btn("Начать новую", "menu:local")],
            [_btn("Закрыть", "menu:home")],
        ]
    )
def online_menu(in_room: bool = False, admin: bool = False) -> InlineKeyboardMarkup:
    rows = []
    if in_room:
        rows.append([_btn("Вернуться в комнату", "online:resume")])
    else:
        rows.append([_btn("Создать комнату", "online:create")])
        rows.append([_btn("Войти по коду", "online:join")])
        if admin:
            rows.append([_btn("Тестовые лобби", "online:testmenu")])
    rows.append([_btn("Закрыть", "menu:home")])
    return InlineKeyboardMarkup(inline_keyboard=rows)
def back_menu() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(inline_keyboard=[[_btn("Закрыть", "menu:home")]])
```

render.py

```
from aiogram.types import InlineKeyboardButton, InlineKeyboardMarkup
from app.game.session import Screen
def to_markup(screen: Screen, seq: int) -> InlineKeyboardMarkup:
    rows = [
        [
            InlineKeyboardButton(text=label, callback_data=f"lg:{seq}:{action}")
            for label, action in row
        ]
        for row in screen.rows
    ]
    return InlineKeyboardMarkup(inline_keyboard=rows)
def to_plain_markup(screen: Screen) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [InlineKeyboardButton(text=label, callback_data=action) for label, action in row]
            for row in screen.rows
            if row
        ]
    )
```

reply.py

```
from aiogram.types import KeyboardButton, ReplyKeyboardMarkup
BTN_ONLINE = "Играть онлайн"
BTN_LOCAL = "Один телефон"
BTN_PACKS = "Наборы слов"
BTN_HISTORY = "История игр"
BTN_PROFILE = "Профиль"
BTN_RATING = "Рейтинг"
BTN_ROOM = "Моя комната"
BTN_SETTINGS = "Настройки"
BTN_ADMIN = "Админ"
ALL_BUTTONS = {
    BTN_ONLINE,
    BTN_LOCAL,
    BTN_PACKS,
    BTN_HISTORY,
    BTN_PROFILE,
    BTN_RATING,
    BTN_ROOM,
    BTN_SETTINGS,
    BTN_ADMIN,
}
def main_reply_keyboard(admin: bool = False) -> ReplyKeyboardMarkup:
    last_row = [BTN_ROOM, BTN_SETTINGS] + ([BTN_ADMIN] if admin else [])
    rows = [
        [BTN_ONLINE, BTN_LOCAL],
        [BTN_PACKS, BTN_HISTORY],
        [BTN_PROFILE, BTN_RATING],
        last_row,
    ]
    return ReplyKeyboardMarkup(
        keyboard=[[KeyboardButton(text=label) for label in row] for row in rows],
        resize_keyboard=True,
        is_persistent=True,
        input_field_placeholder="Сообщение",
    )
```

__init__.py

navigation.py

```
from aiogram import BaseMiddleware
from aiogram.types import CallbackQuery, Message
from app.bot.emoji import strip_decor
from app.bot.keyboards.menu import MENU_TEXT
from app.bot.keyboards.reply import ALL_BUTTONS, main_reply_keyboard
from app.core.security import is_admin
class KeepMenuMiddleware(BaseMiddleware):
    """Keep a separate, permanent reply-keyboard carrier out of screen cleanup.
    Telegram clients may remove the keyboard when its message is deleted. The
    tracker is in memory, so restore the carrier on first navigation after restart.
    """
    async def __call__(self, handler, event, data):
        user = data.get("event_from_user")
        tracker = data.get("tracker")
        message = event.message if isinstance(event, CallbackQuery) else event
        if user is None or tracker is None or not isinstance(message, Message) or message.chat.type != "private":
            return await handler(event, data)
        if isinstance(event, CallbackQuery):
            payload = event.data or ""
            navigation = payload.startswith(("menu:", "online:", "set:", "pk:", "hist:", "stat:", "rm:", "rm_leave:", "lg:"))
        else:
            raw = message.text or ""
            text = strip_decor(raw)
            command = raw.split(maxsplit=1)[0].split("@")[0] if raw else ""
            # These handlers send the carrier themselves.
            deep_join = command == "/start" and len(raw.split(maxsplit=1)) > 1 and raw.split(maxsplit=1)[1].startswith("join_")
            if command in ("/start", "/menu", "/cancel") and not deep_join:
                return await handler(event, data)
            navigation = deep_join or text in ALL_BUTTONS or command in ("/settings", "/rules", "/admin")
        if navigation and tracker.get_anchor(user.id) is None:
            settings = data["settings"]
            sent = await message.bot.send_message(
                message.chat.id, MENU_TEXT,
                reply_markup=main_reply_keyboard(is_admin(settings, user.id, user.username)),
            )
            tracker.set_anchor(user.id, sent.message_id)
        return await handler(event, data)
```

normalize.py

```
from aiogram import BaseMiddleware
from aiogram.types import Message
from app.bot.emoji import strip_decor
from app.bot.keyboards.reply import ALL_BUTTONS
class NormalizeButtonsMiddleware(BaseMiddleware):
    async def __call__(self, handler, event, data):
        if isinstance(event, Message) and event.text:
            stripped = strip_decor(event.text)
            if stripped != event.text and stripped in ALL_BUTTONS:
                event = event.model_copy(update={"text": stripped})
        return await handler(event, data)
```

private.py

```
from aiogram import BaseMiddleware
from aiogram.types import CallbackQuery, Message
from app.bot.ui import safe_answer
class PrivateCallbacksMiddleware(BaseMiddleware):
    async def __call__(self, handler, event, data):
        if isinstance(event, CallbackQuery):
            message = event.message
            if not isinstance(message, Message) or message.chat.type != "private" or message.chat.id != event.from_user.id:
                await safe_answer(event, "Откройте бота в личном чате.")
                return None
        return await handler(event, data)
```

throttle.py

```
import time
from collections import OrderedDict
from aiogram import BaseMiddleware
from aiogram.types import CallbackQuery
from app.bot.ui import safe_answer
```

```
class ThrottleMiddleware(BaseMiddleware):
    def __init__(self, interval: float = 0.25, max_tracked: int = 20000):
        self.interval = interval
        self.max_tracked = max(1, max_tracked)
        self._last = OrderedDict()
        self._notified = OrderedDict()
    def _put(self, entries, key, value):
        entries.pop(key, None)
        if len(entries) >= self.max_tracked:
            entries.popitem(last=False)
        entries[key] = value
    async def __call__(self, handler, event, data):
        user = data.get("event_from_user")
        if user is None or self.interval <= 0:
            return await handler(event, data)
        now = time.monotonic()
        callback = isinstance(event, CallbackQuery)
        key = (user.id, "callback" if callback else "message")
        last = self._last.get(key)
        if last is not None and now - last < self.interval:
            # Do not turn a callback flood into an answerCallbackQuery flood.
            notified = self._notified.get(user.id)
            if callback and (notified is None or now - notified >= 1.0):
                self._put(self._notified, user.id, now)
                await safe_answer(event, "Слишком часто. Повторите через секунду.")
            return None
        self._put(self._last, key, now)
        return await handler(event, data)
track.py
from app.database.worker import run_db
from aiogram import BaseMiddleware
from aiogram.types import CallbackQuery
from app.bot.tracker import ScreenTracker
TRACKED_PREFIXES = ("pk:", "hist:", "stat:", "set:")
TRACKED_EXACT = {"menu:online", "online:testmenu", "online:join", "menu:rules"}
PANEL_EXACT = {"online:create", "online:resume", "menu:local", "menu:resume"}
PANEL_PREFIXES = ("online:test:",)
class TrackScreensMiddleware(BaseMiddleware):
    def __init__(self, tracker: ScreenTracker):
        self.tracker = tracker
    async def __call__(self, handler, event, data):
        user = data.get("event_from_user")
        if not isinstance(event, CallbackQuery) or user is None or event.message is None:
            return await handler(event, data)
        payload = event.data or ""
        message_id = event.message.message_id
        current = self.tracker.get(user.id)
        if payload.startswith("set:") and current is not None and current != message_id:
            from app.bot.ui import safe_answer
            await safe_answer(event, "Настройки обновлены. Используйте новые кнопки.")
            return None
        navigation = payload.startswith(TRACKED_PREFIXES) or payload in TRACKED_EXACT or payload in PANEL_EXACT or payload.startswith(PANEL_PREFIXES) or payload == "menu:home"
        # Actual game callbacks cannot adopt a menu screen. Avoid querying the
        # databases again before and after every game action.
        if not navigation:
            return await handler(event, data)
        repo = data.get("repo")
        runtime = data.get("runtime")
        local = (await run_db(repo.load, user.id)) if repo else None
        protected = bool(local and local.get("message_id") == message_id)
        if not protected and runtime:
            protected = (await run_db(runtime.service.panel_id_of, user.id)) == message_id
        if protected:
            from app.bot.ui import safe_answer
            await safe_answer(event, "Это игровой экран. Откройте раздел через меню.", show_alert=True)
            return None
        previous_screen = self.tracker.get(user.id)
        result = await handler(event, data)
        # Only creation/resume callbacks can promote this message to a panel.
        if payload in PANEL_EXACT or payload.startswith(PANEL_PREFIXES):
            if payload in ("menu:local", "menu:resume"):
                local = (await run_db(repo.load, user.id)) if repo else None
                adopted = bool(local and local.get("message_id") == message_id)
            else:
                adopted = bool(runtime and (await run_db(runtime.service.panel_id_of, user.id)) == message_id)
            if adopted:
                self.tracker.discard(user.id, message_id)
        elif payload.startswith(TRACKED_PREFIXES) or payload in TRACKED_EXACT:
            if self.tracker.get(user.id) == previous_screen:
                self.tracker.set(user.id, message_id)
        return result
pack_view.py
from app.game.session import Screen
from app.services.pack_service import MIN_WORDS, REPORT_REASONS, PackInfo
PAGE_SIZE = 6
WORDS_PAGE = 10
SCOPE_TITLES = {
    "my": "Мои наборы",
    "builtin": "Готовые наборы",
    "fav": "Избранное",
    "public": "Публичная библиотека",
}
SORT_LABELS = {"new": "новые", "name": "по названию", "size": "по размеру"}
SORT_ORDER = ["new", "name", "size"]
def _pages(total: int, size: int) -> int:
    return max(1, -(-total // size))
def _nav(prefix: str, page: int, total: int, size: int) -> list[tuple[str, str]]:
    nav = []
    if page > 0:
        nav.append(("Назад", f"{prefix}:{page - 1}"))
    if page < _pages(total, size) - 1:
        nav.append(("Вперёд", f"{prefix}:{page + 1}"))
    return nav
def menu_screen() -> Screen:
    return Screen(
        "Наборы слов\n\nВыберите раздел.",
        [
            [("Мои наборы", "pk:list:my:0"), ("Готовые наборы", "pk:list:builtin:0")],
            [("Избранное", "pk:list:fav:0"), ("Публичная библиотека", "pk:list:public:0")],
            [("Создать набор", "pk:new"), ("Импорт набора", "pk:import")],
            [("Закрыть", "menu:home")],
        ],
    )
def prompt_screen(text: str, back: str = "pk:menu") -> Screen:
    return Screen(text, [[("Отмена", back)]])
def list_screen(
    scope: str,
    infos: list[PackInfo],
    total: int,
    page: int,
    query: str | None = None,
    sort: str = "new",
    category_title: str | None = None,
) -> Screen:
    filters = []
    if query:
        filters.append(f"поиск: {query}")
    if category_title:
        filters.append(f"категория: {category_title}")
    if sort != "new":
        filters.append(f"сортировка: {SORT_LABELS[sort]}")
    head = f"{SCOPE_TITLES[scope]} (найдено: {total})"
    if filters:
        head += "\nФильтры: " + "; ".join(filters)
    if not infos:
        empty = {
            "my": "У вас пока нет своих наборов. Создайте первый.",
            "fav": "В избранном пока пусто. Добавьте набор из карточки.",
        }.get(scope, "Ничего не найдено.")
        head += f"\n\n{empty}"
    rows: list[list[tuple[str, str]]] = []
    for info in infos:
        mark = ""
        if info.is_favorite:
            mark = "[избр.] "
        if scope == "my":
            mark += "[публ.] " if info.is_public else "[личн.] "
        rows.append([(f"{mark}{info.title} ({info.words_count})", f"pk:view:{info.id}")])
    nav = _nav(f"pk:list:{scope}", page, total, PAGE_SIZE)
    if nav:
        rows.append(nav)
    controls = [("Поиск", f"pk:find:{scope}"), (f"Сортировка: {SORT_LABELS[sort]}", f"pk:sort:{scope}")]
    rows.append(controls)
    if scope in ("builtin", "public"):
        rows.append([("Категория", f"pk:catlist:{scope}")])
```

```
    if query or category_title or sort != "new":
        rows.append([("Сбросить фильтры", f"pk:clear:{scope}")])
    if scope == "my":
        rows.append([("Создать набор", "pk:new")])
    rows.append([("К разделам", "pk:menu")])
    return Screen(head, rows)
def category_screen(scope: str, categories: list[tuple[int, str]]) -> Screen:
    rows = [[("Все категории", f"pk:cat:{scope}:0")]]
    rows += [[(title, f"pk:cat:{scope}:{cid}")] for cid, title in categories]
    rows.append([("Назад", f"pk:list:{scope}:0")])
    return Screen("Выберите категорию.", rows)
def back_target(info: PackInfo, viewer_id: int) -> str:
    if info.owner_id == viewer_id:
        return "pk:list:my:0"
    if info.is_builtin:
        return "pk:list:builtin:0"
    return "pk:list:public:0"
def card_screen(info: PackInfo, viewer_id: int) -> Screen:
    own = info.owner_id == viewer_id
    if info.is_builtin:
        author = "готовый набор"
    else:
        author = info.owner_name or "неизвестно"
    access = "личный"
    if info.is_hidden:
        access = "скрыт модерацией"
    elif info.is_public:
        access = "публичный"
    lines = [
        info.title,
        "",
        f"Автор: {author}",
        f"Категория: {info.category}",
        f"Слов: {info.words_count}",
    ]
    if not info.is_builtin:
        lines.append(f"Доступ: {access}")
    lines.append(f"В избранном: {'да' if info.is_favorite else 'нет'}")
    if info.description:
        lines += ["", info.description]
    if info.words_count == 0:
        lines += ["", "Набор пуст: в игре он недоступен."]
    elif info.words_count < MIN_WORDS:
        lines += ["", f"Слов мало: для комфортной игры нужно не менее {MIN_WORDS}."]
    fav = ("Убрать из избранного" if info.is_favorite else "В избранное", f"pk:fav:{info.id}")
    rows: list[list[tuple[str, str]]] = [[("Слова", f"pk:words:{info.id}:0")]]
    if own:
        rows[0].append(("Добавить слова", f"pk:add:{info.id}"))
        rows.append(
            [
                ("Переименовать", f"pk:ren:{info.id}"),
                ("Сделать личным" if info.is_public else "Сделать публичным", f"pk:vis:{info.id}"),
            ]
        )
    rows.append([("Копировать", f"pk:copy:{info.id}"), ("Экспорт", f"pk:exp:{info.id}")])
    rows.append([fav])
    if own:
        rows.append([("Удалить набор", f"pk:del:{info.id}")])
    elif not info.is_builtin and info.is_public:
        rows.append([("Пожаловаться", f"pk:rep:{info.id}")])
    rows.append([("Назад", back_target(info, viewer_id))])
    return Screen("\n".join(lines), rows)
def words_screen(
    info: PackInfo, words: list[tuple[int, str]], total: int, page: int, own: bool
) -> Screen:
    head = f"{info.title}: слова (всего {total}), страница {page + 1} из {_pages(total, WORDS_PAGE)}"
    rows: list[list[tuple[str, str]]] = []
    if not words:
        head += "\n\nВ наборе пока нет слов."
    elif own:
        head += "\n\nНажмите на слово, чтобы изменить или удалить его."
        for i in range(0, len(words), 2):
            rows.append([(w, f"pk:w:{wid}") for wid, w in words[i : i + 2]])
    else:
        head += "\n\n" + "\n".join(f"{page * WORDS_PAGE + i}. {w}" for i, (_, w) in enumerate(words, 1))
    nav = _nav(f"pk:words:{info.id}", page, total, WORDS_PAGE)
    if nav:
        rows.append(nav)
    if own:
        rows.append([("Добавить слова", f"pk:add:{info.id}")])
    rows.append([("К набору", f"pk:view:{info.id}")])
    return Screen(head, rows)
def word_screen(word_id: int, word: str, pack_id: int) -> Screen:
    return Screen(
        f"Слово: {word}",
        [
            [("Изменить", f"pk:wed:{word_id}"), ("Удалить", f"pk:wdel:{word_id}")],
            [("Назад", f"pk:words:{pack_id}:0")],
        ],
    )
def delete_screen(info: PackInfo) -> Screen:
    return Screen(
        f"Удалить набор «{info.title}» ({info.words_count} слов)?\n\nЭто действие нельзя отменить.",
        [[("Да, удалить", f"pk:delyes:{info.id}")], [("Отмена", f"pk:view:{info.id}")]],
    )
def report_screen(info: PackInfo) -> Screen:
    rows = [[(label, f"pk:repdo:{info.id}:{code}")] for code, label in REPORT_REASONS.items()]
    rows.append([("Отмена", f"pk:view:{info.id}")])
    return Screen(f"Пожаловаться на набор «{info.title}». Укажите причину.", rows)
def done_screen(text: str, pack_id: int) -> Screen:
    return Screen(text, [[("Готово", f"pk:view:{pack_id}")]])
def moderation_screen(items: list[PackInfo], total: int, page: int) -> Screen:
    if not items:
        return Screen("Жалоб нет.", [[("К разделам", "pk:menu")]])
    rows = []
    for info in items:
        mark = "[скрыт] " if info.is_hidden else ""
        rows.append([(f"{mark}{info.title}: жалоб {info.reports}", f"pk:modv:{info.id}")])
    nav = _nav("pk:mod", page, total, PAGE_SIZE)
    if nav:
        rows.append(nav)
    rows.append([("К разделам", "pk:menu")])
    return Screen(f"Жалобы на наборы (всего {total}).", rows)
def moderation_card(info: PackInfo, reasons: list[str]) -> Screen:
    text = (
        f"{info.title}\n\nАвтор: {info.owner_name or 'неизвестно'}\n"
        f"Слов: {info.words_count}\nСтатус: {'скрыт' if info.is_hidden else 'виден'}\n"
        f"Жалобы: {', '.join(reasons) if reasons else 'нет'}"
    )
    return Screen(
        text,
        [
            [("Оставить и снять жалобы", f"pk:modr:{info.id}")],
            [("Скрыть", f"pk:modh:{info.id}"), ("Удалить", f"pk:modd:{info.id}")],
            [("Назад", "pk:mod:0")],
        ],
    )
```

recovery.py

```
"""Retry edits of menus/local games; online panels reload through OnlineRuntime."""
import asyncio
import logging
import time
from weakref import WeakValueDictionary
from aiogram.client.session.middlewares.base import BaseRequestMiddleware
from aiogram.exceptions import TelegramNetworkError, TelegramRetryAfter, TelegramServerError, TelegramAPIError
from aiogram.methods import DeleteMessage, EditMessageText, EditMessageReplyMarkup
log = logging.getLogger(__name__)
class EditRecoveryMiddleware(BaseRequestMiddleware):
    def __init__(self):
        self.pending = {}
        self._locks = WeakValueDictionary()
    async def __call__(self, make_request, bot, method):
        if not isinstance(method, (DeleteMessage, EditMessageText, EditMessageReplyMarkup)):
            return await make_request(bot, method)
        key = (method.chat_id, method.message_id)
        lock = self._locks.setdefault(key, asyncio.Lock())
        async with lock:
```

```
        # A new edit or closing/deleting the screen supersedes its old retry.
        self.pending.pop(key, None)
        markup = getattr(method, 'reply_markup', None)
        online = markup and any((b.callback_data or '').startswith('rm:')
                                for row in markup.inline_keyboard for b in row)
        try:
            return await make_request(bot, method)
        except (TelegramRetryAfter, TelegramNetworkError, TelegramServerError) as err:
            if not isinstance(method, DeleteMessage) and not online:
                delay = err.retry_after if isinstance(err, TelegramRetryAfter) else 5
                if len(self.pending) >= 4096:
                    self.pending.pop(next(iter(self.pending)))
                self.pending[key] = (time.monotonic() + max(1, delay), method)
            raise
    async def retry_pending(self, bot):
        now = time.monotonic()
        due = [(key, entry) for key, entry in list(self.pending.items()) if entry[0] <= now][:4]
        async def retry(key, entry):
            if self.pending.get(key) is not entry:
                return
            # Re-enter preference decoration with the original undecorated edit.
            try:
                await bot(entry[1])
            except TelegramAPIError as err:
                log.warning('Deferred screen edit %s failed: %s', key, err)
        # Bound each cycle so an outage in many chats cannot starve game timers.
        await asyncio.gather(*(retry(key, entry) for key, entry in due))
runtime.py
from app.database.worker import run_db
import asyncio
import logging
import time
from collections import OrderedDict
from weakref import WeakValueDictionary
from aiogram import Bot
from aiogram.exceptions import (
    TelegramAPIError,
    TelegramBadRequest,
    TelegramForbiddenError,
    TelegramNetworkError,
    TelegramRetryAfter,
)
from aiogram.types import CopyTextButton, InlineKeyboardButton, InlineKeyboardMarkup, LinkPreviewOptions
from app.bot.ui import delete_quietly
from app.bot.confirmations import LeaveConfirmations
from app.game.room import Room
from app.game.session import Screen
from app.services.room_service import RoomService
log = logging.getLogger(__name__)
NO_PREVIEW = LinkPreviewOptions(is_disabled=True)
def to_room_markup(screen: Screen) -> InlineKeyboardMarkup:
    rows = []
    for row in screen.rows:
        buttons = []
        for label, action in row:
            if action.startswith("url:"):
                buttons.append(InlineKeyboardButton(text=label, url=action[4:]))
            elif action.startswith("copy:"):
                buttons.append(
                    InlineKeyboardButton(
                        text=label,
                        copy_text=CopyTextButton(text=action[5:]),
                    )
                )
            else:
                buttons.append(InlineKeyboardButton(text=label, callback_data=f"rm:{action}"))
        rows.append(buttons)
    return InlineKeyboardMarkup(inline_keyboard=rows)
class OnlineRuntime:
    def __init__(self, bot: Bot, service: RoomService, bot_username: str, maintenance=(), prefs_for=None):
        self.bot = bot
        self.service = service
        self.bot_username = bot_username
        self._locks = WeakValueDictionary()
        self._last_refresh: dict[int, float] = {}
        self._maintenance = list(maintenance)
        self._prefs_for = prefs_for
        self._render_slots = asyncio.Semaphore(4)
        self._panel_cache: dict[int, tuple[tuple, float]] = {}
        self._retry_after: dict[int, float] = {}
        self._pending_rooms: dict[int, int] = {}
        self.last_cycle = time.monotonic()
        self.edit_recovery = None
        self.cache_ttl = 60.0
        self.leave_confirmations = LeaveConfirmations()
        self._last_chat = OrderedDict()
    def allow_chat_message(self, user_id, interval=1.0):
        now = time.monotonic()
        previous = self._last_chat.get(user_id)
        if previous is not None and now - previous < interval:
            return False
        self._last_chat.pop(user_id, None)
        if len(self._last_chat) >= 4096:
            self._last_chat.popitem(last=False)
        self._last_chat[user_id] = now
        return True
    def _defer_render(self, uid, seconds, room_id):
        if uid not in self._retry_after and len(self._retry_after) >= 4096:
            oldest = next(iter(self._retry_after))
            self._retry_after.pop(oldest)
            self._pending_rooms.pop(oldest, None)
        self._retry_after[uid] = time.monotonic() + max(1.0, seconds)
        self._pending_rooms[uid] = room_id
    def _remember(self, uid: int, signature: tuple) -> None:
        if uid not in self._panel_cache and len(self._panel_cache) >= 4096:
            self._panel_cache.pop(next(iter(self._panel_cache)))
        self._panel_cache[uid] = (signature, time.monotonic())
        self._retry_after.pop(uid, None)
        self._pending_rooms.pop(uid, None)
    def lock(self, room_id: int) -> asyncio.Lock:
        return self._locks.setdefault(room_id, asyncio.Lock())
    def in_room(self, user_id: int) -> bool:
        return self.service.room_id_of(user_id) is not None
    async def adopt_panel(self, room_id: int, user_id: int, message_id: int) -> None:
        self._panel_cache.pop(user_id, None)
        self._retry_after.pop(user_id, None)
        self._pending_rooms.pop(user_id, None)
        room = (await run_db(self.service.get, room_id))
        old = None
        if room is not None:
            for player in room.players:
                if player["id"] == user_id:
                    old = player.get("panel")
        if old and old != message_id:
            await delete_quietly(self.bot, user_id, old)
        (await run_db(self.service.set_panel, room_id, user_id, message_id, player_activity=True))
    async def refresh(self, room_id: int) -> None:
        room = (await run_db(self.service.get, room_id))
        if room is None:
            return
        self._last_refresh[room_id] = time.monotonic()
        now = time.time()
        async def render(player):
            async with self._render_slots:
                await self._render(room_id, room, player, now)
        await asyncio.gather(*(render(player) for player in list(room.players)))
    async def _render(self, room_id: int, room: Room, player: dict, now: float) -> None:
        uid = player["id"]
        screen = room.view_for(uid, self.bot_username, now)
        markup = to_room_markup(screen)
        styles = None
        if self._prefs_for is not None:
            prefs = (await run_db(self._prefs_for, uid))
            styles = (prefs.emoji, prefs.colors)
        markup_json = markup.model_dump_json(exclude_none=True)
        signature = (room_id, player.get("panel"), screen.text, markup_json, styles)
        current = time.monotonic()
        if self._retry_after.get(uid, 0) > current:
            return
        cached = self._panel_cache.get(uid)
        if cached and cached[0] == signature and current - cached[1] < self.cache_ttl:
            return
        if player.get("panel"):
            try:
                await self.bot.edit_message_text(
                    screen.text,
                    chat_id=uid,
                    message_id=player["panel"],
                    reply_markup=markup,
                    parse_mode="HTML",
```

```
link_preview_options=NO_PREVIEW,
            )
            self._remember(uid, signature)
            return
        except TelegramBadRequest as err:
            reason = str(err).lower()
            if "not modified" in reason:
                self._remember(uid, signature)
                return
            if not any(marker in reason for marker in ("message to edit not found", "message can't be edited")):
                self._retry_after.pop(uid, None)
                self._pending_rooms.pop(uid, None)
                log.warning("Panel %s edit rejected: %s", uid, err)
                return
        except TelegramRetryAfter as err:
            self._defer_render(uid, err.retry_after, room_id)
            log.warning("Panel %s rate limited for %s seconds", uid, err.retry_after)
            return
        except TelegramForbiddenError:
            self._retry_after.pop(uid, None)
            self._pending_rooms.pop(uid, None)
            return
        except TelegramNetworkError as err:
            self._defer_render(uid, 5, room_id)
            log.warning("Сеть: не удалось обновить панель %s: %s", uid, err)
            return
        except TelegramAPIError:
            self._defer_render(uid, 10, room_id)
            log.exception("edit failed")
            return
    try:
        sent = await self.bot.send_message(
            uid,
            screen.text,
            reply_markup=markup,
            parse_mode="HTML",

link_preview_options=NO_PREVIEW,
        )
        (await run_db(self.service.set_panel, room_id, uid, sent.message_id))
        self._remember(uid, (room_id, sent.message_id, screen.text, markup_json, styles))
    except TelegramRetryAfter as err:
        self._defer_render(uid, err.retry_after, room_id)
        log.warning("Panel %s rate limited for %s seconds", uid, err.retry_after)
    except TelegramNetworkError as err:
        self._defer_render(uid, 5, room_id)
        log.warning("Сеть: не удалось отправить панель %s: %s", uid, err)
    except (TelegramForbiddenError, TelegramBadRequest):
        self._retry_after.pop(uid, None)
        self._pending_rooms.pop(uid, None)
    except TelegramAPIError:
        self._defer_render(uid, 10, room_id)
        log.exception("send failed")
async def retry_pending(self) -> None:
    """Retry even lobby/result screens, always using current membership/state."""
    now = time.monotonic()
    due = {rid for uid, rid in list(self._pending_rooms.items())
           if self._retry_after.get(uid, 0) <= now}
    for room_id in due:
        async with self.lock(room_id):
            room = await run_db(self.service.get, room_id)
            members = {p["id"] for p in room.players} if room else set()
            for uid, rid in list(self._pending_rooms.items()):
                if rid == room_id and uid not in members:

self._pending_rooms.pop(uid, None)
                    self._retry_after.pop(uid, None)
                    self._panel_cache.pop(uid, None)
            if room:
                await self.refresh(room_id)
async def tick(self) -> None:
    for room_id in (await run_db(self.service.due_room_ids)):
        async with self.lock(room_id):
            update = (await run_db(self.service.tick_room, room_id))
            if update and update.changed:
                await self.refresh(room_id)
async def countdown(self, now: float | None = None, every: float = 10.0) -> int:
    current = time.monotonic() if now is None else now
    active = (await run_db(self.service.active_room_ids))
    self._last_refresh = {k: v for k, v in self._last_refresh.items() if k in active}
    refreshed = 0
    for room_id in active:
        last = self._last_refresh.get(room_id)
        if last is None or current - last >= every:
            async with self.lock(room_id):
                await self.refresh(room_id)
                self._last_refresh[room_id] = current
            refreshed += 1
    return refreshed
async def run_ticker(self) -> None:
    purge_at = 0.0
    while True:
        await asyncio.sleep(3)
        try:
            await self.tick()
            await self.countdown()
            await self.retry_pending()
            if self.edit_recovery is not None:
                await self.edit_recovery.retry_pending(self.bot)
            if time.time() >= purge_at:
                (await run_db(self.service.purge_stale))
                for job in self._maintenance:
                    (await run_db(job))
                purge_at = time.time() + 600
            self.last_cycle = time.monotonic()
        except Exception:
            log.exception("ticker failed")
settings_view.py
from datetime import datetime, timezone
from app.game.session import Screen
from app.services.settings_service import (
    OFFSET_HINTS,
    OFFSETS,
    Prefs,
    local_time,
    zone_title,
)
def _on_off(value: bool) -> str:
    return "включены" if value else "выключены"
def _coerce_prefs(prefs: Prefs | int) -> Prefs:
    return prefs if isinstance(prefs, Prefs) else Prefs(tz_minutes=int(prefs))
def settings_screen(prefs: Prefs | int, now_utc: datetime | None = None) -> Screen:
    prefs = _coerce_prefs(prefs)
    now_utc = now_utc or datetime.now(timezone.utc)
    current = prefs.tz_minutes // 60
    return Screen(
        "⚙ Настройки\n\n"
        f"😀 Эмодзи на кнопках: {_on_off(prefs.emoji)}\n"
        f"Цветные кнопки: {_on_off(prefs.colors)}\n"
        f"🌐 Часовой пояс: {zone_title(current)}\n"
        f"🕒 Местное время: {local_time(now_utc, current)}",
        [
            [(f"Эмодзи: {'ВКЛ' if prefs.emoji else 'ВЫКЛ'}", f"set:emoji:{0 if prefs.emoji else 1}")],
            [(f"Цветные кнопки: {'ВКЛ' if prefs.colors else 'ВЫКЛ'}", f"set:colors:{0 if prefs.colors else 1}")],
            [("Часовой пояс", "set:tzmenu")],
            [("Закрыть", "menu:home")],
        ],
    )
def timezone_screen(prefs: Prefs | int, now_utc: datetime | None = None) -> Screen:
    prefs = _coerce_prefs(prefs)
    now_utc = now_utc or datetime.now(timezone.utc)
    current = prefs.tz_minutes // 60
    rows = []
    for hours in OFFSETS:
        city = OFFSET_HINTS.get(hours, "")
        sign = "+" if hours >= 0 else "-"
        label = f"{local_time(now_utc, hours)} · {city} · UTC{sign}{abs(hours)}"
        if hours == current:
            label = f"✓ {label}"
        rows.append([(label, f"set:tzset:{hours}")])
    rows.append([("← Назад к настройкам", "set:menu")])
    rows.append([("Закрыть", "menu:home")])
    return Screen(
        "🌐 Часовой пояс\n\n"
        "Выберите часовой пояс по местному времени. Изменение сохранится автоматически.\n\n"
        f"Сейчас выбран: {zone_title(current)} · {local_time(now_utc, current)}",
        rows,
    )
stats_view.py
from app.game.session import Screen
from app.services.stats_service import MIN_GAMES_FOR_RATE, Profile, RatingRow
SORT_LABELS = {
    "points": "По очкам",
    "wins": "По победам",
```

```
    "spy": "Победы за шпиона",
    "rate": "Процент побед",
}
def profile_screen(profile: Profile | None, name: str) -> Screen:
    if profile is None:
        return Screen(
            f"Профиль: {name}\n\nВы ещё не сыграли ни одного онлайн-раунда.",
            [[("Закрыть", "menu:home")]],
        )
    text = (
        f"👤 Профиль · {name}\n\n"
        f"🎮 Сыграно раундов: {profile.games}\n"
        f"🏆 Победы: {profile.wins} ({profile.win_rate}%)\n"
        f"😇 Победы обычным игроком: {profile.civilian_wins}\n"
        f"🕵 Победы шпионом: {profile.spy_wins}\n"
        f"💥 Поражения: {profile.losses} · Ничьи: {profile.draws}\n"
        f"⭐ Всего очков: {profile.points}\n"
        f"🏠 Сыграно лобби: {profile.lobbies}"
    )
    return Screen(text, [[("Закрыть", "menu:home")]])
def rating_screen(rows: list[RatingRow], sort: str, viewer_id: int | None = None) -> Screen:
    title = f"🏆 Рейтинг · {SORT_LABELS[sort].lower()}"
    if not rows:
        note = (
            f"\n\nВ рейтинг попадают игроки, сыгравшие не менее {MIN_GAMES_FOR_RATE} раундов."
            if sort == "rate"
            else ""
        )
        body = f"Пока нет данных.{note}"
    else:
        lines = []
        for place, row in enumerate(rows, 1):
            mark = " (вы)" if row.user_id == viewer_id else ""
            medal = {1: "🥇", 2: "🥈", 3: "🥉"}.get(place, f"{place}.")
            lines.append(
                f"{medal} {row.name}{mark}: {row.points} оч., побед {row.wins} из {row.games}"
                f" ({row.win_rate}%), за шпиона {row.spy_wins}"
            )
        body = "\n".join(lines)
        if sort == "rate":
            body += f"\n\nУчитываются игроки с {MIN_GAMES_FOR_RATE}+ раундами."
    buttons = [
        [(label if key != sort else f"[x] {label}", f"stat:rating:{key}")]
        for key, label in SORT_LABELS.items()
    ]
    buttons.append([("Закрыть", "menu:home")])
    return Screen(f"{title}\n\n{body}", buttons)
```

tracker.py

```
import json
import logging
from pathlib import Path

log = logging.getLogger(__name__)
class ScreenTracker:
    def __init__(self, anchor_file: Path | None = None) -> None:
        self._ids: dict[int, int] = {}
        self._anchors: dict[int, int] = {}
        self._anchor_file = anchor_file
        if anchor_file is not None and anchor_file.exists():
            try:
                stored = json.loads(anchor_file.read_text(encoding="utf-8"))
                self._anchors = {int(uid): int(mid) for uid, mid in stored.items() if int(uid) > 0 and int(mid) > 0}
            except (OSError, ValueError, TypeError, AttributeError):
                log.warning("Cannot restore menu keyboard messages", exc_info=True)
    def _save_anchors(self) -> None:
        if self._anchor_file is None:
            return
        try:
            temporary = self._anchor_file.with_suffix(".tmp")

            temporary.write_text(json.dumps(self._anchors), encoding="utf-8")

            temporary.replace(self._anchor_file)
        except OSError:
            log.warning("Cannot save menu keyboard messages", exc_info=True)
    def get(self, user_id: int) -> int | None:
        return self._ids.get(user_id)
    def set(self, user_id: int, message_id: int) -> None:
        self._ids[user_id] = message_id
    def pop(self, user_id: int) -> int | None:
        return self._ids.pop(user_id, None)
    def discard(self, user_id: int, message_id: int) -> None:
        if self._ids.get(user_id) == message_id:
            del self._ids[user_id]
    def get_anchor(self, user_id: int) -> int | None:
        return self._anchors.get(user_id)
    def set_anchor(self, user_id: int, message_id: int) -> None:
        self._anchors[user_id] = message_id
        self._save_anchors()
    def pop_anchor(self, user_id: int) -> int | None:
        old = self._anchors.pop(user_id, None)
        if old is not None:
            self._save_anchors()
        return old
```

ui.py

```
import logging
from aiogram import Bot
from aiogram.exceptions import TelegramAPIError, TelegramBadRequest
from aiogram.types import CallbackQuery, Message
from app.bot.tracker import ScreenTracker
from app.bot.keyboards.menu import MENU_TEXT
from app.bot.keyboards.reply import main_reply_keyboard
from app.core.config import Settings
from app.core.security import is_admin
log = logging.getLogger(__name__)
async def delete_quietly(bot: Bot, chat_id: int, message_id: int) -> None:
    try:
        await bot.delete_message(chat_id=chat_id, message_id=message_id)
    except TelegramAPIError:
        pass
async def clear_tracked(bot: Bot, chat_id: int, user_id: int, tracker: ScreenTracker) -> None:
    old = tracker.pop(user_id)
    if old:
        await delete_quietly(bot, chat_id, old)
async def present(
    message: Message,
    tracker: ScreenTracker,
    text: str,
    markup=None,
    parse_mode: str | None = None,
) -> Message:
    old = tracker.get(message.from_user.id)
    if old:
        try:
            await message.bot.edit_message_text(
                text, chat_id=message.chat.id, message_id=old,
                reply_markup=markup, parse_mode=parse_mode,
            )
            return message.model_copy(update={"message_id": old, "text": text})
        except TelegramBadRequest as err:
            if "not modified" in str(err):
                return message.model_copy(update={"message_id": old, "text": text})
            if not any(reason in str(err).lower() for reason in (
                "message to edit not found", "message can't be edited",
            )):
                raise
            await clear_tracked(message.bot, message.chat.id, message.from_user.id, tracker)
    sent = await message.answer(text, reply_markup=markup, parse_mode=parse_mode)
    tracker.set(message.from_user.id, sent.message_id)
    return sent
async def safe_answer(call: CallbackQuery, text: str | None = None, show_alert: bool = False) -> None:
    try:
        await call.answer(text, show_alert=show_alert)
    except TelegramAPIError as err:
        log.warning("Не удалось ответить на нажатие: %s", err)
async def refresh_main_keyboard(call: CallbackQuery, tracker: ScreenTracker, settings: Settings) -> None:
    """Reply keyboards require a new message; replace only their neutral carrier.
    Delete the old carrier first, so deleting it cannot hide the new keyboard.
    Keep the inline settings screen and game panels untouched.
    """
    uid = call.from_user.id
    old = tracker.pop_anchor(uid)
    if old:
        await delete_quietly(call.bot, call.message.chat.id, old)
    sent = await call.bot.send_message(
        call.message.chat.id, MENU_TEXT,
```

```
reply_markup=main_reply_keyboard(is_admin(settings, uid, call.from_user.username)),
        disable_notification=True,
    )
    tracker.set_anchor(uid, sent.message_id)
__init__.py
config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
@dataclass(frozen=True)
class Settings:
    bot_token: str
    database_url: str
    admin_username: str = "WAWGAME"
    admin_id: int | None = None
    proxy_url: str | None = None
def load_settings() -> Settings:
    load_dotenv()
    token = os.getenv("BOT_TOKEN", "").strip()
    if not token:
        raise RuntimeError("Переменная окружения BOT_TOKEN не задана")
    raw_admin_id = os.getenv("ADMIN_ID", "").strip()
    return Settings(
        bot_token=token,

admin_username=os.getenv("ADMIN_USERNAME", "WAWGAME").strip() or "WAWGAME",
        admin_id=int(raw_admin_id) if raw_admin_id.isdigit() else None,

proxy_url=os.getenv("PROXY_URL", "").strip() or None,

database_url=os.getenv("DATABASE_URL",
"mysql+pymysql://spy:spy@localhost:3306/spy_game?charset=utf8mb4"),
    )
console.py
import ctypes
import os
import sys
import threading
import time
from pathlib import Path
SHOW_FLAG = Path(__file__).resolve().parents[2] / ".window_show"
SW_HIDE = 0
SW_SHOW = 5
SW_RESTORE = 9
def parse_command(line: str) -> str | None:
    command = line.strip().lower()
    if command == "inv":
        return "hide"
    return None
def _hwnd() -> int:
    if os.name != "nt":
        return 0
    return ctypes.windll.kernel32.GetConsoleWindow()
def hide_console() -> bool:
    handle = _hwnd()
    if not handle:
        return False

ctypes.windll.user32.ShowWindow(handle, SW_HIDE)
    return True
def show_console() -> bool:
    handle = _hwnd()
    if not handle:
        return False

ctypes.windll.user32.ShowWindow(handle, SW_SHOW)

ctypes.windll.user32.ShowWindow(handle, SW_RESTORE)

ctypes.windll.user32.SetForegroundWindow(handle)
    return True
def _read_input() -> None:
    while True:
        try:
            line = sys.stdin.readline()
        except (OSError, ValueError):
            return
        if not line:
            return
        if parse_command(line) == "hide":
            print("Окно скрыто. Чтобы вернуть его, запустите show_bot.bat")
            time.sleep(0.3)
            hide_console()
def _watch_flag() -> None:
    while True:
        time.sleep(1)
        if SHOW_FLAG.exists():
            try:
                SHOW_FLAG.unlink()
            except OSError:
                pass
            show_console()
def start_console_control() -> bool:
    if os.name != "nt" or not _hwnd():
        return False
    try:

SHOW_FLAG.unlink(missing_ok=True)
    except OSError:
        pass
    threading.Thread(target=_read_input, daemon=True).start()
    threading.Thread(target=_watch_flag, daemon=True).start()
    print("Бот запущен. Введите inv и нажмите Enter, чтобы скрыть это окно.")
    print("Вернуть окно: show_bot.bat")
    return True
security.py
from app.core.config import Settings
def is_admin(settings: Settings, user_id: int, username: str | None) -> bool:
    # Telegram usernames can change ownership; privileges belong to a stable ID.
    return settings.admin_id is not None and settings.admin_id > 0 and user_id == settings.admin_id
__init__.py
info.py
from sqlalchemy import create_engine, func, inspect, select, text
from sqlalchemy.engine import make_url
from sqlalchemy.orm import sessionmaker
from app.database.models import Base
def ensure_mysql(url: str) -> None:
    backend = make_url(url).get_backend_name()
    if backend != "mysql":
        raise RuntimeError(
            f"Бот работает только с MySQL, а в DATABASE_URL указан «{backend}». "
            "Пример: mysql+pymysql://spy:spy@localhost:3306/spy_game?charset=utf8mb4"
        )
def mask_url(url: str) -> str:
    return make_url(url).render_as_string(hide_password=True)
def check_connection(url: str) -> None:
    engine = create_engine(url, pool_pre_ping=True)
    try:
        with engine.connect() as conn:
            conn.execute(text("SELECT 1"))
    finally:
        engine.dispose()
def describe(factory: sessionmaker, url: str) -> dict:
    parsed = make_url(url)
    with factory() as db:
        bind = db.connection().engine
        dialect = bind.dialect.name
        version_info = bind.dialect.server_version_info or ()
        version = ".".join(str(part) for part in version_info) or "?"
        existing = set(inspect(db.connection()).get_table_names())
        counts = {}
        for table in Base.metadata.sorted_tables:
            if table.name in existing:
                counts[table.name] = db.execute(select(func.count()).select_from(table)).scalar_one()
    return {
        "dialect": dialect,
        "version": version,
        "host": parsed.host or "-",
        "port": parsed.port or "-",
        "database": parsed.database or "-",
        "tables": counts,
    }
def format_info(info: dict) -> str:
    lines = [
        f"База данных: {info['dialect']} {info['version']}",
        f"Сервер: {info['host']}:{info['port']}, схема {info['database']}",
        "",
        "Строк в таблицах:",
    ]
    lines += [f"{name}: {count}" for name, count in info["tables"].items()]
    return "\n".join(lines)
models.py
from datetime import datetime, timezone
from sqlalchemy import BigInteger, Boolean, DateTime, Double, ForeignKey, Integer, String, Text, UniqueConstraint
from sqlalchemy.dialects.mysql import LONGTEXT
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
class Base(DeclarativeBase):
    pass
def _now() -> datetime:
    return datetime.now(timezone.utc)
class LocalGameRecord(Base):
    __tablename__ = "local_games"
    owner_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=False)
    state: Mapped[str] = mapped_column(Text().with_variant(LONGTEXT(), "mysql"), nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=_now, onupdate=_now)
class RoomRecord(Base):
```

```
    __tablename__ = "rooms"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    code: Mapped[str] = mapped_column(String(8), unique=True, index=True)
    invite_token: Mapped[str] = mapped_column(String(32), unique=True, index=True)
    host_id: Mapped[int] = mapped_column(BigInteger)
    state: Mapped[str] = mapped_column(Text().with_variant(LONGTEXT(), "mysql"), nullable=False)
    deadline: Mapped[float | None] = mapped_column(Double, nullable=True, index=True)
    last_activity_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=_now, index=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=_now)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=_now, index=True)
class RoomMember(Base):
    __tablename__ = "room_members"
    user_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=False)
    room_id: Mapped[int] = mapped_column(
        Integer, ForeignKey("rooms.id", ondelete="CASCADE"), index=True
    )
    joined_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=_now)
class PackCategory(Base):
    __tablename__ = "pack_categories"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    title: Mapped[str] = mapped_column(String(64), unique=True)
class QuestionPack(Base):
    __tablename__ = "question_packs"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    slug: Mapped[str] = mapped_column(String(32), unique=True, index=True)
    title: Mapped[str] = mapped_column(String(100))
    description: Mapped[str] = mapped_column(String(255), default="")
    category_id: Mapped[int] = mapped_column(Integer, ForeignKey("pack_categories.id"), index=True)
    owner_id: Mapped[int | None] = mapped_column(BigInteger, nullable=True, index=True)
    owner_name: Mapped[str | None] = mapped_column(String(40), nullable=True)
    is_public: Mapped[bool] = mapped_column(Boolean, default=True)
    is_hidden: Mapped[bool] = mapped_column(Boolean, default=False, server_default="0")
    position: Mapped[int] = mapped_column(Integer, default=0)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=_now)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=_now, onupdate=_now)
class PackWord(Base):
    __tablename__ = "pack_words"
    __table_args__ = (UniqueConstraint("pack_id", "word", name="uq_pack_word"),)
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    pack_id: Mapped[int] = mapped_column(
        Integer, ForeignKey("question_packs.id", ondelete="CASCADE"), index=True
    )
    word: Mapped[str] = mapped_column(String(64))
class GameSession(Base):
    __tablename__ = "game_sessions"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    room_id: Mapped[int | None] = mapped_column(Integer, nullable=True, index=True)
    room_code: Mapped[str] = mapped_column(String(8))
    title: Mapped[str] = mapped_column(String(100))
    host_id: Mapped[int] = mapped_column(BigInteger)
    status: Mapped[str] = mapped_column(String(16), default="active", index=True)
    is_test: Mapped[bool] = mapped_column(Boolean, default=False, server_default="0")
    started_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=_now, index=True)
    ended_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)
class SessionPlayer(Base):
    __tablename__ = "session_players"
    session_id: Mapped[int] = mapped_column(
        Integer, ForeignKey("game_sessions.id", ondelete="CASCADE"), primary_key=True
    )
    user_id: Mapped[int] = mapped_column(
        BigInteger, primary_key=True, autoincrement=False, index=True
    )
    name: Mapped[str] = mapped_column(String(40))
    score: Mapped[int] = mapped_column(Integer, default=0)
class MatchRound(Base):
    __tablename__ = "match_rounds"
    __table_args__ = (UniqueConstraint("session_id", "round_no", name="uq_session_round"),)
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    session_id: Mapped[int] = mapped_column(
        Integer, ForeignKey("game_sessions.id", ondelete="CASCADE"), index=True
    )
    round_no: Mapped[int] = mapped_column(Integer)
    theme: Mapped[str] = mapped_column(String(100))
    word: Mapped[str] = mapped_column(String(64))
    pack_id: Mapped[int | None] = mapped_column(Integer, nullable=True)
    pack_title: Mapped[str | None] = mapped_column(String(100), nullable=True)
    spy_user_id: Mapped[int] = mapped_column(BigInteger)
    spy_name: Mapped[str] = mapped_column(String(40))
    outcome: Mapped[str] = mapped_column(String(24))
    finished_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=_now, index=True)
class RoundPlayer(Base):
    __tablename__ = "round_players"
    round_id: Mapped[int] = mapped_column(
        Integer, ForeignKey("match_rounds.id", ondelete="CASCADE"), primary_key=True
    )
    user_id: Mapped[int] = mapped_column(
        BigInteger, primary_key=True, autoincrement=False, index=True
    )
    name: Mapped[str] = mapped_column(String(40))
    role: Mapped[str] = mapped_column(String(8))
    result: Mapped[str] = mapped_column(String(8))
    score_delta: Mapped[int] = mapped_column(Integer, default=0)
class UserFavorite(Base):
    __tablename__ = "user_favorites"
    user_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=False)
    pack_id: Mapped[int] = mapped_column(
        Integer, ForeignKey("question_packs.id", ondelete="CASCADE"), primary_key=True
    )
class PackReport(Base):
    __tablename__ = "pack_reports"
    __table_args__ = (UniqueConstraint("pack_id", "reporter_id", name="uq_pack_reporter"),)
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    pack_id: Mapped[int] = mapped_column(
        Integer, ForeignKey("question_packs.id", ondelete="CASCADE"), index=True
    )
    reporter_id: Mapped[int] = mapped_column(BigInteger)
    reason: Mapped[str] = mapped_column(String(40))
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=_now)
class UserSetting(Base):
    __tablename__ = "user_settings"
```

```
    user_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=False)
    tz_offset_min: Mapped[int] = mapped_column(Integer, default=0)
    emoji: Mapped[bool] = mapped_column(Boolean, default=False, server_default="0")
    colors: Mapped[bool] = mapped_column(Boolean, default=False, server_default="0")
class UserPenalty(Base):
    __tablename__ = "user_penalties"
    user_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=False)
    strikes: Mapped[int] = mapped_column(Integer, default=0)
    banned_until: Mapped[float] = mapped_column(Double, default=0.0)
    last_leave_at: Mapped[float] = mapped_column(Double, default=0.0)
    reason: Mapped[str] = mapped_column(String(16), default="leave", server_default="leave")
pack_repository.py
from sqlalchemy import select
from sqlalchemy.orm import sessionmaker
from app.database.models import PackCategory, PackWord, QuestionPack
from app.game.packs import Pack, PackLibrary
class PackRepository:
    def __init__(self, factory: sessionmaker):
        self._factory = factory
    def seed(self, packs: list[dict]) -> int:
        """Explicitly import packs; never called automatically at application startup."""
        created = 0
        with self._factory.begin() as db:
            for position, item in enumerate(packs):
                category = db.execute(
                    select(PackCategory).where(PackCategory.title == item["category"])
                ).scalar_one_or_none()
                if category is None:
                    category = PackCategory(title=item["category"])
                    db.add(category)
                    db.flush()
                pack = db.execute(
                    select(QuestionPack).where(QuestionPack.slug == item["id"])
                ).scalar_one_or_none()
                if pack is None:
                    pack = QuestionPack(
                        slug=item["id"],
                        title=item["title"],
                        description=item["description"],
                        category_id=category.id,
                        owner_id=None,
                        is_public=True,
                        position=position,
                    )
                    db.add(pack)
                    db.flush()
                    created += 1
                elif pack.owner_id is None:
                    pack.title = item["title"]
                    pack.description = item["description"]
                    pack.category_id = category.id
                    pack.position = position
                existing = {
                    row[0]
                    for row in db.execute(select(PackWord.word).where(PackWord.pack_id == pack.id))
                }
                for word in item["words"]:
                    if word not in existing:
                        db.add(PackWord(pack_id=pack.id, word=word))
                        existing.add(word)
        return created
    def load_library(self) -> PackLibrary:
        with self._factory() as db:
            categories = {c.id: c.title for c in db.execute(select(PackCategory)).scalars()}
            packs = db.execute(
                select(QuestionPack)
                .where(QuestionPack.owner_id.is_(None))
                .order_by(QuestionPack.position, QuestionPack.id)
            ).scalars().all()
            words: dict[int, list[str]] = {}
            for row in db.execute(select(PackWord.pack_id, PackWord.word).order_by(PackWord.id)):
                words.setdefault(row[0], []).append(row[1])
            return PackLibrary(
                [
                    Pack(
                        id=p.slug,
                        title=p.title,
                        description=p.description,
                        category=categories.get(p.category_id, ""),
                        words=tuple(words.get(p.id, [])),
                    )
                    for p in packs
                    if words.get(p.id)
                ]
            )
repository.py
import json
from datetime import datetime, timedelta, timezone
from sqlalchemy import delete
from sqlalchemy.orm import sessionmaker
from app.database.models import LocalGameRecord
class LocalGameRepository:
    def __init__(self, factory: sessionmaker):
        self._factory = factory
    def load(self, owner_id: int) -> dict | None:
        with self._factory() as db:
            record = db.get(LocalGameRecord, owner_id)
            return json.loads(record.state) if record else None
    def save(self, owner_id: int, state: dict) -> None:
        payload = json.dumps(state, ensure_ascii=False)
        with self._factory.begin() as db:
            record = db.get(LocalGameRecord, owner_id)
            if record:
                record.state = payload
            else:
                db.add(LocalGameRecord(owner_id=owner_id, state=payload))
    def delete(self, owner_id: int) -> None:
        with self._factory.begin() as db:
            record = db.get(LocalGameRecord, owner_id)
            if record:
                db.delete(record)
    def purge_stale(self, days: int = 30) -> int:
        cutoff = datetime.now(timezone.utc) - timedelta(days=days)
        with self._factory.begin() as db:
            result = db.execute(delete(LocalGameRecord).where(LocalGameRecord.updated_at < cutoff))
            return result.rowcount or 0
session.py
from sqlalchemy import create_engine, inspect, text
from sqlalchemy.orm import sessionmaker
from app.database.models import Base
COLUMN_MIGRATIONS = {
    "rooms": [("last_activity_at", "DATETIME NULL")],
    "match_rounds": [("pack_id", "INTEGER NULL"), ("pack_title", "VARCHAR(100) NULL")],
    "game_sessions": [("is_test", "BOOLEAN NOT NULL DEFAULT 0")],
    "user_penalties": [("reason", "VARCHAR(16) NOT NULL DEFAULT 'leave'")],
    "user_settings": [
        ("emoji", "BOOLEAN NOT NULL DEFAULT 0"),
        ("colors", "BOOLEAN NOT NULL DEFAULT 0"),
    ],
    "question_packs": [
        ("is_hidden", "BOOLEAN NOT NULL DEFAULT 0"),
        ("owner_name", "VARCHAR(40) NULL"),
    ],
}
def ensure_schema(engine) -> None:
    inspector = inspect(engine)
    for table, columns in COLUMN_MIGRATIONS.items():
        if not inspector.has_table(table):
            continue
        existing = {col["name"] for col in inspector.get_columns(table)}
        with engine.begin() as conn:
            for name, ddl in columns:
                if name not in existing:
                    conn.execute(text(f"ALTER TABLE {table} ADD COLUMN {name} {ddl}"))
def make_session_factory(url: str) -> sessionmaker:
    connect_args = {}
    if url.startswith("mysql+pymysql:"):
        connect_args = {"connect_timeout": 10, "read_timeout": 30, "write_timeout": 30}
    engine = create_engine(
        url, future=True, pool_pre_ping=True, pool_recycle=1800,
        connect_args=connect_args,
    )
    Base.metadata.create_all(engine)
    ensure_schema(engine)
    return sessionmaker(engine, expire_on_commit=False)
worker.py
"""Run complete synchronous service calls away from Telegram's event loop.
One worker preserves the ordering and cache semantics of the sync services.
```

```
Sessions/transactions are created and closed inside each service call, never
passed between threads. The admission limit also bounds the executor queue.
"""
import asyncio
from concurrent.futures import ThreadPoolExecutor
from functools import partial
from weakref import WeakKeyDictionary
_executor = ThreadPoolExecutor(max_workers=1, thread_name_prefix="bot-db")
_limits = WeakKeyDictionary()
async def run_db(function, /, *args, **kwargs):
    loop = asyncio.get_running_loop()
    limit = _limits.setdefault(loop, asyncio.Semaphore(32))
    await limit.acquire()
    try:
        future = loop.run_in_executor(_executor, partial(function, *args, **kwargs))
    except BaseException:
        limit.release()
        raise
    # Cancelling a handler cannot interrupt a running SQL transaction. Keep its
    # admission slot until the actual operation completes, including on cancel.
    def finished(done):
        limit.release()
        if not done.cancelled():
            done.exception()
    future.add_done_callback(finished)
    return await asyncio.shield(future)
__init__.py
engine.py
from __future__ import annotations
from dataclasses import dataclass, field, fields
from enum import Enum
from app.game.scoring import Outcome, ScoreRules, compute_scores
from app.game.voting import majority_reached, tally
class GameError(Exception):
    pass
class Phase(str, Enum):
    ROLE_DEAL = "role_deal"
    DISCUSSION = "discussion"
    SUSPECT_VOTE = "suspect_vote"
    RUNOFF_VOTE = "runoff_vote"
    LAST_CHANCE = "last_chance"
    FINISHED = "finished"
def normalize(word: str) -> str:
    return " ".join(word.casefold().split())
@dataclass
class Round:
    players: list[str]
    spy_id: str
    word: str
    theme: str
    allowed_words: list[str]
    order: list[str]
    guesses_left: int = 1
    phase: Phase = Phase.ROLE_DEAL
    turn_idx: int = 0
    theme_id: str = ""
    open_guess: bool = False
    circle_no: int = 1
    circle_votes: dict[str, bool] = field(default_factory=dict)
    votes: dict[str, str] = field(default_factory=dict)
    candidates: list[str] = field(default_factory=list)
    secret_guess_word: str | None = None
    secret_guess_correct: bool | None = None
    outcome: Outcome | None = None
    settled: bool = False
    def _require_player(self, player_id: str) -> None:
        if player_id not in self.players:
            raise GameError("Игрок не участвует в раунде")
    def _require_phase(self, *phases: Phase) -> None:
        if self.phase not in phases:
            raise GameError("Действие недоступно в текущей фазе")
    def finish_deal(self) -> None:
        self._require_phase(Phase.ROLE_DEAL)
        self.phase = Phase.DISCUSSION
    def current(self) -> str:
        return self.order[self.turn_idx % len(self.order)]
    def next_player(self) -> str:
        return self.order[(self.turn_idx + 1) % len(self.order)]
    def end_turn(self, player_id: str, wants_vote: bool = False) -> str:
        self._require_phase(Phase.DISCUSSION)
        if player_id != self.current():
            raise GameError("Сейчас не ваш ход")
        self.circle_votes[player_id] = bool(wants_vote)
        self.turn_idx += 1
        if self.turn_idx < len(self.order):
            return "next"
        return self._close_circle()
    def _close_circle(self) -> str:
        self.turn_idx = 0
        yes = sum(1 for value in self.circle_votes.values() if value)
        self.circle_votes = {}
        if majority_reached(yes, len(self.players)):
            if self.secret_guess_correct is True:
                self._finish(Outcome.SPY_WIN_GUESS)
                return "spy_won"
            if self.secret_guess_correct is False:
                self._finish(Outcome.SPY_WRONG_GUESS)
                return "spy_lost"
            self.phase = Phase.SUSPECT_VOTE
            self.votes = {}
            return "vote"
        self.circle_no += 1
        return "new_circle"
    def spy_left(self) -> None:
        self._finish(Outcome.SPY_LEFT)
    def remove_player(self, player_id: str) -> str:
        if player_id == self.spy_id:
            raise GameError("Шпион уходит через spy_left")
        self._require_player(player_id)
        if self.phase == Phase.FINISHED:
            raise GameError("Раунд уже завершён")
        index = self.order.index(player_id)
        self.players.remove(player_id)
        self.order.remove(player_id)
        self.circle_votes.pop(player_id, None)
        self.votes = {
            voter: target
            for voter, target in self.votes.items()
            if voter != player_id and target != player_id
        }
        if self.phase == Phase.DISCUSSION:
            if index < self.turn_idx:
                self.turn_idx -= 1
            if self.turn_idx >= len(self.order):
                return self._close_circle()
            return "next"
        if self.phase == Phase.RUNOFF_VOTE and player_id in self.candidates:
            self.candidates.remove(player_id)
            if len(self.candidates) <= 1:
                survivor = self.candidates[0] if self.candidates else None
                if self.secret_guess_correct is True:
                    self._finish(Outcome.DRAW)
                elif survivor == self.spy_id:
                    self._finish(Outcome.CIVILIANS_WIN)
                else:
                    self._finish(Outcome.SPY_WIN_VOTE)
                return "finished"
        if self.phase in (Phase.SUSPECT_VOTE, Phase.RUNOFF_VOTE):
            if self.players and len(self.votes) == len(self.players):
                self._resolve_votes()
        return "next"
    def secret_guess(self, player_id: str, word: str) -> None:
        if self.open_guess or self.phase not in (
            Phase.DISCUSSION,
            Phase.SUSPECT_VOTE,
            Phase.RUNOFF_VOTE,
        ):
            return
        if player_id != self.spy_id or self.guesses_left <= 0:
            return
        if normalize(word) not in {normalize(w) for w in self.allowed_words}:
            return
        self.guesses_left -= 1
        self.secret_guess_word = word
        self.secret_guess_correct = normalize(word) == normalize(self.word)
    def valid_targets(self, voter_id: str) -> list[str]:
        pool = self.candidates if self.phase == Phase.RUNOFF_VOTE else self.players
        return [p for p in pool if p != voter_id]
    def cast_vote(self, voter_id: str, target_id: str) -> None:
        self._require_phase(Phase.SUSPECT_VOTE, Phase.RUNOFF_VOTE)
        self._require_player(voter_id)
        if target_id not in self.valid_targets(voter_id):
            raise GameError("Недопустимая цель голосования")
        self.votes[voter_id] = target_id
        if len(self.votes) == len(self.players):
            self._resolve_votes()
    def _resolve_votes(self) -> None:
        _, leaders = tally(self.votes)
        guessed = self.secret_guess_correct is True
        if len(leaders) == 1:
            if guessed:
                self._finish(Outcome.DRAW)
            elif leaders[0] == self.spy_id:
```

```
self._finish(Outcome.CIVILIANS_WIN)
        else:

self._finish(Outcome.SPY_WIN_VOTE)
    elif self.phase == Phase.SUSPECT_VOTE:
        self.candidates = leaders
        self.votes = {}
        self.phase = Phase.RUNOFF_VOTE
    else:
        self._finish(Outcome.DRAW if guessed else Outcome.SPY_WIN_VOTE)
  def guess(self, player_id: str, word: str) -> bool:
    allowed = [Phase.SUSPECT_VOTE, Phase.RUNOFF_VOTE, Phase.LAST_CHANCE]
    if self.open_guess:

allowed.append(Phase.DISCUSSION)
    self._require_phase(*allowed)
    if player_id != self.spy_id:
        raise GameError("Угадывать слово может только шпион")
    if self.guesses_left <= 0:
        raise GameError("Попытки закончились")
    if normalize(word) not in {normalize(w) for w in self.allowed_words}:
        raise GameError("Слово не входит в выбранные темы")
    self.guesses_left -= 1
    correct = normalize(word) == normalize(self.word)
    if self.phase == Phase.LAST_CHANCE:
        self._finish(Outcome.DRAW if correct else Outcome.CIVILIANS_WIN)
    elif self.phase == Phase.DISCUSSION:

self._finish(Outcome.SPY_WIN_GUESS if correct else Outcome.SPY_WRONG_GUESS)
    elif correct:
        self._finish(Outcome.DRAW)
    return correct
  def skip_last_chance(self) -> None:

self._require_phase(Phase.LAST_CHANCE)

self._finish(Outcome.CIVILIANS_WIN)
  def _finish(self, outcome: Outcome) -> None:
    if self.phase == Phase.FINISHED:
        raise GameError("Раунд уже завершён")
    self.outcome = outcome
    self.phase = Phase.FINISHED
  def settle(self, rules: ScoreRules) -> dict[str, int]:

self._require_phase(Phase.FINISHED)
    if self.settled or self.outcome is None:
        raise GameError("Очки за раунд уже начислены")
    self.settled = True
    return compute_scores(self.outcome, self.spy_id, self.players, rules)
  def to_dict(self) -> dict:
    return {
        "players": list(self.players),
        "spy_id": self.spy_id,
        "word": self.word,
        "theme": self.theme,
        "allowed_words": list(self.allowed_words),
        "order": list(self.order),
        "guesses_left": self.guesses_left,
        "phase": self.phase.value,
        "turn_idx": self.turn_idx,
        "theme_id": self.theme_id,
        "open_guess": self.open_guess,
        "circle_no": self.circle_no,
        "circle_votes": dict(self.circle_votes),
        "votes": dict(self.votes),
        "candidates": list(self.candidates),
        "secret_guess_word": self.secret_guess_word,
        "secret_guess_correct": self.secret_guess_correct,
        "outcome": self.outcome.value if self.outcome else None,
        "settled": self.settled,
    }
  @classmethod
  def from_dict(cls, data: dict) -> Round:
    known = {f.name for f in fields(cls)}
    payload = {k: v for k, v in data.items() if k in known}
    payload["phase"] = Phase(payload["phase"])
    payload["outcome"] = Outcome(payload["outcome"]) if payload.get("outcome") else None
    return cls(**payload)
packs.py
from __future__ import annotations
from dataclasses import dataclass
@dataclass(frozen=True)
class Pack:
  id: str
  title: str
  description: str
  category: str
  words: tuple[str, ...]
class PackLibrary:
  def __init__(self, packs: list[Pack]):
    self._packs = {pack.id: pack for pack in packs}
    self._order = [pack.id for pack in packs]
  def with_packs(self, extra: list[Pack]) -> PackLibrary:
    merged = dict(self._packs)
    order = list(self._order)
    for pack in extra:
      if pack.id not in merged:
        order.append(pack.id)
      merged[pack.id] = pack
    return PackLibrary([merged[i] for i in order])
  def packs(self) -> list[Pack]:
    return [self._packs[pack_id] for pack_id in self._order]
  def get(self, pack_id: str) -> Pack:
    return self._packs[pack_id]
  def has(self, pack_id: str) -> bool:
    return pack_id in self._packs
  def words(self, pack_ids: list[str]) -> list[str]:
    seen: dict[str, None] = {}
    for pack_id in pack_ids:
      for word in self._packs[pack_id].words:
        seen.setdefault(word, None)
    return list(seen)
def load_default_library() -> PackLibrary:
  """Return an empty library; production code must inject the MySQL library."""
  return PackLibrary([])
roles.py
import random
def choose_spy(players: list[str], previous_spy: str | None, rng: random.Random) -> str:
  candidates = [p for p in players if p != previous_spy] or list(players)
  return rng.choice(candidates)
def build_turn_order(players: list[str], spy_id: str, rng: random.Random) -> list[str]:
  order = list(players)
  rng.shuffle(order)
  civilians = [p for p in players if p != spy_id]
  if civilians and order[0] == spy_id:
    first = rng.choice(civilians)
    order.remove(first)
    order.insert(0, first)
  return order
def pick_word(words: list[str], used: list[str], rng: random.Random) -> str:
  fresh = [w for w in words if w not in used]
  return rng.choice(fresh or list(words))
room.py
from __future__ import annotations
import html
import math
import random
import secrets
import time
from urllib.parse import urlencode
from dataclasses import dataclass
from app.game.engine import GameError, Phase, Round
from app.game.packs import PackLibrary, load_default_library
from app.game.roles import choose_spy
from app.game.roundfactory import create_round
from app.game.scoring import ScoreRules
from app.game.session import REASONS, Screen
from app.game.scoring import Outcome
CODE_ALPHABET = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789"
MIN_PLAYERS = 3
LAST_CHANCE_SECONDS = 30
MAX_NAME_LEN = 20
MAX_MESSAGE_LEN = 200
CHAT_KEEP = 8
ROLE_OPTIONS = ["random", "spy", "civilian"]
ROLE_LABELS = {"random": "случайная", "spy": "шпион", "civilian": "обычный игрок"}
PAGE_SIZE = 8
THEMES_PAGE = 8
OPTIONS = {
  "max": ("max_players", [4, 6, 8, 10, 12]),
  "rounds": ("rounds", [3, 5, 7, 10]),
  "turn": ("turn_seconds", [30, 60, 90, 120]),
}
class RoomError(Exception):
  pass
@dataclass
class ActResult:
  alert: str | None = None
  changed: bool = True
  left: bool = False
  closed: bool = False
  penalized: bool = False
def new_code() -> str:
  return "".join(secrets.choice(CODE_ALPHABET) for _ in range(6))
```

```
def clean_name(raw: str) -> str:
    name = " ".join((raw or "").split())[:MAX_NAME_LEN]
    return name or "Игрок"
def esc(text: str) -> str:
    return html.escape(text, quote=False)
class Room:
    def __init__(
        self,
        code: str,
        token: str,
        host_id: int,
        title: str,
        library: PackLibrary | None = None,
        rng: random.Random | None = None,
        rules: ScoreRules | None = None,
    ):
        self.library = library or load_default_library()
        self.rng = rng or random.Random()
        self.rules = rules or ScoreRules()
        self.code = code
        self.token = token
        self.host_id = host_id
        self.title = title
        self.settings = {
            "max_players": 8,
            "rounds": 5,
            "turn_seconds": 60,
            "themes": [],
            "random_theme": False,
        }
        self.players: list[dict] = []
        self.scores: dict[str, int] = {}
        self.phase = "lobby"
        self.round_no = 0
        self.round: Round | None = None
        self.deadline: float | None = None
        self.chat: list[list] = []
        self.used_words: list[str] = []
        self.previous_spy: str | None = None
        self.history: list[dict] = []
        self.last_deltas: dict[str, int] = {}
        self.guess_ui: dict[str, dict] = {}
        self.host_ui = "lobby"
        self.themes_page = 0
        self.themes_notice = False
        self.session_id: int | None = None
    def enable_test_mode(self, size: int, admin_id: int) -> None:
        self.settings["test_size"] = size
        self.settings["test_admin"] = admin_id
        self.settings["test_role"] = "random"
        self.settings["max_players"] = size
        self.settings["min_players"] = size
        self.title = f"Тестовое лобби на {size}"
    @property
    def is_test(self) -> bool:
        return bool(self.settings.get("test_size"))
    def _p(self, uid: int) -> dict | None:
        for player in self.players:
            if player["id"] == uid:
                return player
        return None
    def _member(self, uid: int) -> dict:
        player = self._p(uid)
        if player is None:
            raise RoomError("Вы не участник этой комнаты")
        return player
    def _require_host(self, uid: int) -> None:
        self._member(uid)
        if uid != self.host_id:
            raise RoomError("Это действие доступно только хосту")
    def _require_spy(self, uid: int) -> None:
        if self.round is None or str(uid) != self.round.spy_id:
            raise RoomError("Вы не шпион: попытка недоступна")
    def name_of(self, pid: str) -> str:
        player = self._p(int(pid))
        return player["name"] if player else "?"
    def _note(self, text: str) -> None:
        self.chat.append([None, text])
        del self.chat[:-CHAT_KEEP * 3]
    def prune_missing_themes(self) -> None:
        if self.phase != "lobby":
            return
        selected = self.settings["themes"]
        available = [slug for slug in selected if self.library.has(slug)]
        if len(available) != len(selected):
            self.settings["themes"] = available
            self.themes_notice = True
    def _themes_notice(self) -> str:
        return "\n\nОдин или несколько выбранных наборов больше недоступны и удалены из выбора. Выберите замену в настройках." if self.themes_notice else ""
    def themes_label(self) -> str:
        if not self.settings["themes"]:
            return "не выбраны"
        return ", ".join(self.library.get(t).title for t in self.settings["themes"] if self.library.has(t)) or "не выбраны"
    def table(self) -> str:
        ranked = sorted(self.players, key=lambda p: (-self.scores.get(str(p["id"]), 0), p["name"]))
        return "\n".join(
            f"{i}. {esc(p['name'])} — {self.scores.get(str(p['id']), 0)}"
            for i, p in enumerate(ranked, 1)
        )
    def join(self, uid: int, name: str) -> None:
        if self._p(uid):
            return
        if self.phase == "final":
            raise RoomError("Лобби уже завершено и больше неактивно. Попросите новое приглашение.")
        if self.phase != "lobby":
            raise RoomError("Игра уже идёт. Присоединиться можно, пока комната в лобби")
        if len(self.players) >= self.settings["max_players"]:
            raise RoomError("Комната заполнена")
        base = clean_name(name)
        taken = {p["name"].casefold() for p in self.players}
        unique, i = base, 2
        while unique.casefold() in taken:
            unique = f"{base} ({i})"
            i += 1
        self.players.append(
            {"id": uid, "name": unique, "ready": uid == self.host_id, "panel": None, "seen": False}
        )
        self.scores.setdefault(str(uid), 0)
        if self.round_no or len(self.players) > 1:
            self._note(f"{unique} присоединился")
    def act(self, uid: int, action: str, now: float) -> ActResult:
        self._member(uid)
        name, _, arg = action.partition(":")
        try:
            return self._act(uid, name, arg, now)
        except GameError as err:
            raise RoomError(str(err)) from err
    def _act(self, uid: int, name: str, arg: str, now: float) -> ActResult:
        if name == "leave":
            return self._leave(uid, now)
        if name == "rd":
            self._need_phase("lobby")
            self._member(uid)["ready"] = arg == "1"
            return ActResult()
        if name == "ui" and arg in ("lobby", "settings", "themes"):
            self._require_host(uid)
            self._need_phase("lobby")
            self.host_ui = arg
            return ActResult()
        if name == "cy" and arg == "role":
            self._require_host(uid)
            if not self.is_test:
                raise RoomError("Выбор роли доступен только в тестовом лобби")
            if self.phase not in ("lobby", "result"):
                raise RoomError("Это действие сейчас недоступно")
            current = self.settings.get("test_role", "random")
            index = ROLE_OPTIONS.index(current) if current in ROLE_OPTIONS else -1
            self.settings["test_role"] = ROLE_OPTIONS[(index + 1) % len(ROLE_OPTIONS)]
            return ActResult()
        if name == "cy" and arg in OPTIONS:
            self._require_host(uid)
            if arg == "max" and self.is_test:
                raise RoomError("В тестовом лобби число игроков фиксировано")
            self._need_phase("lobby")
            self._cycle(arg)
            return ActResult()
        if name == "th" and self.library.has(arg):
            self._require_host(uid)
            self._need_phase("lobby")
            self.themes_notice = False
            themes = self.settings["themes"]
            if arg in themes:
                themes.remove(arg)
            else:
                themes.append(arg)
            return ActResult()
        if name == "noop":
            return ActResult(changed=False)
        if name == "sv":
            self._require_host(uid)
            self._need_phase("lobby")
            key, _, raw = arg.partition(":")
            if key not in OPTIONS or not raw.isdigit() or int(raw) not in OPTIONS[key][1]:
                raise RoomError("Недопустимое значение")
            value = int(raw)
            if key == "max":
                if self.is_test:
                    raise RoomError("В тестовом лобби число игроков фиксировано")
                if value < len(self.players):
                    raise RoomError("В комнате уже больше игроков")
            self.settings[OPTIONS[key][0]] = value
            return ActResult()
        if name in ("tall", "tnone"):
            self._require_host(uid)
            self._need_phase("lobby")
```

```
            self.themes_notice = False
            self.settings["themes"] = (
                [pack.id for pack in self.library.packs()] if name == "tall" else []
            )
            return ActResult()
        if name == "tp" and arg.isdigit():
            self._require_host(uid)
            self._need_phase("lobby")
            self.themes_page = int(arg)
            return ActResult()
        if name == "rt":
            self._require_host(uid)
            self._need_phase("lobby")
            self.settings["random_theme"] = not self.settings["random_theme"]
            return ActResult()
        if name in ("start", "next"):
            self._require_host(uid)
            self._start_round()
            return ActResult()
        if name == "set":
            self._require_host(uid)
            self._need_phase("result")
            self.phase = "lobby"
            self.host_ui = "settings"
            return ActResult()
        if name == "end":
            self._require_host(uid)
            self._need_phase("result")
            self.phase = "final"
            return ActResult()
        if name == "role":
            return self._role(uid, now)
        if name == "force":
            self._require_host(uid)
            self._need_phase("dealing")
            self._begin_discussion(now)
            return ActResult()
        if name == "endturn":
            self._need_playing_discussion()
            current = self.round.current()
            if str(uid) != current and uid != self.host_id:
                raise RoomError("Сейчас не ваш ход")
            event = self.round.end_turn(current, arg == "1" and str(uid) == current)
            self._apply_turn_event(event, now)
            return ActResult()
        if name == "v":
            self._need_phase("playing")
            before = self.round.phase
            self.round.cast_vote(str(uid), arg)
            self._after_event(before, now)
            return ActResult()
        if name == "gu":
            self._need_phase("playing")
            if self.round.phase not in (
                Phase.DISCUSSION,
                Phase.SUSPECT_VOTE,
                Phase.RUNOFF_VOTE,
                Phase.LAST_CHANCE,
            ):
                raise RoomError("Это действие сейчас недоступно")
            self._require_spy(uid)
            if self.round.guesses_left <= 0:
                raise RoomError("Попытки угадать слово закончились")
            self.guess_ui.setdefault(str(uid), {"page": 0, "word": None})
            return ActResult()
        if name in ("pg", "gp", "no", "cancel", "yes", "skip"):
            return self._guess_action(uid, name, arg)
        raise RoomError("Это действие сейчас недоступно")
    def _need_phase(self, phase: str) -> None:
        if self.phase != phase:
            raise RoomError("Это действие сейчас недоступно")
    def _need_playing_discussion(self) -> None:
        if self.phase != "playing" or self.round is None or self.round.phase != Phase.DISCUSSION:
            raise RoomError("Это действие сейчас недоступно")
    def _cycle(self, key: str) -> None:
        field_name, options = OPTIONS[key]
        if field_name == "max_players":
            options = [o for o in options if o >= len(self.players)] or [options[-1]]
        current = self.settings[field_name]
        index = options.index(current) if current in options else -1
        self.settings[field_name] = options[(index + 1) % len(options)]
    def _start_round(self) -> None:
        if self.phase not in ("lobby", "result"):
            raise RoomError("Раунд уже идёт")
        if self.round_no >= self.settings["rounds"]:
            raise RoomError("Все раунды сыграны. Завершите игру")
        minimum = self.settings.get("min_players", MIN_PLAYERS)
        if len(self.players) < minimum:
            raise RoomError(f"Недостаточно игроков: нужно минимум {minimum}")
        if not self.settings["themes"]:
            raise RoomError("Выберите хотя бы одну тему в настройках")
        if self.round_no == 0 and any(not p["ready"] for p in self.players):
            raise RoomError("Не все игроки готовы")
        pids = [str(p["id"]) for p in self.players]
        spy_override = self._forced_spy(pids)
        try:
            self.round, self.used_words = create_round(
                self.library,
                self.rng,
                pids,
                self.settings["themes"],
                self.settings["random_theme"],
                self.used_words,
                self.previous_spy,
                spy_override=spy_override,
                round_no=self.round_no + 1,
                previous_theme=self.round.theme_id if self.round else None,
                open_guess=True,
            )
        except ValueError as err:
            raise RoomError(str(err)) from err
        self.previous_spy = self.round.spy_id
        self.round_no += 1
        self.phase = "dealing"
        self.deadline = None
        self.guess_ui = {}
        self.host_ui = "lobby"
        self.chat = []
        for player in self.players:
            player["seen"] = False
            self.scores.setdefault(str(player["id"]), 0)
    def _forced_spy(self, pids: list[str]) -> str | None:
        if not self.is_test:
            return None
        role = self.settings.get("test_role", "random")
        admin = str(self.settings.get("test_admin"))
        if role == "random" or admin not in pids:
            return None
        if role == "spy":
            return admin
        others = [p for p in pids if p != admin]
        if not others:
            raise RoomError("В лобби на 1 игрока возможна только роль шпиона или случайная")
        return choose_spy(others, self.previous_spy, self.rng)
    def _role_label(self) -> str:
        return ROLE_LABELS.get(self.settings.get("test_role", "random"), "случайная")
    def _role(self, uid: int, now: float) -> ActResult:
        if self.phase not in ("dealing", "playing") or self.round is None:
            raise RoomError("Роль сейчас недоступна")
        if self.round.phase == Phase.FINISHED:
            raise RoomError("Раунд завершён")
        player = self._member(uid)
        first = not player["seen"]
        player["seen"] = True
        if str(uid) == self.round.spy_id:
            text = "Вы шпион! Определите загаданное слово."
        else:
            text = f"Загаданное слово:\n\n{self.round.word.upper()}"
        changed = False
        if self.phase == "dealing":
            changed = first
            if all(p["seen"] for p in self.players):
                self._begin_discussion(now)
                changed = True
        return ActResult(alert=text, changed=changed)
    def _begin_discussion(self, now: float) -> None:
        self.round.finish_deal()
        self.phase = "playing"
        self.deadline = now + self.settings["turn_seconds"]
        self.chat = []
        self._note("Обсуждение началось")
    def current(self) -> str:
        return self.round.current()
    def _apply_turn_event(self, event: str, now: float) -> None:
        if self.round.phase == Phase.FINISHED:
            self._finish_round()
            return
        if event == "vote":
            self.deadline = None
            self._note("Круг завершён: большинство за голосование. Голосование началось")
            return
        self.deadline = now + self.settings["turn_seconds"]
        if event == "new_circle":
```

```
            self._note(f"Круг {self.round.circle_no - 1} завершён: голосования не будет")
    def tick(self, now: float) -> bool:
        if (
            self.phase != "playing"
            or self.round is None
            or self.deadline is None
            or now < self.deadline
        ):
            return False
        if self.round.phase == Phase.DISCUSSION:
            event = self.round.end_turn(self.round.current(), False)
            self._apply_turn_event(event, now)
            return True
        if self.round.phase == Phase.LAST_CHANCE:
            self.round.skip_last_chance()
            self._finish_round()
            return True
        return False
    def say(self, uid: int, text: str, now: float) -> None:
        player = self._member(uid)
        if self.phase != "playing" or self.round is None or self.round.phase != Phase.DISCUSSION:
            raise RoomError("Сейчас нельзя писать в чат")
        if str(uid) != self.round.current():
            raise RoomError("Писать можно только в свой ход")
        cleaned = " ".join(text.split())
        if not cleaned:
            raise RoomError("Пустое сообщение")
        if len(cleaned) > MAX_MESSAGE_LEN:
            raise RoomError(f"Слишком длинное сообщение (максимум {MAX_MESSAGE_LEN} символов)")
        self.chat.append([player["name"], cleaned])
        del self.chat[:-CHAT_KEEP * 3]
    def _after_event(self, phase_before: Phase, now: float) -> None:
        phase = self.round.phase
        if phase == Phase.FINISHED:
            self._finish_round()
        elif phase == Phase.LAST_CHANCE:
            self.guess_ui = {self.round.spy_id: {"page": 0, "word": None}}
            self.deadline = now + LAST_CHANCE_SECONDS
        elif phase == Phase.RUNOFF_VOTE and phase_before != Phase.RUNOFF_VOTE:
            self._note("Ничья: дополнительное голосование")
    def _guess_words(self) -> list[str]:
        return sorted(self.round.allowed_words, key=str.casefold)
    def _guess_action(self, uid: int, name: str, arg: str) -> ActResult:
        self._need_phase("playing")
        me = str(uid)
        ui = self.guess_ui.get(me)
        if ui is None:
            raise RoomError("Панель угадывания закрыта")
        last_chance = self.round.phase == Phase.LAST_CHANCE
        words = self._guess_words()
        if name == "pg" and arg.isdigit():
            ui["page"] = int(arg)
            return ActResult()
        if name == "gp" and arg.isdigit() and int(arg) < len(words):
            ui["word"] = words[int(arg)]
            return ActResult()
        if name == "no":
            ui["word"] = None
            return ActResult()
        if name == "cancel":
            if last_chance:
                raise RoomError("Сейчас можно только угадать слово или отказаться")
            self.guess_ui.pop(me, None)
            return ActResult()
        if name == "skip":
            if not last_chance or me != self.round.spy_id:
                raise RoomError("Это действие сейчас недоступно")
            self.round.skip_last_chance()
            self._finish_round()
            return ActResult()
        if name == "yes":
            word = ui.get("word")
            if not word:
                raise RoomError("Сначала выберите слово")
            self.guess_ui.pop(me, None)
            self._require_spy(uid)
            correct = self.round.guess(me, word)
            if self.round.phase == Phase.FINISHED:
                self._finish_round()
                return ActResult()
            return ActResult(alert="Верно!" if correct else "Неверно. Попытка использована.")
        raise RoomError("Это действие сейчас недоступно")
    def _finish_round(self) -> None:
        deltas = self.round.settle(self.rules)
        for pid, delta in deltas.items():
            self.scores[pid] = self.scores.get(pid, 0) + delta
        self.last_deltas = deltas
        self.history.append(
            {
                "round": self.round_no,
                "spy": self.round.spy_id,
                "word": self.round.word,
                "outcome": self.round.outcome.value,
                "deltas": deltas,
                "names": {str(p["id"]): p["name"] for p in self.players},
            }
        )
        self.phase = "result"
        self.deadline = None
        self.guess_ui = {}
    def _cancel_round(self) -> None:
        self.round = None
        self.phase = "lobby"
        self.round_no = max(0, self.round_no - 1)
        self.deadline = None
        self.guess_ui = {}
        self.host_ui = "lobby"
        for other in self.players:
            other["ready"] = True
            other["seen"] = False
    def _apply_leave_event(self, event: str, now: float) -> None:
        if self.round.phase == Phase.FINISHED:
            self._finish_round()
            return
        if self.phase == "dealing":
            if all(p["seen"] for p in self.players):
                self._begin_discussion(now)
            return
        if event == "vote":
            self.deadline = None
            self._note("Круг завершён: большинство за голосование. Голосование началось")
        elif self.round.phase == Phase.DISCUSSION:
            self.deadline = now + self.settings["turn_seconds"]
    def _leave(self, uid: int, now: float) -> ActResult:
        player = self._member(uid)
        pid = str(uid)
        name = player["name"]
        active = self.phase in ("dealing", "playing") and self.round is not None
        penalized = active and not self.is_test
        self.guess_ui.pop(pid, None)
        note = f"{name} вышел"
        removed = False
        if active:
            remaining = len(self.players) - 1
            minimum = self.settings.get("min_players", MIN_PLAYERS)
            if pid == self.round.spy_id:
                self.round.spy_left()
                self._finish_round()
                note = f"{name} (шпион) покинул игру, победили игроки"
            elif remaining < minimum:
                self.players.remove(player)
                removed = True
                self._cancel_round()
                note = f"{name} вышел, раунд отменён"
            else:
                self.players.remove(player)
                removed = True
                event = self.round.remove_player(pid)
                self._apply_leave_event(event, now)
                note = f"{name} вышел из игры"
        if not removed:
            self.players.remove(player)
        if not self.players:
            return ActResult(left=True, closed=True, penalized=penalized)
        if uid == self.host_id:
            self.host_id = self.players[0]["id"]
            self._note(f"Новый хост: {self.players[0]['name']}")
        self._note(note)
        return ActResult(left=True, penalized=penalized)
    def link(self, bot_username: str) -> str:
        return f"https://t.me/{bot_username}?start=join_{self.token}"
    def view_for(self, uid: int, bot_username: str = "", now: float | None = None) -> Screen:
        now = time.time() if now is None else now
        self._member(uid)
        if self.phase == "lobby":
            return self._lobby_view(uid, bot_username)
        if self.phase == "dealing":
            return self._dealing_view(uid)
        if self.phase == "playing":
            return self._playing_view(uid, now)
```

```
        if self.phase == "result":
            return self._result_view(uid)
        return Screen(
            f"<b>Игра завершена!</b>\n\nИтоговая таблица:\n{self.table()}",
            [[("В меню", "leave")]],
            html=True,
        )
    def _chat_text(self) -> str:
        lines = []
        for author, text in self.chat[-CHAT_KEEP:]:
            lines.append(f"• {esc(text)}" if author is None else f"<b>{esc(author)}:</b> {esc(text)}")
        return "\n".join(lines)
    def _lobby_view(self, uid: int, bot_username: str) -> Screen:
        host = uid == self.host_id
        if host and self.host_ui == "settings":
            return self._settings_view()
        if host and self.host_ui == "themes":
            return self._themes_view()
        lines = []
        for p in self.players:
            crown = " 👑" if p["id"] == self.host_id else ""
            status = "🟢 готов" if p["ready"] else "◯ ожидает"
            lines.append(f"{status} · {esc(p['name'])}{crown}")
        text = (
            f"<b>🎮 {esc(self.title)}</b>\n\n"
            f"🔑 Код комнаты: <code>{self.code}</code>\n"
        )
        if bot_username:
            text += f'<a href="{esc(self.link(bot_username))}">Ссылка</a>\n'
        text += (
            f"\n<b>👥 Игроки · {len(self.players)}/{self.settings['max_players']}</b>"
            f" · готовы {sum(1 for p in self.players if p['ready'])}/{len(self.players)}\n"
            + "\n".join(lines)
            + (f"\n\n🧪 Тестовый режим · роль хоста: {self._role_label()}" if self.is_test else "")
            + f"\n\n<b>🎯 Темы:</b> {esc(self.themes_label())}{self._order_suffix()}"
            + f"\n<b>⚙ Настройки:</b> {self.settings['rounds']} раундов · {self.settings['turn_seconds']} с на ход"
        )
        text += self._themes_notice()
        if self.round_no:
            text += f"\n\n<b>⏳ Результаты после {self.round_no} раундов</b>\n{self.table()}"
        if self.chat:
            text += f"\n\n{self._chat_text()}"
        rows: list[list[tuple[str, str]]] = []
        if host:
            rows.append([("Начать игру", "start")])
            if self.is_test:
                rows.append([(f"Моя роль: {self._role_label()}", "cy:role")])
            rows.append([("Настройки", "ui:settings")])
        else:
            me = self._p(uid)
            rows.append([("Не готов", "rd:0")] if me["ready"] else [("Готов", "rd:1")])
        if bot_username:
            invitation = (
                "🕵 Приглашаю сыграть в «ШПИОН»!\n"
                f"Комната: {self.title}\n"
                f"Код комнаты: {self.code}\n"
                "Открой ссылку и присоединяйся к игре."
            )
            share_url = "https://t.me/share/url?" + urlencode({
                "url": self.link(bot_username), "text": invitation,
            })
            rows.append([
                ("Пригласить", f"url:{share_url}"),
                ("Скопировать ссылку", f"copy:{self.link(bot_username)}"),
            ])
        rows.append([("Выйти", "leave")])
        return Screen(text, rows, html=True)
    def _option_rows(self, title: str, key: str) -> list[list[tuple[str, str]]]:
        field_name, options = OPTIONS[key]
        current = self.settings[field_name]
        buttons = []
        for value in options:
            if key == "max" and value < len(self.players):
                continue
            label = f"[x] {value}" if value == current else str(value)
            buttons.append((label, f"sv:{key}:{value}"))
        return [[(title, "noop")], buttons]
    def _order_label(self) -> str:
        return "случайный" if self.settings["random_theme"] else "по очереди"
    def _order_suffix(self) -> str:
        if len(self.settings["themes"]) > 1:
            return f" (порядок: {self._order_label()})"
        return ""
    def _settings_view(self) -> Screen:
        rows: list[list[tuple[str, str]]] = []
        if not self.is_test:
            rows += self._option_rows("Игроков в комнате", "max")
        rows += self._option_rows("Раундов в игре", "rounds")
        rows += self._option_rows("Секунд на ход", "turn")
        rows += [
            [("Темы: выбрать", "ui:themes")],
            [(f"Порядок тем: {self._order_label()}", "rt")],
            [("Готово, назад в лобби", "ui:lobby")],
        ]
        return Screen(
            "<b>Настройки комнаты</b>\n\nНажмите на нужное значение, оно сохранится сразу.\n"
            f"Темы сейчас: {esc(self.themes_label())}{self._order_suffix()}" + self._themes_notice(),
            rows,
            html=True,
        )
    def _themes_view(self) -> Screen:
        rows = []
        all_packs = self.library.packs()
        pages = max(1, -(-len(all_packs) // THEMES_PAGE))
        page = min(self.themes_page, pages - 1)
        for pack in all_packs[page * THEMES_PAGE : (page + 1) * THEMES_PAGE]:
            mark = "[x]" if pack.id in self.settings["themes"] else "[ ]"
            rows.append([(f"{mark} {pack.title} ({len(pack.words)})", f"th:{pack.id}")])
        nav = []
        if page > 0:
            nav.append(("Назад", f"tp:{page - 1}"))
        if page < pages - 1:
            nav.append(("Вперёд", f"tp:{page + 1}"))
        if nav:
            rows.append(nav)
        rows.append([("Выбрать все", "tall"), ("Очистить", "tnone")])
        rows.append([(f"Порядок тем: {self._order_label()}", "rt")])
        rows.append([("Готово", "ui:settings")])
        return Screen(
            "<b>Темы игры</b>\n\nОтметьте одну или несколько. "
            "В каждом раунде играет одна тема из отмеченных: по очереди или случайно.",
            rows,
            html=True,
        )
    def _dealing_view(self, uid: int) -> Screen:
        seen = sum(1 for p in self.players if p["seen"])
        lines = "\n".join(
            f"{'[x]' if p['seen'] else '[ ]'} {esc(p['name'])}" for p in self.players
        )
        rows = [[("Показать роль", "role")]]
        if uid == self.host_id:
            rows.append([("Начать обсуждение", "force")])
        rows.append([("Выйти", "leave")])
        return Screen(
            f"<b>Раунд {self.round_no} из {self.settings['rounds']}</b>\n\n"
            "Нажмите «Показать роль»: она появится во всплывающем окне и не останется в чате."
            f"\n\nПосмотрели: {seen} из {len(self.players)}\n{lines}",
            rows,
            html=True,
        )
    def _playing_view(self, uid: int, now: float) -> Screen:
        rnd = self.round
        me = str(uid)
        head = (
            f"<b>Раунд {self.round_no} из {self.settings['rounds']}</b> · "
            f"{esc(rnd.theme)}"
        )
        if rnd.phase == Phase.LAST_CHANCE:
            if me == rnd.spy_id:
                return self._guess_view(head, me, now)
            return Screen(
                f"{head}\n\nШпион определён: <b>{esc(self.name_of(rnd.spy_id))}</b>.\n"
```

```
            "У него последний шанс угадать слово. "
            f"Осталось около {self._left_seconds(now, LAST_CHANCE_SECONDS)} с. Если не успеет, победят игроки.",
            [[("Моя роль", "role")], [("Выйти", "leave")]],
            html=True,
        )
    if me in self.guess_ui:
        return self._guess_view(head, me, now)
    if rnd.phase in (Phase.SUSPECT_VOTE, Phase.RUNOFF_VOTE):
        return self._vote_view(head, me)
    return self._discussion_view(head, me, now)

def _discussion_view(self, head: str, me: str, now: float) -> Screen:
    rnd = self.round
    order = rnd.order
    idx = rnd.turn_idx % len(order)
    current = rnd.current()
    left = self._left_seconds(now, self.settings["turn_seconds"])
    listing = "\n".join(
        f"{'>' if i == idx else '-'} {i + 1}. {esc(self.name_of(p))}"
        for i, p in enumerate(order)
    )
    text = (
        f"{head}\n\n🔁 Круг {rnd.circle_no}\n\n"
        f"🎙 Сейчас говорит: <b>{esc(self.name_of(current))}</b>\n⏳ Осталось около {left} с\n"
        f"Далее: {esc(self.name_of(rnd.next_player()))}\n\n{listing}\n\n"
        "В свой ход каждый решает, нужно ли голосование. "
        "Оно начнётся после круга, если за него больше половины игроков."
    )
    if me == current:
        text += "\n\n<b>Ваш ход!</b> Напишите сообщение в чат бота."
    if self.chat:
        text += f"\n\n{self._chat_text()}"
    rows: list[list[tuple[str, str]]] = []
    if me == current:
        rows.append([("Закончить ход", "endturn:0")])
        rows.append([("Закончить ход и предложить голосование", "endturn:1")])
    elif me == str(self.host_id):
        rows.append([("Пропустить ход игрока", "endturn:0")])
    rows.append([("Угадать слово", "gu"), ("Моя роль", "role")])
    rows.append([("Выйти", "leave")])
    return Screen(text, rows, html=True)

def _vote_view(self, head: str, me: str) -> Screen:
    rnd = self.round
    note = ""
    if rnd.phase == Phase.RUNOFF_VOTE:
        names = ", ".join(esc(self.name_of(p)) for p in rnd.candidates)
        note = f"\nДополнительное голосование между: {names}\n"
    chosen = rnd.votes.get(me)
    rows = []
    for target in rnd.valid_targets(me):
        mark = "[x] " if target == chosen else ""
        rows.append([(f"{mark}{self.name_of(target)}", f"v:{target}")])
    rows.append([("Угадать слово", "gu"), ("Моя роль", "role")])
    rows.append([("Выйти", "leave")])
    status = "Ваш голос принят, можно изменить до конца." if chosen else "Кого вы считаете шпионом?"
    return Screen(
        f"{head}\n\n<b>Тайное голосование</b>{note}\n"
        f"Проголосовали: {len(rnd.votes)} из {len(rnd.players)}\n\n{status}",
        rows,
        html=True,
    )

def _left_seconds(self, now: float, limit: int) -> int:
    remaining = max(0.0, (self.deadline or now) - now)
    return min(limit, math.ceil(remaining / 10) * 10)

def _guess_view(self, head: str, me: str, now: float | None = None) -> Screen:
    ui = self.guess_ui.get(me) or {"page": 0, "word": None}
    words = self._guess_words()
    last_chance = self.round.phase == Phase.LAST_CHANCE
    word = ui.get("word")
    if word:
        return Screen(
            f"{head}\n\nВы выбрали: <b>{esc(word)}</b>\n\nОтправить ответ?"
            + (
                "Ошибка сейчас означает поражение."
                if self.round.phase == Phase.DISCUSSION
                else "Верный ответ сейчас даст ничью."
            ),
            [[("Да", "yes"), ("Другое слово", "no")]],
            html=True,
        )
    pages = max(1, -(-len(words) // PAGE_SIZE))
    page = min(ui.get("page", 0), pages - 1)
    chunk = words[page * PAGE_SIZE : (page + 1) * PAGE_SIZE]
    rows = []
    for i in range(0, len(chunk), 2):
        rows.append(
            [(chunk[j], f"gp:{page * PAGE_SIZE + j}") for j in range(i, min(i + 2, len(chunk)))]
        )
    nav = []
    if page > 0:
        nav.append(("Назад", f"pg:{page - 1}"))
    if page < pages - 1:
        nav.append(("Вперёд", f"pg:{page + 1}"))
    if nav:
        rows.append(nav)
    if last_chance:
        rows.append([("Не угадывать", "skip")])
    else:
        rows.append([("Отмена", "cancel")])
    left = (
        f" Осталось около {self._left_seconds(now, LAST_CHANCE_SECONDS)} с."
        if last_chance and now is not None
        else ""
    )
    if last_chance:
        title = f"Последний шанс! Выберите слово.{left}"
    elif self.round.phase == Phase.DISCUSSION:
        title = (
            "Выберите слово. Попытка одна: верный ответ даёт вам победу, "
            "ошибка означает поражение."
        )
    else:
        title = (
            "Выберите слово. Попытка одна: верный ответ во время голосования даёт ничью, "
            "ошибка сжигает попытку."
        )
    return Screen(f"{head}\n\n{title}\nСтраница {page + 1}/{pages}", rows, html=True)

def _secret_guess_line(self) -> str:
    rnd = self.round
    if rnd is None or not rnd.secret_guess_word:
        return ""
    verdict = "верно" if rnd.secret_guess_correct else "неверно"
    return f"\n\nТайная попытка шпиона: {esc(rnd.secret_guess_word)} ({verdict})"

def _result_view(self, uid: int) -> Screen:
    info = self.history[-1]
    lines = "\n".join(
        f"{esc(p['name'])}: +{info['deltas'].get(str(p['id']), 0)}" for p in self.players
    )
    text = (
        f"<b>Итоги раунда {info['round']}</b>\n\n"
        f"Шпион: <b>{esc(self.name_of(info['spy']))}</b>\n"
        f"Слово: <b>{esc(info['word'])}</b>\n"
        f"{REASONS[Outcome(info['outcome'])]}\n\n"
        f"Очки за раунд:\n{lines}\n\nТаблица:\n{self.table()}"
    )
    text += self._secret_guess_line()
    rows: list[list[tuple[str, str]]] = []
    if uid == self.host_id:
        if self.round_no < self.settings["rounds"]:
            rows.append([("Следующий раунд", "next")])
        if self.is_test:
            rows.append([(f"Моя роль: {self._role_label()}", "cy:role")])
        rows.append([("Изменить настройки", "set")])
        rows.append([("Завершить игру", "end")])
    else:
        text += "\n\nЖдём решения хоста."
    rows.append([("Выйти", "leave")])
    return Screen(text, rows, html=True)

def to_dict(self) -> dict:
    return {
        "code": self.code,
```

```
            "token": self.token,
            "host_id": self.host_id,
            "title": self.title,
            "settings": dict(self.settings),
            "players": [dict(p) for p in self.players],
            "scores": dict(self.scores),
            "phase": self.phase,
            "round_no": self.round_no,
            "round": self.round.to_dict() if self.round else None,
            "deadline": self.deadline,
            "chat": [list(c) for c in self.chat],
            "used_words": list(self.used_words),
            "previous_spy": self.previous_spy,
            "history": list(self.history),
            "last_deltas": dict(self.last_deltas),
            "guess_ui": {k: dict(v) for k, v in self.guess_ui.items()},
            "host_ui": self.host_ui,
            "themes_page": self.themes_page,
            "themes_notice": self.themes_notice,
            "session_id": self.session_id,
        }
    @classmethod
    def from_dict(
        cls, data: dict, library: PackLibrary | None = None, rng: random.Random | None = None
    ) -> Room:
        room = cls(data["code"], data["token"], data["host_id"], data["title"], library, rng)
        for key, value in data.items():
            if key == "round":
                room.round = Round.from_dict(value) if value else None
            elif key not in ("code", "token", "host_id", "title", "turn_idx", "guess_ui"):
                setattr(room, key, value)
        stored = data.get("guess_ui")
        if isinstance(stored, dict) and "page" not in stored:
            room.guess_ui = {str(k): dict(v) for k, v in stored.items()}
        room.prune_missing_themes()
        return room
roundfactory.py
import random
from app.game.engine import Round
from app.game.packs import PackLibrary
from app.game.roles import build_turn_order, choose_spy, pick_word
def choose_theme(
    themes: list[str],
    random_order: bool,
    round_no: int,
    previous_theme: str | None,
    rng: random.Random,
) -> str:
    if len(themes) == 1:
        return themes[0]
    if random_order:
        pool = [t for t in themes if t != previous_theme] or list(themes)
        return rng.choice(pool)
    return themes[(max(round_no, 1) - 1) % len(themes)]
def create_round(
    library: PackLibrary,
    rng: random.Random,
    pids: list[str],
    themes: list[str],
    random_theme: bool,
    used_words: list[str],
    previous_spy: str | None,
    guesses: int = 1,
    spy_override: str | None = None,
    round_no: int = 1,
    previous_theme: str | None = None,
    open_guess: bool = False,
) -> tuple[Round, list[str]]:
    if not themes:
        raise ValueError("Не выбрано ни одной темы")
    theme_id = choose_theme(themes, random_theme, round_no, previous_theme, rng)
    if not library.has(theme_id):
        raise ValueError("Выбранная тема недоступна")
    pack = library.get(theme_id)
    words = list(pack.words)
    if not words:
        raise ValueError("В выбранной теме нет слов")
    word = pick_word(words, used_words, rng)
    spy = spy_override if spy_override in pids else choose_spy(pids, previous_spy, rng)
    order = build_turn_order(pids, spy, rng)
    rnd = Round(
        players=list(pids),
        spy_id=spy,
        word=word,
        theme=pack.title,
        allowed_words=words,
        order=order,
        guesses_left=guesses,
        theme_id=theme_id,
        open_guess=open_guess,
    )
    exhausted = set(words) <= set(used_words) | {word}
    used = [w for w in used_words if w not in set(words)] if exhausted else list(used_words)
    used.append(word)
    return rnd, used
scoring.py
from dataclasses import dataclass
from enum import Enum
class Outcome(str, Enum):
    CIVILIANS_WIN = "civilians_win"
    SPY_WIN_VOTE = "spy_win_vote"
    SPY_WIN_GUESS = "spy_win_guess"
    SPY_WRONG_GUESS = "spy_wrong_guess"
    SPY_LEFT = "spy_left"
    DRAW = "draw"
@dataclass(frozen=True)
class ScoreRules:
    spy_win: int = 3
    civilians_win: int = 1
    draw: int = 0
def compute_scores(
    outcome: Outcome, spy_id: str, players: list[str], rules: ScoreRules
) -> dict[str, int]:
    deltas = {player: 0 for player in players}
    if outcome in (Outcome.CIVILIANS_WIN, Outcome.SPY_WRONG_GUESS, Outcome.SPY_LEFT):
        for player in players:
            if player != spy_id:
                deltas[player] = rules.civilians_win
    elif outcome in (Outcome.SPY_WIN_VOTE, Outcome.SPY_WIN_GUESS):
        deltas[spy_id] = rules.spy_win
    else:
        for player in players:
            deltas[player] = rules.draw
    return deltas
session.py
from __future__ import annotations
import random
from dataclasses import dataclass, field
from enum import Enum
from app.game.engine import GameError, Phase, Round
from app.game.packs import PackLibrary, load_default_library
from app.game.roundfactory import create_round
from app.game.scoring import Outcome, ScoreRules
MIN_PLAYERS = 3
MAX_PLAYERS = 12
MAX_NAME_LEN = 20
PAGE_SIZE = 8
THEMES_PAGE = 8
REASONS = {
    Outcome.CIVILIANS_WIN: "Игроки нашли шпиона, и он не угадал слово.",
    Outcome.SPY_WRONG_GUESS: "Шпион ошибся при угадывании слова и проиграл.",
    Outcome.SPY_LEFT: "Шпион покинул игру. Победа игроков.",
    Outcome.SPY_WIN_VOTE: "Игроки ошиблись или не смогли определить шпиона.",
    Outcome.SPY_WIN_GUESS: "Шпион угадал секретное слово.",
    Outcome.DRAW: "Шпион угадал слово во время голосования. Ничья.",
}
class Stage(str, Enum):
    COUNT = "count"
    NAMES = "names"
    REVIEW = "review"
    THEMES = "themes"
    DEAL_HANDOFF = "deal_handoff"
    DEAL_REVEAL = "deal_reveal"
    DEAL_DONE = "deal_done"
    DISCUSSION = "discussion"
    VOTE_REQUEST = "vote_request"
    VOTE_HANDOFF = "vote_handoff"
    VOTE_PICK = "vote_pick"
    GUESS_WHO = "guess_who"
    GUESS_HANDOFF = "guess_handoff"
    GUESS_PICK = "guess_pick"
    GUESS_CONFIRM = "guess_confirm"
    RESULT = "result"
    FINAL = "final"
class Reject(Exception):
    pass
@dataclass
class Screen:
    text: str
    rows: list[list[tuple[str, str]]]
    html: bool = False
@dataclass
class Response:
    screen: Screen | None = None
    alert: str | None = None
    exit: bool = False
def validate_name(raw: str, existing: list[str], skip: int | None = None) -> str:
    name = " ".join(raw.split())
    if not name:
        raise ValueError("Имя не может быть пустым")
    if len(name) > MAX_NAME_LEN:
        raise ValueError(f"Имя длиннее {MAX_NAME_LEN} символов")
    taken = [n.casefold() for i, n in enumerate(existing) if i != skip]
```

```
    if name.casefold() in taken:
        raise ValueError("Такое имя уже есть")
    return name
@dataclass
class LocalSession:
    library: PackLibrary = field(default_factory=load_default_library, repr=False)
    rng: random.Random = field(default_factory=random.Random, repr=False)
    rules: ScoreRules = field(default_factory=ScoreRules, repr=False)
    stage: Stage = Stage.COUNT
    seq: int = 0
    count: int = 0
    names: list[str] = field(default_factory=list)
    edit_idx: int | None = None
    themes: list[str] = field(default_factory=list)
    random_theme: bool = False
    scores: dict[str, int] = field(default_factory=dict)
    round_no: int = 0
    used_words: list[str] = field(default_factory=list)
    previous_spy: str | None = None
    round: Round | None = None
    deal_idx: int = 0
    revealed: bool = False
    vote_idx: int = 0
    turn_idx: int = 0
    theme_page: int = 0
    guess_actor: str | None = None
    guess_return: Stage = Stage.DISCUSSION
    pending_alert: str | None = field(default=None, repr=False)
    guess_page: int = 0
    guess_word: str | None = None
    history: list[dict] = field(default_factory=list)
    last_deltas: dict[str, int] = field(default_factory=dict)
    message_id: int | None = None
    notice: str = ""
    @property
    def pids(self) -> list[str]:
        return [f"p{i}" for i in range(len(self.names))]
    def name_of(self, pid: str) -> str:
        return self.names[int(pid[1:])]
    def table(self) -> str:
        ranked = sorted(self.pids, key=lambda p: (-self.scores.get(p, 0), int(p[1:])))
        return "\n".join(
            f"{i}. {self.name_of(p)} — {self.scores.get(p, 0)}"
            for i, p in enumerate(ranked, 1)
        )
    def _themes_label(self) -> str:
        if not self.themes:
            return "не выбраны"
        return ", ".join(self.library.get(t).title for t in self.themes)
    def _guess_words(self) -> list[str]:
        assert self.round is not None
        return sorted(self.round.allowed_words, key=str.casefold)
    def _spy(self) -> str:
        assert self.round is not None
        return self.round.spy_id
    def screen(self) -> Screen:
        notice = f"{self.notice}\n\n" if self.notice else ""
        screen = self._build_screen()
        return Screen(notice + screen.text, screen.rows)
    def _build_screen(self) -> Screen:
        stage = self.stage
        if stage == Stage.COUNT:
            numbers = [(str(n), f"n:{n}") for n in range(MIN_PLAYERS, MAX_PLAYERS + 1)]
            rows = [numbers[i : i + 5] for i in range(0, len(numbers), 5)]
            if self.names:
                rows.append([("Назад", "rv")])
            rows.append([("В меню", "menu")])
            return Screen(
                f"Игра на одном устройстве\n\nСколько игроков? ({MIN_PLAYERS}–{MAX_PLAYERS})",
                rows,
            )
        if stage == Stage.NAMES:
            return Screen(
                f"Игрок {len(self.names) + 1} из {self.count}\n\nНапишите имя сообщением в чат.",
                [[("Количество игроков", "cnt")], [("В меню", "menu")]],
            )
        if stage == Stage.REVIEW:
            if self.edit_idx is not None:
                return Screen(
                    f"Новое имя для игрока {self.edit_idx + 1} "
                    f"(сейчас: {self.names[self.edit_idx]}).\n\nНапишите имя сообщением.",
                    [[("Отмена", "ce")]],
                )
            lines = "\n".join(f"{i}. {n}" for i, n in enumerate(self.names, 1))
            head = "Настройка игры"
            if self.round_no:
                head += f" (сыграно раундов: {self.round_no}, очки сохранены)"
            edit = [(f"{i}", f"e:{i - 1}") for i in range(1, len(self.names) + 1)]
            rows = [edit[i : i + 6] for i in range(0, len(edit), 6)]
            rows += [
                [("Темы", "th"), ("Количество", "cnt")],
                [("Начать игру", "go")],
                [("В меню", "menu")],
            ]
            return Screen(
                f"{head}\n\nИгроки:\n{lines}\n\nТемы: {self._themes_label()}"
                f"\nПорядок тем: {'случайный' if self.random_theme else 'по очереди'}"
                "\n\n— исправить имя игрока по номеру.",
                rows,
            )
        if stage == Stage.THEMES:
            rows = []
            all_packs = self.library.packs()
            pages = max(1, -(-len(all_packs) // THEMES_PAGE))
            page = min(self.theme_page, pages - 1)
            for pack in all_packs[page * THEMES_PAGE : (page + 1) * THEMES_PAGE]:
                mark = "[x]" if pack.id in self.themes else "[ ]"
                rows.append([(f"{mark} {pack.title} ({len(pack.words)})", f"t:{pack.id}")])
            nav = []
            if page > 0:
                nav.append(("Назад", f"tp:{page - 1}"))
            if page < pages - 1:
                nav.append(("Вперёд", f"tp:{page + 1}"))
            if nav:
                rows.append(nav)
            order = "случайный" if self.random_theme else "по очереди"
            rows.append([("Выбрать все", "tall"), ("Очистить", "tnone")])
            rows.append([(f"Порядок тем: {order}", "rt")])
            rows.append([("Готово", "ok")])
            return Screen(
                "Выберите одну или несколько тем.\n\n"
                "В каждом раунде играет одна тема из отмеченных: по очереди или в случайном порядке.",
                rows,
            )
        if stage == Stage.DEAL_HANDOFF:
            name = self.names[self.deal_idx]
            return Screen(
                f"Раздача ролей · {self.deal_idx + 1} из {len(self.names)}\n\n"
                f"Передайте телефон игроку {name}.\n\nОстальные, не смотрите на экран.",
                [[("Показать роль", "me")]],
            )
        if stage == Stage.DEAL_REVEAL:
            name = self.names[self.deal_idx]
            return Screen(
                f"{name}, это ваш экран.\n\n"
                "Нажмите «Показать»: роль появится во всплывающем окне и не останется в чате."
                "\n\nЗапомните её и нажмите «Дальше».",
                [[("Показать", "show")], [("Дальше", "next")]],
            )
        if stage == Stage.DEAL_DONE and self.round is not None:
            return Screen(
                "Все игроки получили свои роли.\n\n"
                f"Тема раунда: {self.round.theme}\n\n"
                "Игра начинается!",
                [[("Начать раунд", "start")]],
            )
        if stage == Stage.DISCUSSION:
            order = self.round.order
            idx = self.round.turn_idx % len(order)
            listing = "\n".join(
                f"{'>' if i == idx else '-'} {i + 1}. {self.name_of(p)}"
                for i, p in enumerate(order)
            )
            return Screen(
                f"Раунд {self.round_no}, круг {self.round.circle_no}. Тема: {self.round.theme}\n\n"
                f"Говорит: {self.name_of(self.round.current())}\n"
```

```
            f"Далее: {self.name_of(self.round.next_player())}\n\n"
            f"Порядок:\n{listing}\n\n"
            "В свой ход каждый решает, нужно ли голосование. Оно начнётся после круга, "
            "если за него больше половины игроков.",
            [
                [("Передать ход", "nx:0")],
                [("Передать ход и предложить голосование", "nx:1")],
                [("Угадать слово", "gu")],
            ],
        )
    if stage == Stage.VOTE_HANDOFF:
        name = self.name_of(self.round.players[self.vote_idx])
        extra = ""
        if self.round.phase == Phase.RUNOFF_VOTE:
            names = ", ".join(self.name_of(p) for p in self.round.candidates)
            extra = f"\n\nДополнительное голосование между: {names}."
        return Screen(
            f"Тайное голосование · {self.vote_idx + 1} из {len(self.round.players)}.{extra}"
            f"\n\nПередайте телефон игроку {name}.",
            [[("Голосовать", "me")], [("Угадать слово", "gu")]],
        )
    if stage == Stage.VOTE_PICK:
        voter = self.round.players[self.vote_idx]
        rows = [
            [(self.name_of(t), f"v:{t}")] for t in self.round.valid_targets(voter)
        ]
        return Screen(f"{self.name_of(voter)}, кого вы считаете шпионом?", rows)
    if stage == Stage.GUESS_WHO:
        rows = [[(self.name_of(p), f"gw:{p}")] for p in self.round.players]
        rows.append([("Назад", "back")])
        return Screen(
            "Попытка угадать слово.\n\n"
            "Возьмите телефон и выберите своё имя. Остальные, отвернитесь.",
            rows,
        )
    if stage == Stage.GUESS_HANDOFF:
        name = self.name_of(self._spy())
        return Screen(
            f"Шпион определён. Игрок {name}, у вас последний шанс угадать слово."
            f"\n\nПередайте телефон игроку {name}.",
            [[("Угадать слово", "me")]],
        )
    if stage == Stage.GUESS_PICK:
        words = self._guess_words()
        pages = max(1, -(-len(words) // PAGE_SIZE))
        page = min(self.guess_page, pages - 1)
        chunk = words[page * PAGE_SIZE : (page + 1) * PAGE_SIZE]
        rows = []
        for i in range(0, len(chunk), 2):
            rows.append(
                [
                    (chunk[j], f"gp:{page * PAGE_SIZE + j}")
                    for j in range(i, min(i + 2, len(chunk)))
                ]
            )
        nav = []
        if page > 0:
            nav.append(("Назад", f"pg:{page - 1}"))
        if page < pages - 1:
            nav.append(("Вперёд", f"pg:{page + 1}"))
        if nav:
            rows.append(nav)
        if self.round.phase == Phase.LAST_CHANCE:
            rows.append([("Не угадывать", "skip")])
        else:
            rows.append([("Отмена", "cancel")])
        return Screen(
            f"Выберите слово. Тема: {self.round.theme}. Страница {page + 1}/{pages}",
            rows,
        )
    if stage == Stage.GUESS_CONFIRM:
        return Screen(
            f"Вы выбрали: {self.guess_word}\n\nОтправить ответ? Попытка одна. "
            "Результат станет известен в конце раунда.",
            [[("Да", "yes"), ("Другое слово", "no")]],
        )
    if stage == Stage.RESULT:
        return self._result_screen()
    return Screen(
        f"Игра завершена!\n\nИтоговая таблица:\n{self.table()}",
        [[("В меню", "menu")]],
    )

def _secret_guess_line(self) -> str:
    rnd = self.round
    if rnd is None or not rnd.secret_guess_word:
        return ""
    verdict = "верно" if rnd.secret_guess_correct else "неверно"
    return f"\n\nТайная попытка шпиона: {rnd.secret_guess_word} ({verdict})"

def _result_screen(self) -> Screen:
    info = self.history[-1]
    lines = []
    for pid in self.pids:
        delta = info["deltas"].get(pid, 0)
        lines.append(f"{self.name_of(pid)}: +{delta}")
    return Screen(
        f"Итоги раунда {info['round']}\n\n"
        f"Шпион: {self.name_of(info['spy'])}\n"
        f"Слово: {info['word']}\n\n"
        f"{REASONS[Outcome(info['outcome'])]}\n\n"
        f"Очки за раунд:\n" + "\n".join(lines) + f"\n\nТаблица:\n{self.table()}"
        + self._secret_guess_line(),
        [
            [("Следующий раунд", "nr")],
            [("Изменить настройки", "set")],
            [("Завершить игру", "end")],
            [("В меню", "menu")],
        ],
    )

def act(self, seq: int, action: str) -> Response:
    if seq != self.seq:
        return Response(screen=self.screen(), alert="Эта кнопка устарела")
    if action == "show":
        try:
            return Response(alert=self._role_text())
        except Reject as err:
            return Response(screen=self.screen(), alert=str(err))
    if action == "menu" and self.stage in (
        Stage.COUNT, Stage.NAMES, Stage.REVIEW, Stage.RESULT, Stage.FINAL
    ):
        return Response(exit=True)
    self.notice = ""
    try:
        self._dispatch(action)
    except (Reject, GameError) as err:
        return Response(screen=self.screen(), alert=str(err))
    alert = self.pending_alert
    self.pending_alert = None
    self.seq += 1
    return Response(screen=self.screen(), alert=alert)

def _role_text(self) -> str:
    if self.stage != Stage.DEAL_REVEAL or self.round is None:
        raise Reject("Роль сейчас недоступна")
    self.revealed = True
    pid = self.round.players[self.deal_idx]
    name = self.name_of(pid)
    if pid == self.round.spy_id:
        return f"{name}, вы шпион! Определите загаданное слово."
    return f"{name}, ваше загаданное слово:\n\n{self.round.word.upper()}"

def _dispatch(self, action: str) -> None:
    stage = self.stage
    name, _, arg = action.partition(":")
    if stage == Stage.COUNT:
        if name == "n" and arg.isdigit() and MIN_PLAYERS <= int(arg) <= MAX_PLAYERS:
            self.count = int(arg)
            self.names = self.names[: self.count]
            self.stage = Stage.REVIEW if len(self.names) == self.count else Stage.NAMES
            return
        if name == "rv" and self.names:
            self.stage = Stage.REVIEW
            return
    elif stage == Stage.NAMES:
        if name == "cnt":
            self.stage = Stage.COUNT
            return
    elif stage == Stage.REVIEW:
        if name == "e" and arg.isdigit() and int(arg) < len(self.names):
            self.edit_idx = int(arg)
            return
        if name == "ce":
            self.edit_idx = None
            return
        if self.edit_idx is not None:
```

```
            raise Reject("Сначала введите имя или нажмите «Отмена»")
        if name == "th":
            self.stage = Stage.THEMES
            return
        if name == "cnt":
            self.stage = Stage.COUNT
            return
        if name == "go":
            self._start_game()
            return
    elif stage == Stage.THEMES:
        if name == "t" and self.library.has(arg):
            if arg in self.themes:
                self.themes.remove(arg)
            else:
                self.themes.append(arg)
            return
        if name == "tp" and arg.isdigit():
            self.theme_page = int(arg)
            return
        if name == "tall":
            self.themes = [pack.id for pack in self.library.packs()]
            return
        if name == "tnone":
            self.themes = []
            return
        if name == "rt":
            self.random_theme = not self.random_theme
            return
        if name == "ok":
            self.stage = Stage.REVIEW
            return
    elif stage == Stage.DEAL_HANDOFF:
        if name == "me":
            self.revealed = False
            self.stage = Stage.DEAL_REVEAL
            return
    elif stage == Stage.DEAL_REVEAL:
        if name == "next":
            if not self.revealed:
                raise Reject("Сначала посмотрите свою роль")
            self.revealed = False
            self.deal_idx += 1
            if self.deal_idx >= len(self.names):
                self.stage = Stage.DEAL_DONE
            else:
                self.stage = Stage.DEAL_HANDOFF
            return
    elif stage == Stage.DEAL_DONE:
        if name == "start":
            self.round.finish_deal()
            self.turn_idx = 0
            self.stage = Stage.DISCUSSION
            return
    elif stage == Stage.DISCUSSION:
        if name == "nx":
            event = self.round.end_turn(self.round.current(), arg == "1")
            self._after_turn(event)
            return
        if name == "gu":
            self.guess_return = Stage.DISCUSSION
            self.stage = Stage.GUESS_WHO
            return
    elif stage == Stage.VOTE_HANDOFF:
        if name == "me":
            self.stage = Stage.VOTE_PICK
            return
        if name == "gu":
            self.guess_return = Stage.VOTE_HANDOFF
            self.stage = Stage.GUESS_WHO
            return
    elif stage == Stage.VOTE_PICK:
        if name == "v":
            voter = self.round.players[self.vote_idx]
            phase_before = self.round.phase
            self.round.cast_vote(voter, arg)
            self.vote_idx += 1
            self._after_vote(phase_before)
            return
    elif stage == Stage.GUESS_WHO:
        if name == "back":
            self.stage = self.guess_return
            return
        if name == "gw" and arg in self.round.players:
            self.guess_actor = arg
            self.guess_page = 0
            self.stage = Stage.GUESS_PICK
            return
    elif stage == Stage.GUESS_HANDOFF:
        if name == "me":
            self.guess_actor = self._spy()
            self.guess_page = 0
            self.stage = Stage.GUESS_PICK
            return
    elif stage == Stage.GUESS_PICK:
        if name == "pg" and arg.isdigit():
            self.guess_page = int(arg)
            return
        if name == "gp" and arg.isdigit() and int(arg) < len(self._guess_words()):
            self.guess_word = self._guess_words()[int(arg)]
            self.stage = Stage.GUESS_CONFIRM
            return
        if name == "cancel" and self.round.phase != Phase.LAST_CHANCE:
            self.stage = self.guess_return
            return
        if name == "skip" and self.round.phase == Phase.LAST_CHANCE:
            self.round.skip_last_chance()
            self._finish_round()
            return
    elif stage == Stage.GUESS_CONFIRM:
        if name == "no":
            self.stage = Stage.GUESS_PICK
            return
        if name == "yes":
            word = self.guess_word or ""
            actor = self.guess_actor or ""
            self.guess_word = None
            if self.round.phase == Phase.LAST_CHANCE:
                self.round.guess(self._spy(), word)
                self._finish_round()
                return
            self.round.secret_guess(actor, word)
            self.pending_alert = "Ответ принят."
            self.stage = self.guess_return
            return
    elif stage == Stage.RESULT:
        if name == "nr":
            self._start_round()
            return
        if name == "set":
            self.stage = Stage.REVIEW
            return
        if name == "end":
            self.stage = Stage.FINAL
            return
    raise Reject("Это действие сейчас недоступно")
def _after_turn(self, event: str) -> None:
    if self.round.phase == Phase.FINISHED:
        self._finish_round()
    elif event == "vote":
        self.vote_idx = 0
        self.stage = Stage.VOTE_HANDOFF
        self.notice = "Круг завершён: большинство за голосование."
    elif event == "new_circle":
        self.notice = (
            "Круг завершён: большинство против голосования. "
            f"Начинается круг {self.round.circle_no}."
        )
def _after_vote(self, phase_before: Phase) -> None:
    phase = self.round.phase
    if phase == Phase.FINISHED:
        self._finish_round()
    elif phase == Phase.LAST_CHANCE:
        self.stage = Stage.GUESS_HANDOFF
    elif phase == Phase.RUNOFF_VOTE and phase_before != Phase.RUNOFF_VOTE:
        self.vote_idx = 0
        self.stage = Stage.VOTE_HANDOFF
    else:
        self.stage = Stage.VOTE_HANDOFF
def handle_text(self, text: str) -> Response | None:
    if self.stage == Stage.NAMES:
        try:
            self.names.append(validate_name(text, self.names))
        except ValueError as err:
            self.notice = str(err)
            return Response(screen=self.screen())
        self.notice = ""
        if len(self.names) >= self.count:
            self.stage = Stage.REVIEW
        self.seq += 1
        return Response(screen=self.screen())
    if self.stage == Stage.REVIEW and self.edit_idx is not None:
        try:
            self.names[self.edit_idx] = validate_name(text, self.names, self.edit_idx)
        except ValueError as err:
            self.notice = str(err)
            return Response(screen=self.screen())
        self.notice = ""
        self.edit_idx = None
        self.seq += 1
        return Response(screen=self.screen())
    return None
```

```
    def _start_game(self) -> None:
        if not MIN_PLAYERS <=
len(self.names) <= MAX_PLAYERS:
            raise Reject("Некорректное число
игроков")
        if not self.themes:
            raise Reject("Выберите хотя бы
одну тему")
        self.scores = {p: self.scores.get(p, 0)
for p in self.pids}
        self._start_round()
    def _start_round(self) -> None:
        pids = self.pids
        try:
            self.round, self.used_words =
create_round(
                self.library,
                self.rng,
                pids,
                self.themes,
                self.random_theme,
                self.used_words,
                self.previous_spy,
                round_no=self.round_no + 1,

previous_theme=self.round.theme_id if
self.round else None,
            )
        except ValueError as err:
            raise Reject(str(err)) from err
        self.previous_spy = self.round.spy_id
        self.round_no += 1
        self.deal_idx = 0
        self.revealed = False
        self.vote_idx = 0
        self.guess_actor = None
        self.guess_page = 0
        self.guess_word = None
        self.stage = Stage.DEAL_HANDOFF
    def _finish_round(self) -> None:
        deltas = self.round.settle(self.rules)
        for pid, delta in deltas.items():
            self.scores[pid] =
self.scores.get(pid, 0) + delta
        self.last_deltas = deltas
        self.history.append(
            {
                "round": self.round_no,
                "spy": self.round.spy_id,
                "word": self.round.word,
                "outcome":
self.round.outcome.value,
                "deltas": deltas,
            }
        )
        self.stage = Stage.RESULT
    def neutralize(self) -> None:
        if self.stage ==
Stage.DEAL_REVEAL:
            self.stage =
Stage.DEAL_HANDOFF
        elif self.stage == Stage.VOTE_PICK:
            self.stage =
Stage.VOTE_HANDOFF
        elif self.stage in (Stage.GUESS_PICK,
Stage.GUESS_CONFIRM,
Stage.GUESS_WHO):
            if self.round and self.round.phase ==
Phase.LAST_CHANCE:
                self.stage =
Stage.GUESS_HANDOFF
            else:
                self.stage = self.guess_return
        self.revealed = False
        self.guess_word = None
        self.edit_idx = None
        self.seq += 1
    def to_dict(self) -> dict:
        return {
            "stage": self.stage.value,
            "seq": self.seq,
            "count": self.count,
            "names": list(self.names),
            "edit_idx": self.edit_idx,
            "themes": list(self.themes),
            "random_theme":
self.random_theme,
            "scores": dict(self.scores),
            "round_no": self.round_no,
            "used_words":
list(self.used_words),
            "previous_spy": self.previous_spy,
            "round": self.round.to_dict() if
self.round else None,
            "deal_idx": self.deal_idx,
            "revealed": self.revealed,
            "vote_idx": self.vote_idx,
            "guess_actor": self.guess_actor,
            "guess_return":
self.guess_return.value,
            "theme_page": self.theme_page,
            "guess_page": self.guess_page,
            "guess_word": self.guess_word,
            "history": list(self.history),
            "last_deltas": dict(self.last_deltas),
            "message_id": self.message_id,
        }
    @classmethod
    def from_dict(
        cls,
        data: dict,
        library: PackLibrary | None = None,
        rng: random.Random | None = None,
    ) -> LocalSession:
        session = cls(
            library=library or
load_default_library(), rng=rng or
random.Random()
        )
        for key, value in data.items():
            if key == "stage":
                session.stage =
Stage.DISCUSSION if value ==
"vote_request" else Stage(value)
            elif key == "turn_idx":
                continue
            elif key == "guess_return":
                session.guess_return =
Stage(value)
            elif key == "round":
                session.round =
Round.from_dict(value) if value else None
            else:
                setattr(session, key, value)
        return session
voting.py
from collections import Counter
def majority_reached(yes_votes: int,
active_players: int) -> bool:
    return yes_votes * 2 > active_players
def votes_needed(active_players: int) -> int:
    return active_players // 2 + 1
def tally(votes: dict[str, str]) ->
tuple[dict[str, int], list[str]]:
    if not votes:
        return {}, []
    counts = Counter(votes.values())
    top = max(counts.values())
    leaders = sorted(player for player,
number in counts.items() if number == top)
    return dict(counts), leaders
health.py
"""Health probe usable as ``python -m
app.health`` without bot credentials."""
import asyncio
import json
import logging
import os
import time
from pathlib import Path
from
aiogram.client.session.middlewares.base
import BaseRequestMiddleware
from aiogram.methods import GetUpdates
from sqlalchemy import text
from app.database.worker import run_db
log = logging.getLogger(__name__)
DEFAULT_PATH =
Path(__file__).resolve().parents[1] / '.bot-
health.json'
def health_path():
    return
Path(os.environ.get('BOT_HEALTH_FILE
', str(DEFAULT_PATH)))
def check_health(path=None, *,
now=None, max_age=30):
    try:
        status = json.loads((path or
health_path()).read_text(encoding='utf-8'))
        age = (time.time() if now is None else
now) - status['updated_at']
        if not status['healthy']:
            return False, status.get('reason',
'unhealthy')
        if not 0 <= age <= max_age:
            return False, 'heartbeat expired'
        return True, 'healthy'
    except (OSError, ValueError, KeyError,
TypeError):
        return False, 'heartbeat unavailable'
class
PollingHealthMiddleware(BaseRequestMi
ddleware):
    def __init__(self):
        self.last_poll = time.monotonic()
    async def __call__(self, make_request,
bot, method):
        result = await make_request(bot,
method)
        if isinstance(method, GetUpdates):
            self.last_poll = time.monotonic()
        return result
def probe_database(factory):
    with factory() as db:
        db.execute(text('SELECT
1')).scalar_one()
def write_status(path, healthy, reason):
    temporary = path.with_suffix('.tmp')
    temporary.write_text(json.dumps({
        'pid': os.getpid(), 'updated_at':
time.time(),
        'healthy': healthy, 'reason': reason,
    }), encoding='utf-8')
    temporary.replace(path)
async def monitor_health(factory, runtime,
ticker, polling, path=None, *, interval=5):
    path = path or health_path()
    while True:
        reason = ''
        if ticker.done():
            reason = 'timer task stopped'
        elif time.monotonic() -
runtime.last_cycle > 120:
            reason = 'timer task stalled'
        elif time.monotonic() -
polling.last_poll > 120:
            reason = 'Telegram polling stalled'
        if not reason:
            try:
                await
asyncio.wait_for(run_db(probe_database,
factory), timeout=10)
            except Exception:
```

```
            reason = 'database probe failed or timed out'
            log.warning('Bot health: %s', reason)
        await asyncio.to_thread(write_status, path, not reason, reason or 'ok')
        await asyncio.sleep(interval)
if __name__ == '__main__':
    healthy, message = check_health()
    print(message)
    raise SystemExit(0 if healthy else 1)
__init__.py
admin_service.py
import re
import time
from dataclasses import dataclass
from sqlalchemy import func, or_, select
from sqlalchemy.orm import sessionmaker
from app.database.models import (
    PackReport,
    PackWord,
    QuestionPack,
    RoomMember,
    RoundPlayer,
    SessionPlayer,
    UserPenalty,
)
from app.game.packs import PackLibrary
from app.services.pack_service import PackError, PackService
from app.services.penalty import PERMANENT_UNTIL, describe_ban
MAX_BAN_MINUTES = 3650 * 24 * 60
PERMANENT_WORDS = {"навсегда", "forever", "perm", "permanent"}
UNITS = {
    "": 1, "м": 1, "мин": 1, "m": 1, "min": 1,
    "ч": 60, "час": 60, "h": 60,
    "д": 1440, "дн": 1440, "день": 1440, "d": 1440,
}
class AdminError(Exception):
    pass
def parse_duration(text: str) -> int | None:
    value = (text or "").strip().lower()
    if value in PERMANENT_WORDS:
        return None
    match = re.fullmatch(r"(\d+)\s*([a-zа-я]*)", value)
    if not match or match.group(2) not in UNITS:
        raise ValueError("Срок задаётся так: 30 (минут), 2ч, 7д или навсегда")
    minutes = int(match.group(1)) * UNITS[match.group(2)]
    if minutes <= 0:
        raise ValueError("Срок должен быть больше нуля")
    if minutes > MAX_BAN_MINUTES:
        raise ValueError("Слишком большой срок, используйте «навсегда»")
    return minutes
@dataclass
class BanRow:
    user_id: int
    name: str
    strikes: int
    seconds_left: float
    reason: str
    @property
    def left_text(self) -> str:
        return describe_ban(self.seconds_left)
    @property
    def reason_text(self) -> str:
        return "администратор" if self.reason == "admin" else "выход из игры"
@dataclass
class AdminPack:
    id: int
    slug: str
    title: str
    owner_id: int | None
    owner_name: str | None
    is_public: bool
    is_hidden: bool
    words: int
    reports: int
    def line(self) -> str:
        access = "скрыт" if self.is_hidden else ("публичный" if self.is_public else "личный")
        owner = self.owner_name or self.owner_id
        return f"{self.id}: {self.title} (автор {owner}, {access}, слов {self.words}, жалоб {self.reports})"
class AdminService:
    def __init__(self, factory: sessionmaker, clock=time.time):
        self._factory = factory
        self._clock = clock
    def _name_of(self, db, user_id: int) -> str:
        row = db.execute(
            select(SessionPlayer.name)
            .where(SessionPlayer.user_id == user_id)

            .order_by(SessionPlayer.session_id.desc())
            .limit(1)
        ).first()
        return row[0] if row else "неизвестно"
    def ban(self, user_id: int, minutes: int | None) -> float:
        if user_id <= 0:
            raise AdminError("Некорректный Telegram ID")
        until = PERMANENT_UNTIL if minutes is None else self._clock() + minutes * 60
        with self._factory.begin() as db:
            row = db.get(UserPenalty, user_id)
            if row is None:
                row = UserPenalty(user_id=user_id, strikes=0, last_leave_at=0.0)
                db.add(row)
            row.banned_until = until
            row.reason = "admin"
        return until
    def unban(self, user_id: int, reset_strikes: bool = False) -> bool:
        with self._factory.begin() as db:
            row = db.get(UserPenalty, user_id)
            if row is None:
                return False
            had = row.banned_until > self._clock() or (reset_strikes and row.strikes > 0)
            row.banned_until = 0.0
            row.reason = "leave"
            if reset_strikes:
                row.strikes = 0
                row.last_leave_at = 0.0
            return had
    def list_bans(self) -> list[BanRow]:
        now = self._clock()
        with self._factory() as db:
            rows = db.execute(
                select(UserPenalty)
                .where(UserPenalty.banned_until > now)

                .order_by(UserPenalty.banned_until.desc())
            ).scalars()
            return [
                BanRow(
                    user_id=r.user_id,
                    name=self._name_of(db, r.user_id),
                    strikes=r.strikes,
                    seconds_left=r.banned_until - now,
                    reason=r.reason or "leave",
                )
                for r in rows
            ]
    def user_info(self, user_id: int) -> dict:
        now = self._clock()
        with self._factory() as db:
            row = db.get(UserPenalty, user_id)
            rounds = db.execute(

                select(func.count()).select_from(RoundPlayer).where(RoundPlayer.user_id == user_id)
            ).scalar_one()
            packs = db.execute(

                select(func.count()).select_from(QuestionPack).where(QuestionPack.owner_id == user_id)
            ).scalar_one()
            in_room = db.get(RoomMember, user_id) is not None
            left = max(0.0, (row.banned_until - now)) if row else 0.0
            return {
                "user_id": user_id,
                "name": self._name_of(db, user_id),
                "strikes": row.strikes if row else 0,
                "ban_seconds": left,
                "ban_reason": (row.reason if row else "leave") or "leave",
                "rounds": rounds,
                "packs": packs,
                "in_room": in_room,
            }
    def _pack_row(self, db, pack: QuestionPack) -> AdminPack:
        words = db.execute(

            select(func.count()).select_from(PackWord).where(PackWord.pack_id == pack.id)
        ).scalar_one()
        reports = db.execute(

            select(func.count()).select_from(PackReport).where(PackReport.pack_id == pack.id)
        ).scalar_one()
        return AdminPack(
            id=pack.id,
            slug=pack.slug,
            title=pack.title,
            owner_id=pack.owner_id,
            owner_name=pack.owner_name,
            is_public=bool(pack.is_public),
            is_hidden=bool(pack.is_hidden),
            words=words,
            reports=reports,
        )
    def list_packs(
        self, query: str | None = None, limit: int = 20, offset: int = 0
    ) -> tuple[list[AdminPack], int]:
        conds = [QuestionPack.owner_id.is_not(None)]
        term = " ".join((query or "").split())
        if term:
            options = [
                QuestionPack.title.contains(term, autoescape=True),
```

```
QuestionPack.owner_name.contains(term, autoescape=True),
                QuestionPack.slug == term,
            ]
            if term.isdigit():
                options += [QuestionPack.id == int(term), QuestionPack.owner_id == int(term)]
            conds.append(or_(*options))
        with self._factory() as db:
            total = db.execute(

select(func.count()).select_from(QuestionPack).where(*conds)
            ).scalar_one()
            packs = (
                db.execute(
                    select(QuestionPack)
                    .where(*conds)

.order_by(QuestionPack.id.desc())
                    .limit(limit)
                    .offset(offset)
                )
                .scalars()
                .all()
            )
            return [self._pack_row(db, p) for p in packs], total
    def get_pack(self, pack_id: int) -> AdminPack:
        with self._factory() as db:
            pack = db.get(QuestionPack, pack_id)
            if pack is None:
                raise AdminError("Набор с таким номером не найден")
            if pack.owner_id is None:
                raise AdminError("Готовые наборы хранятся в базе данных и редактируются напрямую в MySQL")
            return self._pack_row(db, pack)
    def _moderator(self) -> PackService:
        return PackService(self._factory, PackLibrary([]))
    def delete_pack(self, pack_id: int) -> AdminPack:
        pack = self.get_pack(pack_id)
        try:

self._moderator().moderate(pack_id, "delete")
        except PackError as err:
            raise AdminError(str(err)) from err
        return pack
    def set_hidden(self, pack_id: int, hidden: bool) -> AdminPack:
        self.get_pack(pack_id)
        try:

self._moderator().moderate(pack_id, "hide" if hidden else "restore")
        except PackError as err:
            raise AdminError(str(err)) from err
        return self.get_pack(pack_id)
    def reported_packs(self) -> list[AdminPack]:
        with self._factory() as db:
            reported = select(PackReport.pack_id)
            packs = (
                db.execute(
                    select(QuestionPack)
                    .where(

QuestionPack.owner_id.is_not(None),
or_(QuestionPack.is_hidden.is_(True), QuestionPack.id.in_(reported)),
                    )

.order_by(QuestionPack.id.desc())
                )
                .scalars()
                .all()
            )
            return [self._pack_row(db, p) for p in packs]
```

history_admin_service.py

```
import json
from sqlalchemy import delete, select
from app.database.models import PackWord, QuestionPack, RoomRecord
from app.services.admin_service import AdminError
from app.services.pack_service import PackService, PackError, MIN_WORDS, MAX_WORDS, split_words
from app.game.packs import PackLibrary
class HistoryAdminService:
    """Privileged operations; handlers must check the administrator on each request."""
    def __init__(self, factory):
        self.factory = factory
        self.packs = PackService(factory, PackLibrary([]))
    def pack(self, pack_id):
        with self.factory() as db:
            p = db.get(QuestionPack, pack_id)
            if p is None:
                raise AdminError("Набор уже удалён. Запись о нём сохранена в истории.")
            words = db.execute(select(PackWord.word).where(PackWord.pack_id == pack_id).order_by(PackWord.id)).scalars().all()
            return {"id": p.id, "title": p.title, "public": p.is_public, "hidden": p.is_hidden, "owner": p.owner_id, "words": words}
    def update(self, pack_id, action, value=None):
        with self.factory.begin() as db:
            p = db.execute(select(QuestionPack).where(QuestionPack.id == pack_id).with_for_update()).scalar_one_or_none()
            if p is None:
                raise AdminError("Набор уже удалён")
            if action == "rename":
                p.title = self.packs._check_title(value)
            elif action == "words":
                words, skipped = self.packs._normalize_words(split_words(value))
                if skipped or not words or len(words) > MAX_WORDS:
                    raise AdminError(f"Введите от 1 до {MAX_WORDS} разных слов, каждое не длиннее 40 символов, без повторов.")
                if p.is_public and len(words) < MIN_WORDS:
                    raise AdminError(f"В публичном наборе должно быть не менее {MIN_WORDS} слов. Сначала сделайте его приватным.")

db.execute(delete(PackWord).where(PackWord.pack_id == pack_id))
db.add_all(PackWord(pack_id=pack_id, word=w) for w in words)
            elif action == "public":
                if p.is_hidden:
                    raise AdminError("Сначала снимите блокировку набора")
                count = len(db.execute(select(PackWord.id).where(PackWord.pack_id == pack_id)).all())
                if count < MIN_WORDS:
                    raise AdminError(f"Для публикации нужно не менее {MIN_WORDS} слов")
                p.is_public = True
            elif action == "private":
                p.is_public = False
            elif action == "hide":
                p.is_hidden = True
                p.is_public = False
            elif action == "restore":
                p.is_hidden = False
            elif action == "delete":
                for rec in db.execute(select(RoomRecord)).scalars():
                    state = json.loads(rec.state)
                    if state.get("phase") != "final" and p.slug in state.get("settings", {}).get("themes", []):
                        raise AdminError("Набор выбран в действующей комнате. Заблокируйте его; удалить можно после завершения комнаты или смены тем.")
                self.packs._purge(db, pack_id)
            else:
                raise AdminError("Неизвестное действие")
```

history_service.py

```
from dataclasses import dataclass, field
from sqlalchemy import func, select
from sqlalchemy.orm import sessionmaker
from app.database.models import GameSession, MatchRound, RoundPlayer, SessionPlayer
@dataclass
class UserRound:
    finished_at: object
    lobby_title: str
    theme: str
    word: str
    spy_name: str
    outcome: str
    role: str
    result: str
    score_delta: int
    round_id: int = 0
    pack_id: int | None = None
    pack_title: str | None = None
@dataclass
class LobbyRow:
    id: int
    title: str
    started_at: object
    ended_at: object
    status: str
    players: list[tuple[str, int]]
    winners: list[str]
    rounds_count: int
    player_ids: list[tuple[int, str]] = field(default_factory=list)
@dataclass
class LobbyRound:
    round_no: int
    theme: str
    word: str
    spy_name: str
    outcome: str
    finished_at: object
```

```
    scores: list[tuple[str, str, int]]
    id: int = 0
    pack_id: int | None = None
    pack_title: str | None = None
def leaders(players: list[tuple[str, int]]) -> list[str]:
    if not players:
        return []
    top = max(score for _, score in players)
    if top <= 0:
        return []
    return [name for name, score in players if score == top]
class HistoryService:
    def __init__(self, factory: sessionmaker):
        self._factory = factory
    def user_rounds(self, user_id: int, page: int, size: int) -> tuple[list[UserRound], int]:
        with self._factory() as db:
            total = db.execute(

select(func.count()).select_from(RoundPlayer).where(RoundPlayer.user_id == user_id)
            ).scalar_one()
            stmt = (
                select(RoundPlayer, MatchRound, GameSession)
                .join(MatchRound, RoundPlayer.round_id == MatchRound.id)
                .join(GameSession, MatchRound.session_id == GameSession.id)
                .where(RoundPlayer.user_id == user_id)

.order_by(MatchRound.finished_at.desc(), MatchRound.id.desc())
                .limit(size)
                .offset(page * size)
            )
            rows = [
                UserRound(
                    finished_at=match.finished_at,
                    lobby_title=session.title,
                    theme=match.theme,
                    word=match.word,
                    spy_name=match.spy_name,
                    outcome=match.outcome,
                    role=rp.role,
                    result=rp.result,
                    score_delta=rp.score_delta,
                    round_id=match.id,
pack_id=match.pack_id,
pack_title=match.pack_title,
                )
                for rp, match, session in db.execute(stmt)
            ]
            return rows, total
    def _lobby_rows(self, db, sessions) -> list[LobbyRow]:
        if not sessions:
            return []
        ids = [s.id for s in sessions]
        grouped = {sid: [] for sid in ids}
        for player in db.execute(

select(SessionPlayer).where(SessionPlayer.session_id.in_(ids))

.order_by(SessionPlayer.score.desc(), SessionPlayer.name, SessionPlayer.user_id)
        ).scalars():

grouped[player.session_id].append(player)
        counts = dict(db.execute(
            select(MatchRound.session_id, func.count())

.where(MatchRound.session_id.in_(ids)).group_by(MatchRound.session_id)
        ).all())
        result = []
        for session in sessions:
            members = grouped[session.id]
            players = [(p.name, p.score) for p in members]
            result.append(LobbyRow(
                id=session.id, title=session.title, started_at=session.started_at,
                ended_at=session.ended_at, status=session.status, players=players,
                winners=leaders(players), rounds_count=counts.get(session.id, 0),
                player_ids=[(p.user_id, p.name) for p in members],
            ))
        return result
    def _lobby_row(self, db, session: GameSession) -> LobbyRow:
        return self._lobby_rows(db, [session])[0]
    def _is_member(self, db, user_id: int, session_id: int) -> bool:
        return db.get(SessionPlayer, (session_id, user_id)) is not None
    def user_lobbies(self, user_id: int, page: int, size: int) -> tuple[list[LobbyRow], int]:
        with self._factory() as db:
            total = db.execute(

select(func.count()).select_from(SessionPlayer).where(SessionPlayer.user_id == user_id)
            ).scalar_one()
            stmt = (
                select(GameSession)
                .join(SessionPlayer, SessionPlayer.session_id == GameSession.id)
                .where(SessionPlayer.user_id == user_id)

.order_by(GameSession.started_at.desc(), GameSession.id.desc())
                .limit(size)
                .offset(page * size)
            )
            sessions = db.execute(stmt).scalars().all()
            return self._lobby_rows(db, sessions), total
    def lobby(self, user_id: int, session_id: int) -> LobbyRow | None:
        with self._factory() as db:
            if not self._is_member(db, user_id, session_id):
                return None
            session = db.get(GameSession, session_id)
            return self._lobby_row(db, session) if session else None
    def lobby_rounds(
        self, user_id: int, session_id: int, page: int, size: int
    ) -> tuple[list[LobbyRound], int] | None:
        with self._factory() as db:
            if not self._is_member(db, user_id, session_id):
                return None
            total = db.execute(

select(func.count()).select_from(MatchRound).where(MatchRound.session_id == session_id)
            ).scalar_one()
            stmt = (
                select(MatchRound)
                .where(MatchRound.session_id == session_id)

.order_by(MatchRound.round_no)
                .limit(size)
                .offset(page * size)
            )
            matches = db.execute(stmt).scalars().all()
            grouped = {match.id: [] for match in matches}
            if matches:
                for player in db.execute(

select(RoundPlayer).where(RoundPlayer.round_id.in_(grouped))

.order_by(RoundPlayer.score_delta.desc(), RoundPlayer.name, RoundPlayer.user_id)
                ).scalars():

grouped[player.round_id].append((player.name, player.role, player.score_delta))
            result = []
            for match in matches:
                scores = grouped[match.id]
                result.append(
                    LobbyRound(
                        id=match.id,
pack_id=match.pack_id,
pack_title=match.pack_title,
                        round_no=match.round_no,
                        theme=match.theme,
                        word=match.word,

spy_name=match.spy_name,
                        outcome=match.outcome,

finished_at=match.finished_at,
                        scores=scores,
                    )
                )
            return result, total
    def round_detail(self, user_id: int, round_id: int) -> dict | None:
        with self._factory() as db:
            match = db.get(MatchRound, round_id)
            if match is None or not self._is_member(db, user_id, match.session_id):
                return None
            players = db.execute(select(RoundPlayer).where(RoundPlayer.round_id == round_id).order_by(RoundPlayer.score_delta.desc(), RoundPlayer.name)).scalars()
            return {"id": match.id, "session_id": match.session_id, "round_no": match.round_no,
                    "pack_id": match.pack_id, "pack_title": match.pack_title,
                    "theme": match.theme, "word": match.word,
                    "players": [(p.user_id, p.name, p.score_delta, p.role) for p in players]}
    def player_round(self, user_id: int, session_id: int, target: int) -> int | None:
        with self._factory() as db:
            if not self._is_member(db, user_id, session_id):
                return None
```

```
        return
db.execute(select(MatchRound.id).join(Ro
undPlayer, RoundPlayer.round_id ==
MatchRound.id).where(MatchRound.sessio
n_id == session_id, RoundPlayer.user_id
==
target).order_by(MatchRound.id.desc()).li
mit(1)).scalar_one_or_none()
limits.py
from collections import OrderedDict
import math
class FailedJoinLimiter:
    def __init__(self, clock, attempts=5,
window=60.0, max_users=10000):
        self.clock, self.attempts, self.window,
self.max_users = clock, attempts, window,
max_users
        self._users = OrderedDict()
    def wait_seconds(self, user_id):
        value = self._users.get(user_id)
        if value is None:
            return 0
        start, failures = value
        remaining = start + self.window -
self.clock()
        if remaining <= 0:
            self._users.pop(user_id, None)
            return 0
        return max(1, math.ceil(remaining)) if
failures >= self.attempts else 0
    def failed(self, user_id):
        now = self.clock()
        start, failures =
self._users.pop(user_id, (now, 0))
        if now - start >= self.window:
            start, failures = now, 0
        if len(self._users) >= self.max_users:
            self._users.popitem(last=False)
        self._users[user_id] = (start, failures +
1)
    def succeeded(self, user_id):
        self._users.pop(user_id, None)
pack_service.py
import re
import secrets
from dataclasses import dataclass
from datetime import datetime, timezone
from sqlalchemy import and_, delete, func,
or_, select
from sqlalchemy.orm import sessionmaker
from app.database.models import (
    PackCategory,
    PackReport,
    PackWord,
    QuestionPack,
    UserFavorite,
)
from app.game.packs import Pack,
PackLibrary
MIN_WORDS = 10
MAX_WORDS = 500
MAX_WORD_LEN = 40
MAX_TITLE_LEN = 60
MAX_USER_PACKS = 20
REPORT_THRESHOLD = 3
MY_CATEGORY = "Мои наборы"
SCOPES = ("my", "builtin", "fav", "public")
SORTS = ("new", "name", "size")
REPORT_REASONS = {
    "spam": "Спам",
    "offensive": "Оскорбительное
содержимое",
    "errors": "Ошибки или мусор",
    "other": "Другое",
}
WORD_SPLIT = re.compile(r"[\n;,]+")
class PackError(Exception):
    pass

@dataclass
class PackInfo:
    id: int
    slug: str
    title: str
    description: str
    category: str
    owner_id: int | None
    owner_name: str | None
    is_public: bool
    is_hidden: bool
    words_count: int
    is_favorite: bool
    reports: int = 0
    @property
    def is_builtin(self) -> bool:
        return self.owner_id is None
def clean_text(raw: str) -> str:
    return " ".join((raw or "").split())
def word_key(word: str) -> str:
    return word.casefold().replace("ё", "е")
def split_words(text: str) -> list[str]:
    return [clean_text(part) for part in
WORD_SPLIT.split(text or "") if
clean_text(part)]
class PackService:
    def __init__(self, factory: sessionmaker,
base_library: PackLibrary):
        self._factory = factory
        self._base = base_library
    def _visible_clause(self, user_id: int):
        return or_(
            QuestionPack.owner_id.is_(None),
            QuestionPack.owner_id == user_id,

and_(QuestionPack.is_public.is_(True),
QuestionPack.is_hidden.is_(False)),
        )
    def _pack(self, db, pack_id: int, user_id:
int | None = None, owned: bool = False,
lock: bool = False) -> QuestionPack:
        pack =
db.execute(select(QuestionPack).where(Qu
estionPack.id ==
pack_id).with_for_update().execution_opti
ons(populate_existing=True)).scalar_one_o
r_none() if lock else db.get(QuestionPack,
pack_id)
        if pack is None:
            raise PackError("Набор не
найден")
        if owned:
            if pack.owner_id is None or
pack.owner_id != user_id:
                raise PackError("Это действие
доступно только автору набора")
        elif user_id is not None:
            visible = (
                pack.owner_id is None
                or pack.owner_id == user_id
                or (pack.is_public and not
pack.is_hidden)
            )
            if not visible:
                raise PackError("Набор
недоступен")
        return pack
    def _info(self, db, pack: QuestionPack,
user_id: int | None) -> PackInfo:
        count = db.execute(

select(func.count()).select_from(PackWord
).where(PackWord.pack_id == pack.id)
        ).scalar_one()
        category = db.get(PackCategory,
pack.category_id)
        favorite = False
        if user_id is not None:
            favorite = db.get(UserFavorite,
(user_id, pack.id)) is not None
        reports = db.execute(

select(func.count()).select_from(PackRepor
t).where(PackReport.pack_id == pack.id)
        ).scalar_one()
        return PackInfo(
            id=pack.id,
            slug=pack.slug,
            title=pack.title,
            description=pack.description or "",
            category=category.title if category
else "",
            owner_id=pack.owner_id,
            owner_name=pack.owner_name,
            is_public=bool(pack.is_public),
            is_hidden=bool(pack.is_hidden),
            words_count=count,
            is_favorite=favorite,
            reports=reports,
        )
    def info(self, user_id: int, pack_id: int) ->
PackInfo:
        with self._factory() as db:
            return self._info(db, self._pack(db,
pack_id, user_id), user_id)
    def categories(self) -> list[tuple[int, str]]:
        with self._factory() as db:
            rows =
db.execute(select(PackCategory.id,
PackCategory.title).order_by(PackCategory
.title))
            return [(row[0], row[1]) for row in
rows]
    def list_packs(
        self,
        user_id: int,
        scope: str,
        page: int = 0,
        size: int = 6,
        query: str | None = None,
        sort: str = "new",
        category_id: int | None = None,
    ) -> tuple[list[PackInfo], int]:
        if scope not in SCOPES:
            raise PackError("Неизвестный
раздел")
        if sort not in SORTS:
            sort = "new"
        conds = []
        if scope == "my":

conds.append(QuestionPack.owner_id ==
user_id)
        elif scope == "builtin":
            conds +=
[QuestionPack.owner_id.is_(None),
QuestionPack.is_public.is_(True),
QuestionPack.is_hidden.is_(False)]
        elif scope == "public":
            conds += [

QuestionPack.owner_id.is_not(None),

QuestionPack.is_public.is_(True),

QuestionPack.is_hidden.is_(False),
            ]
        else:
            conds += [

QuestionPack.id.in_(select(UserFavorite.pa
ck_id).where(UserFavorite.user_id ==
user_id)),
                self._visible_clause(user_id),
            ]
        if category_id:
```

```
conds.append(QuestionPack.category_id
== category_id)
        if query:
            term = clean_text(query)
            conds.append(
                or_(

QuestionPack.title.contains(term,
autoescape=True),
                    QuestionPack.id.in_(

select(PackWord.pack_id).where(PackWor
d.word.contains(term, autoescape=True))
                    ),
                )
            )
        size_expr = (

select(func.count()).where(PackWord.pack
_id == QuestionPack.id).scalar_subquery()
        )
        if sort == "name":
            order = [QuestionPack.title,
QuestionPack.id]
        elif sort == "size":
            order = [size_expr.desc(),
QuestionPack.id]
        elif scope == "builtin":
            order = [QuestionPack.position,
QuestionPack.id]
        else:
            order =
[QuestionPack.updated_at.desc(),
QuestionPack.id.desc()]
        with self._factory() as db:
            total = db.execute(

select(func.count()).select_from(QuestionP
ack).where(*conds)
            ).scalar_one()
            packs = (
                db.execute(
                    select(QuestionPack)
                    .where(*conds)
                    .order_by(*order)
                    .limit(size)
                    .offset(page * size)
                )
                .scalars()
                .all()
            )
            return [self._info(db, p, user_id) for
p in packs], total
    def _normalize_words(self, raw_words:
list[str]) -> tuple[list[str], int]:
        words, skipped, seen = [], 0, set()
        for raw in raw_words:
            word = clean_text(raw)
            if not word or len(word) >
MAX_WORD_LEN or word_key(word) in
seen:
                skipped += 1
                continue
            seen.add(word_key(word))
            words.append(word)
        return words, skipped
    def _check_title(self, title: str) -> str:
        cleaned = clean_text(title)
        if not cleaned:
            raise PackError("Название не
может быть пустым")
        if len(cleaned) > MAX_TITLE_LEN:
            raise PackError(f"Название
длиннее {MAX_TITLE_LEN}
символов")
        return cleaned
    def _my_category(self, db) ->
PackCategory:
        category = db.execute(

select(PackCategory).where(PackCategory.
title == MY_CATEGORY)
        ).scalar_one_or_none()
        if category is None:
            category =
PackCategory(title=MY_CATEGORY)
            db.add(category)
            db.flush()
        return category
    def _new_pack(
        self, db, user_id: int, owner_name: str,
title: str, description: str, words: list[str]
    ) -> QuestionPack:
        count = db.execute(

select(func.count()).select_from(QuestionP
ack).where(QuestionPack.owner_id ==
user_id)
        ).scalar_one()
        if count >= MAX_USER_PACKS:
            raise PackError(f"Можно создать
не более {MAX_USER_PACKS}
наборов")
        category = self._my_category(db)
        pack = QuestionPack(
            slug=f"tmp{secrets.token_hex(6)}",
            title=title,
            description=description[:255],
            category_id=category.id,
            owner_id=user_id,

owner_name=clean_text(owner_name)[:40
] or "Игрок",
            is_public=False,
            is_hidden=False,
            position=0,
        )
        db.add(pack)
        db.flush()
        pack.slug = f"u{pack.id}"
        for word in words[:MAX_WORDS]:
            db.add(PackWord(pack_id=pack.id,
word=word))
        return pack
    def create_pack(
        self,
        user_id: int,
        owner_name: str,
        title: str,
        words: list[str] | None = None,
        description: str = "",
    ) -> int:
        title = self._check_title(title)
        cleaned, _ =
self._normalize_words(words or [])
        with self._factory.begin() as db:
            pack = self._new_pack(db, user_id,
owner_name, title, description, cleaned)
            return pack.id
    def add_words(self, user_id: int, pack_id:
int, raw_words: list[str]) -> tuple[int, int]:
        words, skipped =
self._normalize_words(raw_words)
        with self._factory.begin() as db:
            pack = self._pack(db, pack_id,
user_id, owned=True)
            existing = {
                word_key(row[0])
                for row in
db.execute(select(PackWord.word).where(
PackWord.pack_id == pack.id))
            }
            total = len(existing)
            added = 0
            for word in words:
                if word_key(word) in existing or
total >= MAX_WORDS:
                    skipped += 1
                    continue

db.add(PackWord(pack_id=pack.id,
word=word))
                existing.add(word_key(word))
                total += 1
                added += 1
            pack.updated_at =
datetime.now(timezone.utc)
            return added, skipped
    def list_words(
        self, user_id: int, pack_id: int, page:
int, size: int
    ) -> tuple[list[tuple[int, str]], int]:
        with self._factory() as db:
            self._pack(db, pack_id, user_id)
            total = db.execute(

select(func.count()).select_from(PackWord
).where(PackWord.pack_id == pack_id)
            ).scalar_one()
            rows = db.execute(
                select(PackWord.id,
PackWord.word)
                .where(PackWord.pack_id ==
pack_id)
                .order_by(PackWord.word,
PackWord.id)
                .limit(size)
                .offset(page * size)
            )
            return [(row[0], row[1]) for row in
rows], total
    def word(self, user_id: int, word_id: int) ->
tuple[int, str]:
        with self._factory() as db:
            row = db.get(PackWord, word_id)
            if row is None:
                raise PackError("Слово не
найдено")
            self._pack(db, row.pack_id, user_id,
owned=True)
            return row.pack_id, row.word
    def edit_word(self, user_id: int, word_id:
int, new_word: str) -> int:
        cleaned = clean_text(new_word)
        if not cleaned or len(cleaned) >
MAX_WORD_LEN:
            raise PackError(f"Слово должно
быть от 1 до {MAX_WORD_LEN}
символов")
        with self._factory.begin() as db:
            row = db.get(PackWord, word_id)
            if row is None:
                raise PackError("Слово не
найдено")
            self._pack(db, row.pack_id, user_id,
owned=True)
            others = {
                word_key(r[0])
                for r in db.execute(

select(PackWord.word).where(PackWord.p
ack_id == row.pack_id, PackWord.id !=
word_id)
                )
            }
            if word_key(cleaned) in others:
                raise PackError("Такое слово
уже есть в наборе")
            row.word = cleaned
            return row.pack_id
    def delete_word(self, user_id: int,
word_id: int) -> int:
```

```
        with self._factory.begin() as db:
            row = db.get(PackWord, word_id)
            if row is None:
                raise PackError("Слово не найдено")
            pack = self._pack(db, row.pack_id, user_id, owned=True, lock=True)
            count = db.execute(select(func.count()).select_from(PackWord).where(PackWord.pack_id == pack.id)).scalar_one()
            if pack.is_public and count <= MIN_WORDS:
                raise PackError(f"В публичном наборе должно остаться не менее {MIN_WORDS} слов. Сначала сделайте набор личным.")
            pack_id = row.pack_id
            db.delete(row)
            return pack_id
    def rename(self, user_id: int, pack_id: int, title: str) -> None:
        title = self._check_title(title)
        with self._factory.begin() as db:
            self._pack(db, pack_id, user_id, owned=True).title = title
    def set_public(self, user_id: int, pack_id: int, public: bool) -> bool:
        with self._factory.begin() as db:
            pack = self._pack(db, pack_id, user_id, owned=True, lock=True)
            if public:
                if pack.is_hidden:
                    raise PackError("Набор скрыт модерацией и не может быть опубликован")
                count = db.execute(
                    select(func.count()).select_from(PackWord).where(PackWord.pack_id == pack.id)
                ).scalar_one()
                if count < MIN_WORDS:
                    raise PackError(f"Для публикации нужно не менее {MIN_WORDS} слов")
            pack.is_public = public
            return public
    def copy_pack(self, user_id: int, owner_name: str, pack_id: int) -> int:
        with self._factory.begin() as db:
            source = self._pack(db, pack_id, user_id)
            words = [
                row[0]
                for row in db.execute(
                    select(PackWord.word).where(PackWord.pack_id == source.id).order_by(PackWord.id)
                )
            ]
            title = clean_text(f"Копия: {source.title}")[:MAX_TITLE_LEN]
            pack = self._new_pack(db, user_id, owner_name, title, source.description or "", words)
            return pack.id
    def _purge(self, db, pack_id: int) -> None:
        db.execute(delete(PackWord).where(PackWord.pack_id == pack_id))
        db.execute(delete(UserFavorite).where(UserFavorite.pack_id == pack_id))
        db.execute(delete(PackReport).where(PackReport.pack_id == pack_id))
        db.execute(delete(QuestionPack).where(QuestionPack.id == pack_id))
    def delete_pack(self, user_id: int, pack_id: int) -> None:
        with self._factory.begin() as db:
            self._pack(db, pack_id, user_id, owned=True)
            self._purge(db, pack_id)
    def toggle_favorite(self, user_id: int, pack_id: int) -> bool:
        with self._factory.begin() as db:
            self._pack(db, pack_id, user_id)
            row = db.get(UserFavorite, (user_id, pack_id))
            if row:
                db.delete(row)
                return False
            db.add(UserFavorite(user_id=user_id, pack_id=pack_id))
            return True
    def export_text(self, user_id: int, pack_id: int) -> tuple[str, str]:
        with self._factory() as db:
            pack = self._pack(db, pack_id, user_id)
            words = [
                row[0]
                for row in db.execute(
                    select(PackWord.word).where(PackWord.pack_id == pack.id).order_by(PackWord.id)
                )
            ]
            return pack.slug, "\n".join([f"#{pack.title}"] + words) + "\n"
    def import_text(self, user_id: int, owner_name: str, text: str) -> int:
        lines = [line for line in (text or "").splitlines() if line.strip()]
        title = "Импортированный набор"
        if lines and lines[0].lstrip().startswith("#"):
            title = lines.pop(0).lstrip().lstrip("#").strip() or title
        words, _ = self._normalize_words(split_words("\n".join(lines)))
        if not words:
            raise PackError("В файле не найдено ни одного корректного слова")
        return self.create_pack(user_id, owner_name, title, words)
    def report(self, user_id: int, pack_id: int, reason_code: str) -> bool:
        reason = REPORT_REASONS.get(reason_code)
        if reason is None:
            raise PackError("Неизвестная причина жалобы")
        with self._factory.begin() as db:
            pack = self._pack(db, pack_id, user_id)
            if pack.owner_id is None or pack.owner_id == user_id or not pack.is_public:
                raise PackError("Пожаловаться можно только на чужой публичный набор")
            if db.execute(
                select(PackReport.id).where(
                    PackReport.pack_id == pack_id, PackReport.reporter_id == user_id
                )
            ).first():
                raise PackError("Вы уже отправляли жалобу на этот набор")
            db.add(PackReport(pack_id=pack_id, reporter_id=user_id, reason=reason))
            db.flush()
            count = db.execute(
                select(func.count()).select_from(PackReport).where(PackReport.pack_id == pack_id)
            ).scalar_one()
            if count >= REPORT_THRESHOLD:
                pack.is_hidden = True
            return bool(pack.is_hidden)
    def moderation_list(self, page: int, size: int) -> tuple[list[PackInfo], int]:
        reported = select(PackReport.pack_id)
        cond = or_(QuestionPack.is_hidden.is_(True), QuestionPack.id.in_(reported))
        with self._factory() as db:
            total = db.execute(
                select(func.count()).select_from(QuestionPack).where(cond)
            ).scalar_one()
            packs = (
                db.execute(
                    select(QuestionPack)
                    .where(cond)
                    .order_by(QuestionPack.updated_at.desc(), QuestionPack.id.desc())
                    .limit(size)
                    .offset(page * size)
                )
                .scalars()
                .all()
            )
            return [self._info(db, p, None) for p in packs], total
    def moderation_info(self, pack_id: int) -> tuple[PackInfo, list[str]]:
        with self._factory() as db:
            pack = self._pack(db, pack_id)
            reasons = [
                row[0]
                for row in db.execute(
                    select(PackReport.reason).where(PackReport.pack_id == pack_id).order_by(PackReport.id)
                )
            ]
            return self._info(db, pack, None), reasons
    def moderate(self, pack_id: int, action: str) -> None:
        with self._factory.begin() as db:
            pack = self._pack(db, pack_id)
            if pack.owner_id is None:
                raise PackError("Готовые наборы модерации не подлежат")
            if action == "restore":
                pack.is_hidden = False
                db.execute(delete(PackReport).where(PackReport.pack_id == pack_id))
            elif action == "hide":
                pack.is_hidden = True
                pack.is_public = False
            elif action == "delete":
                self._purge(db, pack_id)
            else:
                raise PackError("Неизвестное действие")
    def _to_packs(self, db, packs: list[QuestionPack]) -> list[Pack]:
        if not packs:
```

```
            return []
        ids = [p.id for p in packs]
        words: dict[int, list[str]] = {}
        for row in db.execute(
            select(PackWord.pack_id,
PackWord.word).where(PackWord.pack_id
.in_(ids)).order_by(PackWord.id)
        ):
            words.setdefault(row[0],
[]).append(row[1])
        categories = {c.id: c.title for c in
db.execute(select(PackCategory)).scalars()
}
        return [
            Pack(
                id=p.slug,
                title=p.title,
                description=p.description or "",

category=categories.get(p.category_id, ""),
                words=tuple(words.get(p.id, [])),
            )
            for p in packs
            if words.get(p.id)
        ]
    def library_for(self, user_id: int) ->
PackLibrary:
        # Read built-in packs too: admin
changes must apply without restarting.
        with self._factory() as db:
            favorites =
select(UserFavorite.pack_id).where(UserFa
vorite.user_id == user_id)
            packs =
db.execute(select(QuestionPack).where(

QuestionPack.is_hidden.is_(False),
                or_(

and_(QuestionPack.owner_id.is_(None),
QuestionPack.is_public.is_(True)),
                    QuestionPack.owner_id ==
user_id,

and_(QuestionPack.id.in_(favorites),
QuestionPack.is_public.is_(True)),
                ),
            ).order_by(QuestionPack.position,
QuestionPack.id)).scalars().all()
            return
PackLibrary(self._to_packs(db, packs))
    def packs_by_slugs(self, slugs: list[str]) -
> list[Pack]:
        if not slugs:
            return []
        with self._factory() as db:
            packs = (

db.execute(select(QuestionPack).where(Qu
estionPack.slug.in_(slugs))).scalars().all()
            )
            return self._to_packs(db, packs)
penalty.py
LEAVE_BAN_MINUTES = [5, 30, 120,
720, 1440]
STRIKE_RESET_SECONDS = 7 * 24 *
3600
LEAVE_CONFIRM_SECONDS = 10
def next_penalty(strikes: int, last_leave_at:
float, now: float) -> tuple[int, int]:
    if strikes > 0 and now - last_leave_at >
STRIKE_RESET_SECONDS:
        strikes = 0
    new_strikes = strikes + 1
    minutes =
LEAVE_BAN_MINUTES[min(new_strike
s, len(LEAVE_BAN_MINUTES)) - 1]
    return new_strikes, minutes
def format_duration(seconds: float) -> str:
    total = max(0, int(seconds + 0.999))
    minutes = -(-total // 60)
    if minutes < 60:
        return f"{max(minutes, 1)} мин"
    hours, rest = divmod(minutes, 60)
    return f"{hours} ч {rest} мин" if rest else
f"{hours} ч"
PERMANENT_UNTIL = 4102444800.0
PERMANENT_THRESHOLD_SECOND
S = 3650 * 24 * 3600
def describe_ban(seconds: float) -> str:
    if seconds >
PERMANENT_THRESHOLD_SECOND
S:
        return "навсегда"
    return format_duration(seconds)
def ban_message(seconds: float, reason: str)
-> str:
    left = describe_ban(seconds)
    if reason == "admin":
        if left == "навсегда":
            return "Вы заблокированы
администратором навсегда."
        return f"Вы заблокированы
администратором. Осталось: {left}."
    return f"Вы заблокированы за выход из
игры во время раунда. Осталось: {left}."
room_service.py
import json
import random
import re
import secrets
import time
from dataclasses import dataclass, field
from datetime import datetime, timedelta,
timezone
from sqlalchemy import delete, select,
update
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import sessionmaker
from app.database.models import (
    GameSession,
    MatchRound,
    QuestionPack,
    RoomMember,
    RoomRecord,
    RoundPlayer,
    SessionPlayer,
    UserPenalty,
)
from app.game.engine import Phase
from app.services.limits import
FailedJoinLimiter
from app.game.packs import PackLibrary,
load_default_library
from app.game.room import Room,
RoomError, new_code
from app.services.penalty import (
    ban_message,
    format_duration,
    next_penalty,
)
@dataclass
class Update:
    room_id: int
    room: Room | None
    alert: str | None = None
    changed: bool = True
    left_user: int | None = None
    penalty: tuple[int, int] | None = None
class RoomService:
    def __init__(
        self,
        factory: sessionmaker,
        library: PackLibrary | None = None,
        rng: random.Random | None = None,
        clock=time.time,
        pack_service=None,
    ):
        self._factory = factory
        self._library = library or
load_default_library()
        self._rng = rng
        self._clock = clock
        self._packs = pack_service
        self.join_limiter =
FailedJoinLimiter(clock)
    def _host_library(self, host_id: int,
themes: list[str] | None = None):
        if self._packs is None:
            return self._library
        library =
self._packs.library_for(host_id)
        missing = [t for t in (themes or []) if not
library.has(t)]
        if missing:
            library =
library.with_packs(self._packs.packs_by_sl
ugs(missing))
        return library
    def _room(self, rec: RoomRecord) ->
Room:
        state = json.loads(rec.state)
        themes = state.get("settings",
{}).get("themes")
        if self._packs is not None and
state.get("phase") != "lobby" and themes:
            # A running round already carries its
own words and roles. Only
            # selected packs are needed to
restore its views; theme selection
            # in the lobby still loads the full,
current library.
            library =
PackLibrary(self._packs.packs_by_slugs(th
emes))
        else:
            library =
self._host_library(rec.host_id, themes)
        return Room.from_dict(state, library,
self._rng)
    def _activity_time(self):
        return
datetime.fromtimestamp(self._clock(),
timezone.utc)
    def _store(self, rec: RoomRecord, room:
Room, *, player_activity=False) -> None:
        room.prune_missing_themes()
        rec.state = json.dumps(room.to_dict(),
ensure_ascii=False)
        rec.host_id = room.host_id
        active = (
            room.phase == "playing"
            and room.round is not None
            and room.round.phase in
(Phase.DISCUSSION,
Phase.LAST_CHANCE)
        )
        rec.deadline = room.deadline if active
else None
        rec.updated_at =
datetime.now(timezone.utc)
        if player_activity:
            rec.last_activity_at =
self._activity_time()
    def _lock(self, db, room_id: int) ->
RoomRecord | None:
        stmt =
select(RoomRecord).where(RoomRecord.i
d ==
room_id).with_for_update().execution_opti
ons(populate_existing=True)
        return
db.execute(stmt).scalar_one_or_none()
```

```
    def panel_id_of(self, user_id: int) -> int | None:
        """Read the panel identity without reconstructing a game or its packs."""
        with self._factory() as db:
            state = db.execute(
                select(RoomRecord.state)
                .join(RoomMember, RoomMember.room_id == RoomRecord.id)
                .where(RoomMember.user_id == user_id)
            ).scalar_one_or_none()
            if state is None:
                return None
            player = next((p for p in json.loads(state).get("players", []) if p["id"] == user_id), None)
            return player.get("panel") if player else None
    def _record_history(self, db, rec: RoomRecord, room: Room) -> None:
        if room.phase != "result" or not room.history:
            return
        last = room.history[-1]
        if room.session_id is None:
            session = GameSession(
                room_id=rec.id,
                room_code=room.code,
                title=room.title,
                host_id=room.host_id,
                status="active",
                is_test=room.is_test,
                started_at=rec.created_at,
            )
            db.add(session)
            db.flush()
            room.session_id = session.id
        known = db.execute(
            select(MatchRound.id).where(
                MatchRound.session_id == room.session_id, MatchRound.round_no == last["round"]
            )
        ).first()
        if known:
            return
        spy_id = int(last["spy"])
        names = last.get("names") or {str(p["id"]): p["name"] for p in room.players}
        pack = db.execute(select(QuestionPack).where(QuestionPack.slug == room.round.theme_id)).scalar_one_or_none() if room.round else None
        match = MatchRound(
            session_id=room.session_id,
            round_no=last["round"],
            theme=room.round.theme if room.round else "",
            word=last["word"],
            pack_id=pack.id if pack else None,
            pack_title=room.round.theme if room.round else None,
            spy_user_id=spy_id,
            spy_name=names.get(last["spy"], "?"),
            outcome=last["outcome"],
            finished_at=datetime.now(timezone.utc),
        )
        db.add(match)
        db.flush()
        for pid, delta in last["deltas"].items():
            uid = int(pid)
            player_name = names.get(pid, "Игрок")
            role = "spy" if uid == spy_id else "civilian"
            if last["outcome"] == "draw":
                result = "draw"
            elif last["outcome"] in ("civilians_win", "spy_wrong_guess", "spy_left"):
                result = "loss" if role == "spy" else "win"
            else:
                result = "win" if role == "spy" else "loss"
            db.add(
                RoundPlayer(
                    round_id=match.id,
                    user_id=uid,
                    name=player_name,
                    role=role,
                    result=result,
                    score_delta=delta,
                )
            )
            total = room.scores.get(pid, 0)
            row = db.get(SessionPlayer, (room.session_id, uid))
            if row:
                row.name = player_name
                row.score = total
            else:
                db.add(
                    SessionPlayer(
                        session_id=room.session_id, user_id=uid, name=player_name, score=total
                    )
                )
    def _end_session(self, db, session_id: int | None, status: str) -> None:
        if session_id is None:
            return
        db.execute(
            update(GameSession)
            .where(GameSession.id == session_id, GameSession.status == "active")
            .values(status=status, ended_at=datetime.now(timezone.utc))
        )
    def ban_seconds(self, user_id: int) -> float:
        return self.ban_status(user_id)[0]
    def ban_status(self, user_id: int) -> tuple[float, str]:
        with self._factory() as db:
            row = db.get(UserPenalty, user_id)
            if row is None:
                return 0.0, "leave"
            return max(0.0, row.banned_until - self._clock()), row.reason or "leave"
    def _check_ban(self, db, user_id: int) -> None:
        row = db.get(UserPenalty, user_id)
        if row is not None:
            left = row.banned_until - self._clock()
            if left > 0:
                raise RoomError(ban_message(left, row.reason or "leave"))
    def _penalty_preview(self, db, user_id: int, now: float) -> tuple[int, int]:
        row = db.get(UserPenalty, user_id)
        return next_penalty(row.strikes if row else 0, row.last_leave_at if row else 0.0, now)
    def _apply_penalty(self, db, user_id: int, now: float) -> tuple[int, int]:
        strikes, minutes = self._penalty_preview(db, user_id, now)
        row = db.get(UserPenalty, user_id)
        if row is None:
            row = UserPenalty(user_id=user_id)
            db.add(row)
        row.strikes = strikes
        until = now + minutes * 60
        if until >= (row.banned_until or 0.0):
            row.banned_until = until
            row.reason = "leave"
        row.last_leave_at = now
        return strikes, minutes
    def room_id_of(self, user_id: int) -> int | None:
        with self._factory() as db:
            member = db.get(RoomMember, user_id)
            return member.room_id if member else None
    def get(self, room_id: int) -> Room | None:
        with self._factory.begin() as db:
            rec = db.get(RoomRecord, room_id)
            if rec is None:
                return None
            original_themes = json.loads(rec.state).get("settings", {}).get("themes", [])
            room = self._room(rec)
            if room.phase == "lobby" and original_themes != room.settings["themes"]:
                rec = self._lock(db, room_id)
                if rec is None:
                    return None
                room = self._room(rec)
                self._store(rec, room)
            return room
    def create(self, user_id: int, name: str, test_size: int | None = None) -> Update:
        if test_size not in (None, 1, 2):
            raise RoomError("Тестовое лобби возможно только на 1 или 2 игроков")
        with self._factory.begin() as db:
            if db.get(RoomMember, user_id):
                raise RoomError("Вы уже находитесь в комнате")
            if test_size is None:
                self._check_ban(db, user_id)
            code = new_code()
            while db.execute(select(RoomRecord.id).where(RoomRecord.code == code)).first():
                code = new_code()
            token = secrets.token_urlsafe(9)
            room = Room(code, token, user_id, "", self._host_library(user_id), self._rng)
            room.join(user_id, name)
            room.title = f"Комната {room.players[0]['name']}"
            if test_size:
                room.enable_test_mode(test_size, user_id)
            rec = RoomRecord(code=code, invite_token=token, host_id=user_id, state="")
            db.add(rec)
            db.flush()
            self._store(rec, room, player_activity=True)
            db.add(RoomMember(user_id=user_id, room_id=rec.id))
            return Update(room_id=rec.id, room=room)
```

```
    def _join(self, db, rec: RoomRecord | None, user_id: int, name: str) -> Update:
        if rec is None:
            raise RoomError("Комната не найдена. Проверьте код или ссылку")
        member = db.get(RoomMember, user_id)
        if member:
            if member.room_id == rec.id:
                rec.last_activity_at = self._activity_time()
                return Update(room_id=rec.id, room=self._room(rec), changed=False)
            raise RoomError("Вы уже в другой комнате. Сначала выйдите из неё")
        self._check_ban(db, user_id)
        locked = self._lock(db, rec.id)
        if locked is None:
            raise RoomError("Лобби больше неактивно. Попросите новое приглашение.")
        room = self._room(locked)
        room.join(user_id, name)
        self._store(locked, room, player_activity=True)

db.add(RoomMember(user_id=user_id, room_id=rec.id))
        return Update(room_id=rec.id, room=room)
    def _check_join_limit(self, user_id):
        wait = self.join_limiter.wait_seconds(user_id)
        if wait:
            raise RoomError(f"Слишком много неудачных попыток входа. Повторите через {wait} с.")
    def join_code(self, user_id: int, name: str, code: str) -> Update:
        self._check_join_limit(user_id)
        code = (code or "").strip().upper()
        try:
            if re.fullmatch(r"[A-Z0-9]{6}", code) is None:
                raise RoomError("Код комнаты должен содержать 6 латинских букв или цифр")
            with self._factory.begin() as db:
                rec = db.execute(select(RoomRecord).where(RoomRecord.code == code)).scalar_one_or_none()
                result = self._join(db, rec, user_id, name)
        except RoomError:
            self.join_limiter.failed(user_id)
            raise
        self.join_limiter.succeeded(user_id)
        return result
    def join_token(self, user_id: int, name: str, token: str) -> Update:
        self._check_join_limit(user_id)
        try:
            if not isinstance(token, str) or re.fullmatch(r"[A-Za-z0-9_-]{1,32}", token) is None:
                raise RoomError("Ссылка приглашения недействительна. Попросите новое приглашение.")
            with self._factory.begin() as db:
                rec = db.execute(select(RoomRecord).where(RoomRecord.invite_token == token)).scalar_one_or_none()
                if rec is None:
                    raise RoomError("Лобби неактивно или ссылка недействительна. Попросите новое приглашение.")
                result = self._join(db, rec, user_id, name)
        except RoomError:
            self.join_limiter.failed(user_id)
            raise
        self.join_limiter.succeeded(user_id)
        return result
    @staticmethod
    def _validate_panel(room, user_id, panel_id):
        if panel_id is None:
            return
        player = next((p for p in room.players if p["id"] == user_id), None)
        if player is None or player.get("panel") != panel_id:
            raise RoomError("Этот игровой экран устарел. Откройте «Моя комната».")
    def _leave_prompt(self, db, room, user_id):
        penalizable = room.phase in ("dealing", "playing") and room.round is not None and not room.is_test
        if not penalizable:
            return "Вы действительно хотите покинуть комнату?"
        strikes, minutes = self._penalty_preview(db, user_id, self._clock())
        return (
            "Выход во время раунда приведёт к блокировке онлайн-игр на "
            f"{format_duration(minutes * 60)} (нарушение №{strikes}).\n\n"
            "Вы действительно хотите выйти?"
        )
    def leave_confirmation(self, user_id: int, *, expected_room_id=None, expected_panel_id=None) -> str:
        """Return the confirmation-dialog text without changing room state."""
        with self._factory.begin() as db:
            member = db.get(RoomMember, user_id)
            if member is None:
                raise RoomError("Вы не находитесь в комнате")
            rec = db.get(RoomRecord, member.room_id)
            if rec is None:
                raise RoomError("Комната больше не существует")
            if expected_room_id is not None and member.room_id != expected_room_id:
                raise RoomError("Это подтверждение относится к другой комнате")
            room = self._room(rec)
            self._validate_panel(room, user_id, expected_panel_id)
            rec.last_activity_at = self._activity_time()
            return self._leave_prompt(db, room, user_id)
    def act(self, user_id: int, action: str, *, expected_room_id=None, expected_panel_id=None, expected_leave_text=None) -> Update:
        with self._factory.begin() as db:
            member = db.get(RoomMember, user_id)
            if member is None:
                raise RoomError("Вы не находитесь в комнате")
            if expected_room_id is not None and member.room_id != expected_room_id:
                raise RoomError("Этот экран относится к другой комнате. Откройте «Моя комната».")
            rec = self._lock(db, member.room_id)
            if rec is None:
                db.delete(member)
                raise RoomError("Комната больше не существует")
            room = self._room(rec)
            self._validate_panel(room, user_id, expected_panel_id)
            if expected_leave_text is not None and self._leave_prompt(db, room, user_id) != expected_leave_text:
                raise RoomError("Условия выхода изменились. Нажмите «Выйти» и подтвердите заново.")
            now = self._clock()
            if action == "leave_confirm":
                action = "leave"
            if action in ("start", "next") and self._packs is not None:
                available = self._packs.library_for(room.host_id)
                if any(not available.has(slug) for slug in room.settings.get("themes", [])):
                    raise RoomError("Выбранный набор удалён, заблокирован или стал недоступен. Выберите темы заново.")
            result = room.act(user_id, action, now)
            penalty = None
            if result.left and result.penalized:
                penalty = self._apply_penalty(db, user_id, now)
            if result.left:
                db.delete(member)
            if result.closed:
                db.flush()
                self._end_session(db, room.session_id, "abandoned")

db.execute(delete(RoomMember).where(RoomMember.room_id == rec.id))
                db.delete(rec)
                return Update(
                    room_id=rec.id, room=None, left_user=user_id, changed=False, penalty=penalty
                )
            self._record_history(db, rec, room)
            if room.phase == "final":
                self._end_session(db, room.session_id, "finished")
            self._store(rec, room, player_activity=True)
            return Update(
                room_id=rec.id,
                room=room,
                alert=result.alert,
                changed=result.changed,
                left_user=user_id if result.left else None,
                penalty=penalty,
            )
    def say(self, user_id: int, text: str) -> Update:
        with self._factory.begin() as db:
            member = db.get(RoomMember, user_id)
            if member is None:
                raise RoomError("Вы не находитесь в комнате")
            rec = self._lock(db, member.room_id)
            if rec is None:
```

```
            raise RoomError("Комната больше не существует")
        room = self._room(rec)
        room.say(user_id, text, self._clock())
        self._store(rec, room, player_activity=True)
        return Update(room_id=rec.id, room=room)
    def due_room_ids(self) -> list[int]:
        with self._factory() as db:
            stmt = select(RoomRecord.id).where(
                RoomRecord.deadline.is_not(None),
                RoomRecord.deadline <= self._clock()
            )
            return [row[0] for row in db.execute(stmt)]
    def active_room_ids(self) -> list[int]:
        with self._factory() as db:
            stmt = select(RoomRecord.id).where(RoomRecord.deadline.is_not(None))
            return [row[0] for row in db.execute(stmt)]
    def tick_room(self, room_id: int) -> Update | None:
        with self._factory.begin() as db:
            rec = self._lock(db, room_id)
            if rec is None:
                return None
            room = self._room(rec)
            changed = room.tick(self._clock())
            self._record_history(db, rec, room)
            self._store(rec, room)
            return Update(room_id=rec.id, room=room, changed=changed)
    def set_panel(self, room_id: int, user_id: int, message_id: int, *, player_activity=False) -> None:
        with self._factory.begin() as db:
            rec = self._lock(db, room_id)
            if rec is None:
                return
            room = self._room(rec)
            for player in room.players:
                if player["id"] == user_id:
                    player["panel"] = message_id
            self._store(rec, room, player_activity=player_activity)
    def purge_stale(self, hours: int = 24) -> int:
        cutoff = self._activity_time() - timedelta(hours=hours)
        with self._factory.begin() as db:
            ids = [
                row[0]
                for row in db.execute(select(RoomRecord.id).where(
                    RoomRecord.last_activity_at < cutoff).with_for_update())
            ]
            if ids:
                db.execute(
                    update(GameSession)
                    .where(GameSession.room_id.in_(ids), GameSession.status == "active")
                    .values(status="abandoned", ended_at=datetime.now(timezone.utc))
                )
                for room_id in ids:
                    db.execute(delete(RoomMember).where(RoomMember.room_id == room_id))
                    db.execute(delete(RoomRecord).where(RoomRecord.id == room_id))
            return len(ids)
settings_service.py
from dataclasses import dataclass
from datetime import datetime, timedelta, timezone
from sqlalchemy.orm import sessionmaker
from app.database.models import UserSetting
OFFSETS = [-8, -5, -3, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
OFFSET_HINTS = {
    -8: "Лос-Анджелес",
    -5: "Нью-Йорк",
    -3: "Бразилия",
    0: "Лондон",
    1: "Берлин",
    2: "Калининград",
    3: "Москва",
    4: "Самара",
    5: "Екатеринбург",
    6: "Омск",
    7: "Новосибирск",
    8: "Иркутск",
    9: "Якутск",
    10: "Владивосток",
    11: "Магадан",
    12: "Камчатка",
}
def offset_label(hours: int) -> str:
    sign = "+" if hours >= 0 else "-"
    return f"UTC{sign}{abs(hours)}"
def zone_title(hours: int) -> str:
    hint = OFFSET_HINTS.get(hours)
    return f"{hint} ({offset_label(hours)})" if hint else offset_label(hours)
def local_time(now_utc: datetime, hours: int) -> str:
    return (now_utc + timedelta(hours=hours)).strftime("%H:%M")
@dataclass(frozen=True)
class Prefs:
    tz_minutes: int = 0
    emoji: bool = False
    colors: bool = False
class SettingsService:
    def __init__(self, factory: sessionmaker):
        self._factory = factory
        self._cache: dict[int, Prefs] = {}
    def prefs(self, user_id: int) -> Prefs:
        cached = self._cache.get(user_id)
        if cached is not None:
            return cached
        with self._factory() as db:
            row = db.get(UserSetting, user_id)
            prefs = (
                Prefs(row.tz_offset_min, bool(row.emoji), bool(row.colors)) if row else Prefs()
            )
        self._cache[user_id] = prefs
        return prefs
    def tz_minutes(self, user_id: int) -> int:
        return self.prefs(user_id).tz_minutes
    def emoji_enabled(self, user_id: int) -> bool:
        return self.prefs(user_id).emoji
    def colors_enabled(self, user_id: int) -> bool:
        return self.prefs(user_id).colors
    def _update(self, user_id: int, **changes) -> None:
        with self._factory.begin() as db:
            row = db.get(UserSetting, user_id)
            if row is None:
                row = UserSetting(user_id=user_id, tz_offset_min=0, emoji=False, colors=False)
                db.add(row)
            for key, value in changes.items():
                setattr(row, key, value)
        self._cache.pop(user_id, None)
    def set_tz_hours(self, user_id: int, hours: int) -> None:
        if hours not in OFFSETS:
            raise ValueError("Недопустимый часовой пояс")
        self._update(user_id, tz_offset_min=hours * 60)
    def set_emoji(self, user_id: int, enabled: bool) -> None:
        self._update(user_id, emoji=bool(enabled))
    def set_colors(self, user_id: int, enabled: bool) -> None:
        self._update(user_id, colors=bool(enabled))
stats_service.py
from dataclasses import dataclass
from sqlalchemy import Float, case, cast, func, select
from sqlalchemy.orm import sessionmaker
from app.database.models import GameSession, MatchRound, RoundPlayer, SessionPlayer
MIN_GAMES_FOR_RATE = 5
SORTS = ("points", "wins", "spy", "rate")
@dataclass
class Profile:
    user_id: int
    name: str
    games: int
    wins: int
    civilian_wins: int
    spy_wins: int
    losses: int
    draws: int
    points: int
    lobbies: int
    @property
    def win_rate(self) -> float:
        return round(100 * self.wins / self.games, 1) if self.games else 0.0
@dataclass
class RatingRow:
    user_id: int
    name: str
    points: int
    games: int
    wins: int
    spy_wins: int
    @property
    def win_rate(self) -> float:
        return round(100 * self.wins / self.games, 1) if self.games else 0.0
def _wins():
    return func.sum(case((RoundPlayer.result == "win", 1), else_=0))
def _spy_wins():
    return func.sum(
        case(((RoundPlayer.role == "spy") & (RoundPlayer.result == "win"), 1), else_=0)
    )
def _real_rounds(stmt):
    return stmt.join(MatchRound, RoundPlayer.round_id == MatchRound.id).join(
        GameSession, MatchRound.session_id == GameSession.id
    ).where(GameSession.is_test.is_(False))
class StatsService:
```

```
    def __init__(self, factory: sessionmaker):
        self._factory = factory
    def profile(self, user_id: int) -> Profile | None:
        with self._factory() as db:
            row = db.execute(
                _real_rounds(
                    select(
                        func.count(),
                        _wins(),
                        _spy_wins(),

func.sum(case((RoundPlayer.result == "loss", 1), else_=0)),

func.sum(case((RoundPlayer.result == "draw", 1), else_=0)),

func.sum(RoundPlayer.score_delta),

func.max(RoundPlayer.name),
                    )
                ).where(RoundPlayer.user_id == user_id)
            ).one()
            games = row[0] or 0
            if games == 0:
                return None
            lobbies = db.execute(
                select(func.count())
                .select_from(SessionPlayer)
                .join(GameSession, SessionPlayer.session_id == GameSession.id)
                .where(SessionPlayer.user_id == user_id, GameSession.is_test.is_(False))
            ).scalar_one()
            wins = int(row[1] or 0)
            spy_wins = int(row[2] or 0)
            return Profile(
                user_id=user_id,
                name=row[6] or "Игрок",
                games=games,
                wins=wins,
                civilian_wins=wins - spy_wins,
                spy_wins=spy_wins,
                losses=int(row[3] or 0),
                draws=int(row[4] or 0),
                points=int(row[5] or 0),
                lobbies=lobbies,
            )
    def leaderboard(self, sort: str = "points", limit: int = 10) -> list[RatingRow]:
        if sort not in SORTS:
            sort = "points"
        points = func.sum(RoundPlayer.score_delta)
        games = func.count()
        wins = _wins()
        spy_wins = _spy_wins()
        stmt = _real_rounds(
            select(
                RoundPlayer.user_id,
                func.max(RoundPlayer.name),
                points,
                games,
                wins,
                spy_wins,
            )
        ).group_by(RoundPlayer.user_id)
        if sort == "points":
            stmt = stmt.order_by(points.desc(), wins.desc(), RoundPlayer.user_id)
        elif sort == "wins":
            stmt = stmt.order_by(wins.desc(), points.desc(), RoundPlayer.user_id)
        elif sort == "spy":
            stmt = stmt.order_by(spy_wins.desc(), points.desc(), RoundPlayer.user_id)
        else:
            rate = cast(wins, Float) / cast(games, Float)
            stmt = (
                stmt.having(games >= MIN_GAMES_FOR_RATE)
                .order_by(rate.desc(), games.desc(), RoundPlayer.user_id)
            )
        with self._factory() as db:
            return [
                RatingRow(
                    user_id=row[0],
                    name=row[1] or "Игрок",
                    points=int(row[2] or 0),
                    games=row[3],
                    wins=int(row[4] or 0),
                    spy_wins=int(row[5] or 0),
                )
                for row in db.execute(stmt.limit(limit))
            ]
__init__.py
admin_panel.py
import os
import sys
from dotenv import load_dotenv
from app.database.info import check_connection, ensure_mysql, mask_url
from app.database.migrate import run_migrations
from app.database.session import make_session_factory
from app.services.admin_service import AdminError, AdminService, parse_duration
DEFAULT_URL = "mysql+pymysql://spy:spy@localhost:3306/spy_game?charset=utf8mb4"
EXIT_WORDS = {"0", "q", "quit", "exit", "выход", "в"}
MENU = """
=== Админ-панель «Шпион» ===
1. Список блокировок
2. Заблокировать игрока
3. Разблокировать игрока
4. Информация об игроке
5. Список пользовательских наборов (поиск)
6. Удалить набор слов
7. Наборы с жалобами или скрытые
0. Выход
"""
def ask_user_id(ask, say) -> int | None:
    raw = ask("Telegram ID игрока (Enter, чтобы отменить): ").strip()
    if not raw:
        return None
    if not raw.isdigit() or int(raw) <= 0:
        say("ID должен быть положительным числом.")
        return None
    return int(raw)
def ask_yes(ask, prompt: str) -> bool:
    return ask(f"{prompt} (д/н): ").strip().lower() in ("д", "да", "y", "yes")
def show_bans(admin: AdminService, say) -> None:
    rows = admin.list_bans()
    if not rows:
        say("Активных блокировок нет.")
        return
    for row in rows:
        say(
            f"ID {row.user_id}, {row.name}: осталось {row.left_text}, "
            f"причина: {row.reason_text}, нарушений: {row.strikes}"
        )
def do_ban(admin: AdminService, ask, say) -> None:
    user_id = ask_user_id(ask, say)
    if user_id is None:
        return
    raw = ask("Срок (например 30, 2ч, 7д или навсегда): ")
    try:
        minutes = parse_duration(raw)
    except ValueError as err:
        say(str(err))
        return
    label = "навсегда" if minutes is None else raw.strip()
    if not ask_yes(ask, f"Заблокировать игрока {user_id} на срок: {label}?"):
        say("Отменено.")
        return
    admin.ban(user_id, minutes)
    say(f"Игрок {user_id} заблокирован: {label}.")
def do_unban(admin: AdminService, ask, say) -> None:
    user_id = ask_user_id(ask, say)
    if user_id is None:
        return
    info = admin.user_info(user_id)
    if info["ban_seconds"] <= 0 and info["strikes"] == 0:
        say("У этого игрока нет блокировки и нарушений.")
        return
    reset = ask_yes(ask, f"Нарушений: {info['strikes']}. Обнулить счётчик нарушений?")
    admin.unban(user_id, reset_strikes=reset)
    say(f"Игрок {user_id} разблокирован" + (", счётчик нарушений обнулён." if reset else "."))
def show_user(admin: AdminService, ask, say) -> None:
    user_id = ask_user_id(ask, say)
    if user_id is None:
        return
    info = admin.user_info(user_id)
    from app.services.penalty import ban_message
    status = (
        ban_message(info["ban_seconds"], info["ban_reason"])
        if info["ban_seconds"] > 0
        else "блокировки нет"
    )
    say(f"ID {info['user_id']}, имя: {info['name']}")
    say(f"Статус: {status}")
    say(f"Нарушений: {info['strikes']}, сыграно раундов: {info['rounds']}")
    say(f"Своих наборов: {info['packs']}, сейчас в комнате: {'да' if info['in_room'] else 'нет'}")
def list_packs(admin: AdminService, ask, say) -> None:
    query = ask("Поиск по названию, автору или номеру (Enter, чтобы показать все): ")
    packs, total = admin.list_packs(query, limit=25)
    if not packs:
        say("Ничего не найдено.")
        return
    for pack in packs:
        say(pack.line())
    if total > len(packs):
```

```
        say(f"Показано {len(packs)} из {total}. Уточните поиск.")
def do_delete_pack(admin: AdminService, ask, say) -> None:
    raw = ask("Номер набора (Enter, чтобы отменить): ").strip()
    if not raw:
        return
    if not raw.isdigit():
        say("Номер должен быть числом.")
        return
    try:
        pack = admin.get_pack(int(raw))
    except AdminError as err:
        say(str(err))
        return
    say(pack.line())
    if ask("Для подтверждения введите УДАЛИТЬ: ").strip() != "УДАЛИТЬ":
        say("Отменено.")
        return
    admin.delete_pack(pack.id)
    say(f"Набор «{pack.title}» удалён вместе со словами, избранным и жалобами.")
def show_reported(admin: AdminService, ask, say) -> None:
    packs = admin.reported_packs()
    if not packs:
        say("Наборов с жалобами и скрытых нет.")
        return
    for pack in packs:
        say(pack.line())
    raw = ask("Номер набора для действия (Enter, чтобы вернуться): ").strip()
    if not raw.isdigit():
        return
    action = ask("1 вернуть в публичные и снять жалобы, 2 скрыть, 3 удалить: ").strip()
    try:
        if action == "1":
            admin.set_hidden(int(raw), False)
            say("Набор возвращён, жалобы сняты.")
        elif action == "2":
            admin.set_hidden(int(raw), True)
            say("Набор скрыт.")
        elif action == "3":
            pack = admin.get_pack(int(raw))
            if ask("Для подтверждения введите УДАЛИТЬ: ").strip() == "УДАЛИТЬ":
                admin.delete_pack(pack.id)
                say(f"Набор «{pack.title}» удалён.")
            else:
                say("Отменено.")
        else:
            say("Неизвестное действие.")
    except AdminError as err:
        say(str(err))
ACTIONS = {
    "1": lambda a, ask, say: show_bans(a, say),
    "2": do_ban,
    "3": do_unban,
    "4": show_user,
    "5": list_packs,
    "6": do_delete_pack,
    "7": show_reported,
}
def run_menu(admin: AdminService, ask=input, say=print) -> None:
    while True:
        say(MENU)
        choice = ask("Выберите пункт: ").strip().lower()
        if choice in EXIT_WORDS:
            say("До свидания.")
            return
        action = ACTIONS.get(choice)
        if action is None:
            say("Нет такого пункта.")
            continue
        try:
            action(admin, ask, say)
        except AdminError as err:
            say(str(err))
def main() -> int:
    load_dotenv()
    url = os.getenv("DATABASE_URL", DEFAULT_URL)
    try:
        ensure_mysql(url)
        check_connection(url)
        run_migrations(url)
        factory = make_session_factory(url)
    except Exception as err:
        print(f"Не удалось подключиться к базе данных: {err}")
        print(f"Подключение: {mask_url(url)}")
        return 1
    print(f"База данных: {mask_url(url)}")
    run_menu(AdminService(factory))
    return 0
if __name__ == "__main__":
    sys.exit(main())
db_check.py
import os
import sys
from dotenv import load_dotenv
from app.database.info import check_connection, describe, ensure_mysql, format_info, mask_url
from app.database.session import make_session_factory
DEFAULT_URL = "mysql+pymysql://spy:spy@localhost:3306/spy_game?charset=utf8mb4"
def run(url: str, require_mysql: bool = True) -> str:
    if require_mysql:
        ensure_mysql(url)
    check_connection(url)
    factory = make_session_factory(url)
    return f"Подключение: {mask_url(url)}\n\n" + format_info(describe(factory, url))
def main() -> int:
    load_dotenv()
    url = os.getenv("DATABASE_URL", DEFAULT_URL)
    try:
        print(run(url))
    except Exception as err:
        print(f"Не удалось проверить базу данных: {err}")
        print(f"Подключение: {mask_url(url)}")
        print("Проверьте, что служба MySQL запущена, а логин и пароль в DATABASE_URL верны.")
        return 1
    return 0
if __name__ == "__main__":
    sys.exit(main())
export_schema.py
from pathlib import Path
from sqlalchemy.dialects import mysql
from sqlalchemy.schema import CreateIndex, CreateTable
from app.database.models import Base
OUTPUT = Path(__file__).resolve().parent / "schema.sql"
def build_schema() -> str:
    dialect = mysql.dialect()
    parts = [
        "-- Схема базы данных бота «Шпион» для MySQL 8.",
        "-- Файл только для справки: бот сам создаёт и обновляет таблицы при запуске (Alembic).",
        "-- Создан скриптом scripts/export_schema.py из app/database/models.py.",
        "",
    ]
    for table in Base.metadata.sorted_tables:

parts.append(str(CreateTable(table).compile(dialect=dialect)).strip() + " DEFAULT CHARSET=utf8mb4;")
        for index in sorted(table.indexes, key=lambda i: i.name or ""):

parts.append(str(CreateIndex(index).compile(dialect=dialect)).strip() + ";")
        parts.append("")
    return "\n".join(parts)
if __name__ == "__main__":
    OUTPUT.write_text(build_schema(), encoding="utf-8")
    print(f"Схема записана в {OUTPUT}")
admin.bat
@echo off
chcp 65001 >nul
set PYTHONIOENCODING=utf-8
cd /d "%~dp0"
if not exist "venv\Scripts\python.exe" (
    echo Не найдено виртуальное окружение venv.
    pause
    exit /b 1
)
"venv\Scripts\python.exe" -m scripts.admin_panel
pause
check_db.bat
@echo off
chcp 65001 >nul
cd /d "%~dp0"
if not exist "venv\Scripts\python.exe" (
    echo Не найдено виртуальное окружение venv.
    pause
    exit /b 1
)
"venv\Scripts\python.exe" -m scripts.db_check
pause
restart_bot.bat
@echo off
chcp 65001 >nul
cd /d "%~dp0"
echo Остановка бота...
call "%~dp0stop_bot.bat" nopause
timeout /t 2 >nul
if exist "venv\Scripts\python.exe" (
    echo Обновление зависимостей...
    "venv\Scripts\python.exe" -m pip install -q -r requirements.txt
)
echo Запуск бота...
call "%~dp0start_bot.bat" nopause
if errorlevel 1 pause
show_bot.bat
@echo off
chcp 65001 >nul
cd /d "%~dp0"
set "BOT_DIR=%~dp0"
```

```
powershell -NoProfile -Command "$d=$env:BOT_DIR; $p=Get-CimInstance Win32_Process | Where-Object { $_.CommandLine -like '*app.main*' -and $_.CommandLine -like ('*'+$d+'*') }; if($p){exit 1}else{exit 0}"
if not errorlevel 1 (
  echo Бот не запущен. Запустите его через start_bot.bat
  if not "%~1"=="nopause" pause
  exit /b 1
)
type nul > "%~dp0.window_show"
echo Команда отправлена. Окно бота появится через секунду.
timeout /t 2 >nul
start_bot.bat
@echo off
chcp 65001 >nul
cd /d "%~dp0"
set "BOT_DIR=%~dp0"
if not exist "venv\Scripts\python.exe" (
  echo Не найдено виртуальное окружение venv.
  echo Создайте его командой: python -m venv venv
  echo Затем установите зависимости: venv\Scripts\pip install -r requirements.txt
  if not "%~1"=="nopause" pause
  exit /b 1
)
if not exist ".env" (
  echo Не найден файл .env. Скопируйте .env.example в .env и впишите BOT_TOKEN.
  if not "%~1"=="nopause" pause
  exit /b 1
)
powershell -NoProfile -Command "$d=$env:BOT_DIR; $p=Get-CimInstance Win32_Process | Where-Object { $_.CommandLine -like '*app.main*' -and $_.CommandLine -like ('*'+$d+'*') }; if($p){exit 1}else{exit 0}"
if errorlevel 1 (
  echo Бот уже запущен. Для перезапуска используйте restart_bot.bat
  if not "%~1"=="nopause" pause
  exit /b 1
)
start "Spy Game Bot" conhost.exe cmd /k ""%~dp0venv\Scripts\python.exe" -m app.main"
echo Бот запущен в отдельном окне "Spy Game Bot".
echo В окне бота введите inv и нажмите Enter, чтобы скрыть его. Вернуть окно: show_bot.bat
timeout /t 3 >nul
stop_bot.bat
@echo off
chcp 65001 >nul
cd /d "%~dp0"
set "BOT_DIR=%~dp0"
powershell -NoProfile -Command "$d=$env:BOT_DIR; $p=Get-CimInstance Win32_Process | Where-Object { $_.CommandLine -like '*app.main*' -and $_.CommandLine -like ('*'+$d+'*') }; if($p){ $p | ForEach-Object { Stop-Process -Id $_.ProcessId -Force -ErrorAction SilentlyContinue }; exit 0 } else { exit 2 }"
set "RESULT=%errorlevel%"
taskkill /FI "WINDOWTITLE eq Spy Game Bot*" /F >nul 2>&1
if "%RESULT%"=="0" (
  echo Бот остановлен.
) else (
  echo Бот не запущен.
)
if not "%~1"=="nopause" pause
Dockerfile
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY alembic.ini .
COPY migrations ./migrations
COPY app ./app
HEALTHCHECK --interval=15s --timeout=5s --start-period=60s --retries=3 CMD python -m app.health
CMD ["python", "-m", "app.main"]
docker-compose.yml
services:
  db:
    image: mysql:8.4
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: spy_game
      MYSQL_USER: spy
      MYSQL_PASSWORD: spy
      MYSQL_ROOT_PASSWORD: root
    command: --character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci
    ports:
      - "127.0.0.1:3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-proot"]
      interval: 5s
      timeout: 5s
      retries: 20
  bot:
    build: .
    restart: unless-stopped
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "python", "-m", "app.health"]
      interval: 15s
      timeout: 5s
      retries: 3
      start_period: 60s
    env_file:
      - .env
    environment:
      DATABASE_URL: mysql+pymysql://spy:spy@db:3306/spy_game?charset=utf8mb4
volumes:
  mysql_data:
requirements.txt
aiogram==3.31.0
SQLAlchemy==2.0.54
PyMySQL==1.2.3
cryptography==50.0.2
python-dotenv==1.2.4
alembic==1.20.0
Mako==1.4.3
```

Приложение Б
Диаграммы UML

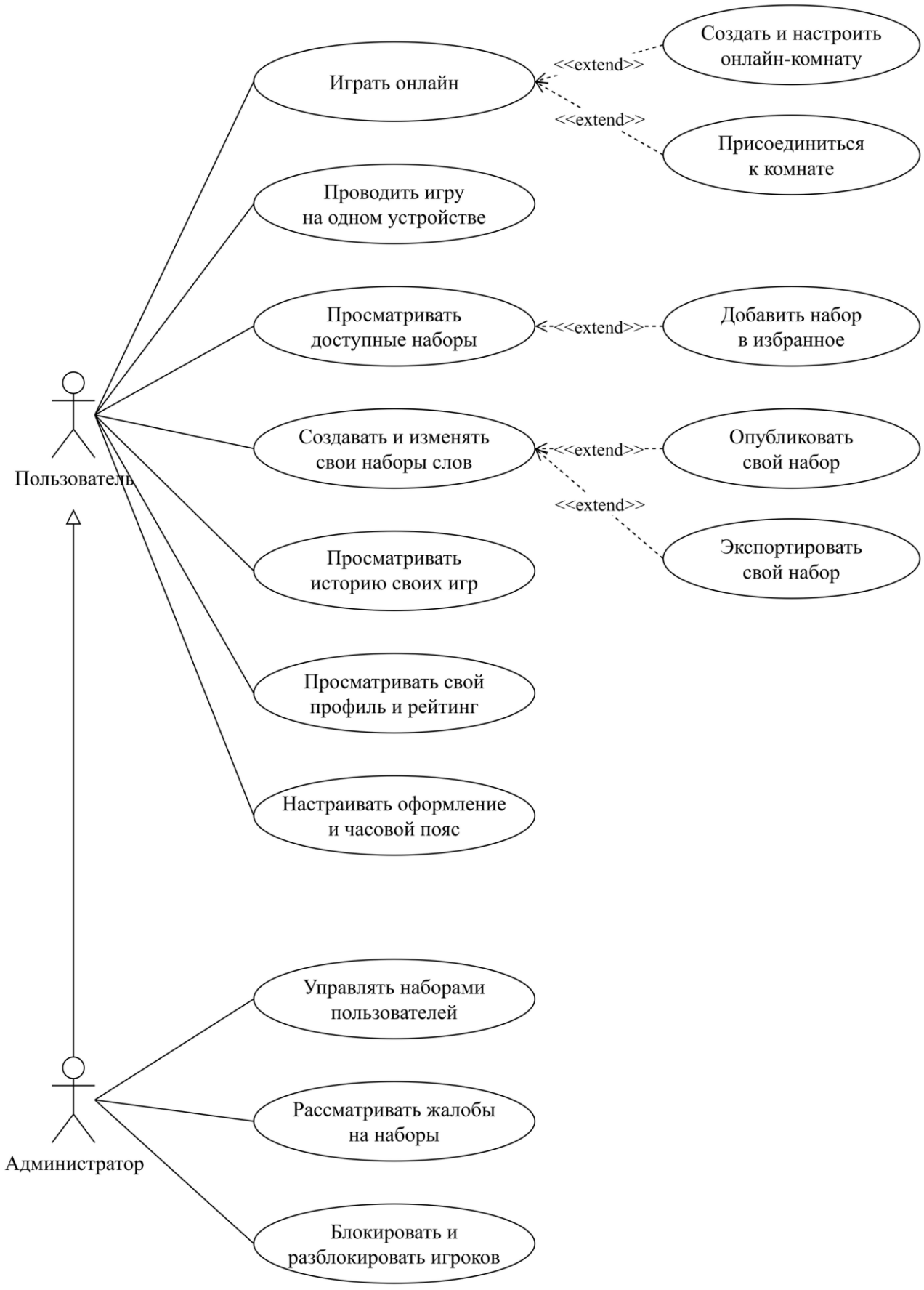


Рисунок П.Б.1 – Диаграмма вариантов использования

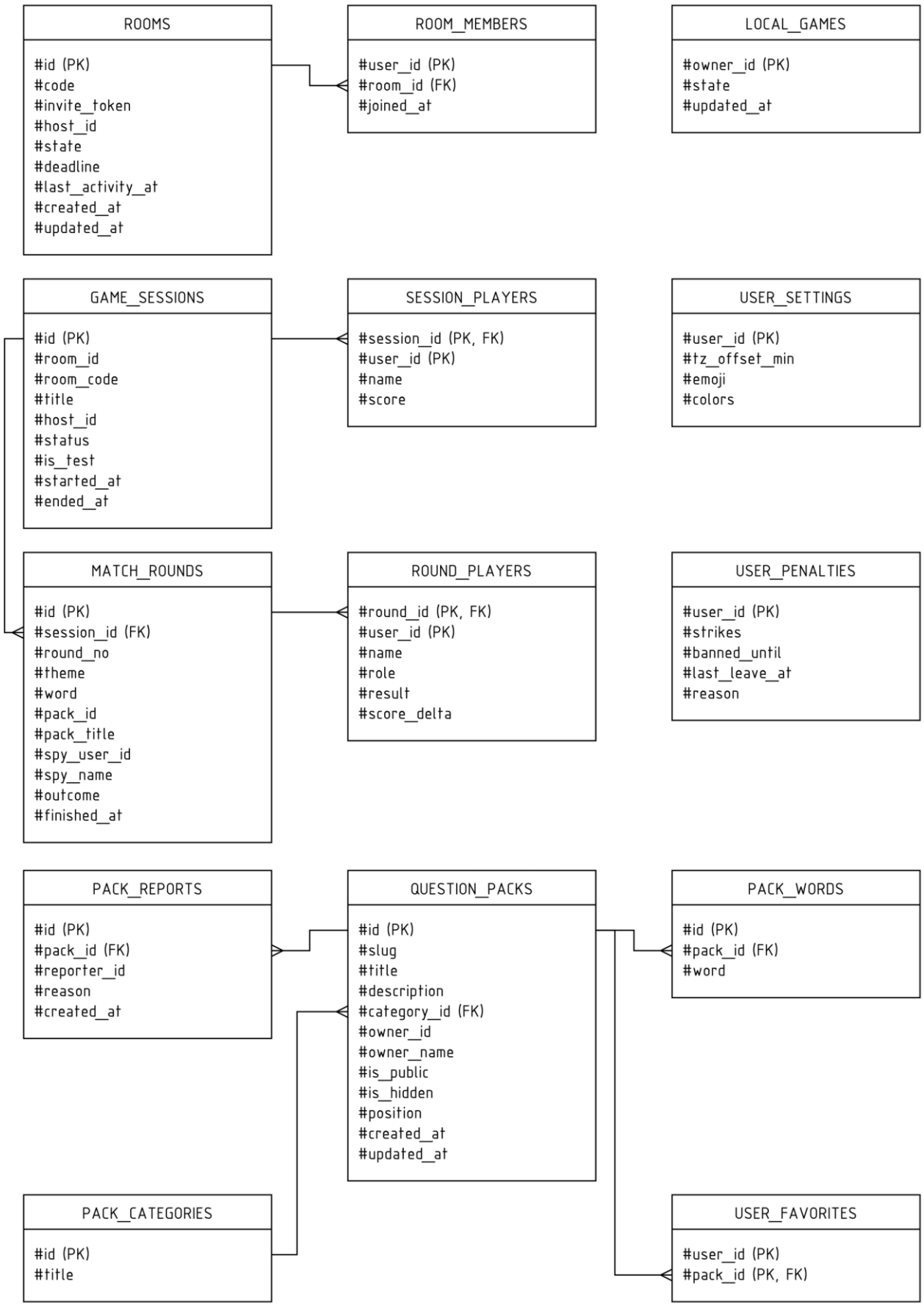


Рисунок П.Б.2 – Модель данных

Приложение В
Тест-кейсы

Таблица В.1 – Тест-кейсы

Идентификатор	Название	Шаги выполнения	Ожидаемый результат	Фактический результат	Итог тестирования
1	2	3	4	5	6
T_01	Открытие главного меню	1 Отправить /start обычным пользователем. 2 Открыть меню администратором.	Главная клавиатура доступна; «Админ» виден только администратору.	Меню открыто; административная кнопка показана по правам.	Успешно
T_02	Команды правил и настроек	1 Отправить /rules. 2 Отправить /settings.	Открываются правила и экран настроек.	Правила и настройки открыты.	Успешно
T_03	Вход по коду комнаты	1 Создать комнату; передать её код второму игроку; выполнить вход.	Игрок присоединён к существующей комнате.	Игрок вошёл в существующую комнату.	Успешно
T_04	Создание и приглашение	1 Создать комнату; присоединить участников по коду и токену.	Оба способа входа относятся к одной комнате.	Код и ссылка ведут в одну комнату.	Успешно
T_05	Ссылка закрытой комнаты	1 Закрыть лобби; открыть сохранённое приглашение.	Сообщается о неактивной комнате, новое меню доступно.	Показано сообщение о неактивной комнате.	Успешно
T_06	Копирование приглашения	1 Открыть лобби; сравнить прямую ссылку и кнопку копирования.	Используется один и тот же токен приглашения.	Ссылка соответствует приглашению комнаты.	Успешно
T_07	Отмена ввода кода	1 Открыть ввод кода; отменить действие; закрыть экран.	Состояние ввода очищено, навигация доступна.	Ввод отменён; меню доступно.	Успешно
T_08	Перебор приглашений	1 Выполнить серию неверных попыток входа.	Применено ограничение, затем возможность входа восстанавливается.	Применено временное ограничение входа.	Успешно

Продолжение таблицы В.1

1	2	3	4	5	6
Т_09	Формат входного кода	1 Передать недопустимое значение кода.	Некорректный ввод отклонён до запроса к БД.	Недопустимый код отклонён.	Успешно
Т_10	Условия запуска партии	1 Проверить запуск без темы и готовности. 2 Собрать трёх участников; выбрать тему и подтвердить готовность.	Запуск доступен при выбранной теме и готовности трёх участников.	Условия запуска соблюдены; начата раздача ролей.	Успешно
Т_11	Права ведущего комнаты	1 Войти обычным участником. 2 Попытаться изменить параметры и запустить партию.	Управление параметрами и запуск доступны ведущему.	Действия обычного участника отклонены.	Успешно
Т_12	Подтверждение выхода	1 Нажать «Выйти». 2 Проверить членство до подтверждения. 3 Подтвердить выход.	Выход выполняется после подтверждения; панель комнаты удаляется.	Участник вышел после подтверждения; панель удалена.	Успешно
Т_13	Начало локальной партии	1 Открыть «Один телефон»; задать имена и пройти подготовку.	Партия создаётся через нижнюю клавиатуру.	Открыта подготовка локальной игры.	Успешно
Т_14	Продолжение локальной партии	1 Открыть локальный режим повторно после сохранения.	Предлагается продолжить сохранённую игру.	Показано предложение продолжить игру.	Успешно
Т_15	Скрытые сведения локально	1 Просмотреть публичные экраны во время раунда.	Роль и слово не появляются на общих экранах.	Секретные сведения отсутствуют на общем экране.	Успешно
Т_16	Верное угадывание онлайн до голосования	1 Шпион в онлайн-комнате вводит верное слово до голосования.	Шпион получает победу и три очка.	Победил шпион; начислено три очка.	Успешно
Т_17	Неверное угадывание онлайн до голосования	1 Шпион в онлайн-комнате вводит другое разрешённое слово.	Побеждают обычные игроки по одному очку.	Победили игроки; каждому начислено одно очко.	Успешно

Продолжение таблицы В.1

1	2	3	4	5	6
Т_18	Верное угадывание онлайн при голосовании	1 Во время голосования шпион вводит верное слово.	Фиксируется ничья, очки не начисляются.	Объявлена ничья; очки не начислены.	Успешно
Т_19	Неверное угадывание онлайн при голосовании	1 Во время голосования шпион ошибается.	Попытка потрачена, голосование продолжается.	Попытка потрачена; голосование продолжено.	Успешно
Т_20	Угадывание обычным игроком онлайн	1 Обычный участник онлайн-комнаты пытается угадать слово.	Операция недоступна, исход не изменяется.	Обычному игроку угадывание недоступно.	Успешно
Т_21	Однократное завершение	1 Смоделировать почти одновременные угадывание и голос.	Раунд завершается и учитывается однократно.	Раунд завершён и учтён один раз.	Успешно
Т_22	Таймер онлайн-хода	1 Начать обсуждение в онлайн-комнате; дождаться истечения таймера.	Очередность продвигается по таймеру.	Ход передан следующему игроку.	Успешно
Т_23	Шпион найден голосованием	1 Начать голосование. 2 Большинством голосов выбрать шпиона.	Раунд завершён; обычные игроки получают по одному очку.	Победили обычные игроки; очки начислены.	Успешно
Т_24	Ошибка голосования и повторная ничья	1 Выбрать обычного игрока. 2 В другом раунде получить повторную ничью.	В обоих случаях побеждает шпион и получает три очка.	Победил шпион; начислено три очка.	Успешно
Т_25	Старый игровой экран	1 Нажать навигационную кнопку на старой панели.	Старое действие не заменяет активную игровую панель.	Старая кнопка не изменила текущую панель.	Успешно
Т_26	Панель комнаты и разделы	1 Открыть раздел меню во время онлайн-игры; вернуться.	Панель комнаты сохранена либо корректно заменена при возврате.	Панель комнаты сохранена при переходах.	Успешно

Продолжение таблицы В.1

1	2	3	4	5	6
T_27	Локальная панель и разделы	1 Открыть меню во время локальной игры.	Сохранённый игровой экран не удалён.	Экран локальной игры сохранён.	Успешно
T_28	Нижняя клавиатура	1 Открывать и закрывать разделы меню.	Постоянная клавиатура остаётся доступной.	Нижняя клавиатура остаётся доступной.	Успешно
T_29	Изменение часового пояса	1 Выбрать пояс; вернуться в настройки.	Экран актуален, лишнее сообщение не создаётся.	Часовой пояс сохранён; настройки открыты.	Успешно
T_30	Права администратора	1 Указать похожий username без верного числового ID.	Административные права не выдаются.	Права выданы только настроенному ID.	Успешно
T_31	Подтверждение блокировки	1 Открыть карточку игрока; попытаться заблокировать себя.	Для бана требуется подтверждение, администратор защищён.	Блокировка требует подтверждения; самоблокировка запрещена.	Успешно
T_32	Приватность набора	1 Создать личный набор; запросить его от другого игрока; опубликовать.	Чужой личный набор скрыт, публичный доступен.	Чужой личный набор скрыт; публичный доступен.	Успешно
T_33	Минимум публичного набора	1 Опубликовать десять слов; попытаться удалить одно; сделать набор личным.	Нарушение минимума блокируется, личный набор можно уменьшить.	Нарушение минимума слов заблокировано.	Успешно
T_34	Удалённая тема лобби	1 Выбрать пользовательский набор; удалить его; повторно открыть комнату.	Нет KeyError, отсутствующая тема исключена.	Лобби открыто; удалённая тема исключена.	Успешно
T_35	Сохранение раунда без набора	1 Начать раунд; удалить источник темы допустимым путём.	Снимок действующего раунда не теряет слово.	Слово и результат начатого раунда сохранены.	Успешно

Продолжение таблицы В.1

1	2	3	4	5	6
Т_36	Очистка заброшенной партии	1 Смоделировать отсутствие игроков и техническое срабатывание таймера.	Техническое обновление не препятствует очистке комнаты.	Заброшенная комната удалена.	Успешно
Т_37	Продление реальным действием	1 Выполнить принятое действие игрока.	Обновляется время активности комнаты.	Время активности обновлено действием игрока.	Успешно
Т_38	Создание и экспорт набора	1 Создать набор через диалог; выполнить экспорт.	Набор и экспорт содержат сохранённые слова.	Набор создан и выгружен в файл.	Успешно
Т_39	Поиск набора	1 Открыть поиск; ввести название набора.	Отображаются найденные наборы.	Найден набор, соответствующий запросу.	Успешно
Т_40	Копирование набора	1 Скопировать доступный набор.	Создана личная копия.	Создана личная копия набора.	Успешно
Т_41	Удаление слов вместе с набором	1 Удалить собственный набор.	Связанные слова удалены.	Набор и связанные слова удалены.	Успешно
Т_43	Набор в онлайн-игре	1 Выбрать свой набор; начать раунд.	Используется слово выбранного набора.	В раунде использован выбранный набор.	Успешно
Т_44	Чужой личный набор	1 Попытаться выбрать чужой личный набор.	Набор недоступен для выбора.	Чужой личный набор не выбран.	Успешно
Т_45	Название набора в истории	1 Начать раунд; изменить название набора; завершить.	История хранит название начала раунда.	История хранит название начала раунда.	Успешно
Т_46	Проверка названия и лимита	1 Передать недопустимое название; превысить лимит.	Недопустимые операции отклонены.	Недопустимое название и превышение лимита отклонены.	Успешно

Продолжение таблицы В.1

1	2	3	4	5	6
Т_47	Статистика профиля	1 Сохранить результаты; запросить профиль.	Статистика соответствует результатам.	Статистика соответствует результатам игр.	Успешно
Т_48	Доступ к истории	1 Запросить историю от другого пользователя.	Действуют ограничения доступа.	Ограничения доступа к истории соблюдены.	Успешно
Т_49	Закрытие экрана	1 Открыть раздел; нажать «Закрыть».	Текущий экран закрыт.	Текущий экран закрыт.	Успешно
Т_50	Эмодзи на кнопках	1 Выключить эмодзи; сформировать кнопки настроек.	Текст кнопок не содержит необязательных эмодзи.	На кнопках нет необязательных эмодзи.	Успешно